HE-based On-the-Fly MPC, Revisited: Universal Composability, Approximate and Imperfect Computation, Circuit Privacy

Ganyuan Cao[1], Sylvain Chatel[2] and Christian Knabenhans[3]

[1] Télécom Paris, Institut Polytechnique de Paris, Palaiseau, France
ganyuan.cao@telecom-paris.fr
[2] CISPA, Helmholtz Center for Information Security, Saarbrücken, Germany
sylvain.chatel@cispa.de
[3] EPFL, Lausanne, Switzerland
christian.knabenhans@epfl.ch

Abstract. On-the-fly multi-party computation (OtF-MPC, López-Alt, Tromer, and Vaikuntanathan, STOC 2012) allows clients to join a computation dynamically without remaining online, while outsourcing the computation to an untrusted but powerful server. Existing OtF-MPC constructions rely on composing multi-party homomorphic encryption with zero-knowledge or succinct arguments. While both components have seen substantial efficiency and expressivity improvements in recent years, the security of their composition has not been revisited under these modern refinements.

In this work, we revisit OtF-MPC through the lens of composable security. We extend the original analysis to systematically capture recent advances in these two building blocks, and to reason about their secure composition. We focus on multi-group homomorphic encryption (MGHE), a unifying abstraction that generalizes threshold and multi-key HE, and emerges as the natural backbone for OtF-MPC.

Our contributions are fourfold. (1) We introduce the first Universal Composability (UC) ideal functionality for MGHE, jointly modeling client and server privacy, approximate or imperfect computation, and threshold security. (2) We define fine-grained security notions for MGHE and show that they are sufficient to realize this functionality against semi-malicious adversaries. (3) We revisit generic compilers that combine MGHE with any-simulation-extractable zkSNARKs, and prove that they realize OtF-MPC against fully malicious adversaries. (4) We introduce a split-witness ideal functionality for NIZKs, which separates the witness component that must be supplied by the ideal adversary from an existential component that only needs to be certified. We show that a Naor–Yung-style composition of MGHE and split-witness NIZKs realizes OtF-MPC while avoiding full extraction from the NIZK.

Keywords: On-the-Fly MPC · Multi-Group Homomorphic Encryption (MGHE) · zkSNARK/NIZK · Universal Composability (UC).

1 Introduction

Secure multi-party computation (MPC) enables mutually distrusting parties to jointly compute a function over their private inputs while keeping these inputs secret. *On-the-fly* MPC [LTV12] is a variant in which a *client* $\mathcal{P}_{\mathsf{cli}}$ can dynamically join a specific computation task without being constantly online, and the computation is delegated to an untrusted but powerful *server* $\mathcal{P}_{\mathsf{srv}}$. Thanks to this flexibility, on-the-fly MPC serves as a generalized framework for numerous real-world applications [JNO16, SVdV16, TC16]. Its canonical construction is given by compilers that combine a *multi-key* (MK) [LTV12, MW16] or *threshold* (Th) [AJL+12, HHK+25] *homomorphic encryption* (HE) scheme, enabling computation over data encrypted under different keys or in a threshold setting, with a *non-interactive (succinct) zero-knowledge argument (of knowledge)* (NIZK/zkSNARK) proving the validity of the computation.

Meanwhile, multi-party HE has become considerably more efficient and scalable: the recent NIST call for multi-party threshold schemes [BP25] devotes a section to FHE-based solutions, and several proposals combine multi-party FHE with SNARGs for active security [BBC+26, AABA+26]. Separately, the security of NIZKs and zkSNARKs is now much better understood, particularly regarding composability and nuanced simulation-security notions. This leads to a paradoxical situation: many applications of on-the-fly MPC are now concretely efficient and on the verge of deployment, yet the foundational compiler and its analysis remain inadequate for virtually all of them, failing to give provable security guarantees.

Table 1: A comparison of existing compilers for (on-the-fly) MPC from HE. ●: discussed and analyzed in detail; ◎: informally analyzed or with gaps in the security proof; ○: not considered. Specifically, [AJL+12, MW16] does not consider the simulation security of NIZK, which is necessary for UC security. [MTBH21, HHK+25] primarily discussed in the context of (n, n)-thresholds, but mention that their schemes can be extended to (t, n)-threshold via secret sharing.

	Imperfect/Approx. HE	Circuit Privacy	(t, n)-Threshold	Composability
[LTV12]	○	○	○	○
[AJL+12]	●	○	●	◎
[MW16]	○	○	○	◎
[MTBH21]	◎	○	◎	○
[HHK+25]	●	○	◎	○
This Work	●	●	●	●

In this work, we revisit HE-based on-the-fly MPC as a compiler from (multi-party) HE and NIZKs/zkSNARKs, extending it to support the optimized schemes that have emerged from over a decade of advances in both primitives. Developing a modern analogue of these compilers is challenging for three reasons. 1) Existing protocols are often claimed secure against a *semi-malicious* adversary [AJL+12],

who may adaptively choose randomness during execution, while the underlying HE security notions they rely on are defined only for *semi-honest* adversaries, which is a mismatch between assumed adversarial capabilities and the guarantees actually provided. 2) Many works introduce ad hoc notions tailored to specific attacks, leaving unclear which are appropriate or sufficient for systematically constructing on-the-fly MPC. 3) The security of composing multiple (often orthogonal) optimizations and primitives is not always well understood. In Table 1, we summarize existing compilers building on-the-fly MPC from (multi-party) HE and the limitations of their security analyses; we state our main results in Section 1.1 and overview our techniques in Sections 2 and 3.

Challenge 1: Approximate Computation and Imperfect Correctness. Many modern on-the-fly MPC applications leverage HE with *approximate arithmetic* [CKKS17] for efficiency, e.g., machine learning tasks, and state-of-the-art implementations often accept *imperfect correctness* [HS15], trading a small decryption-failure probability for better efficiency. Such schemes are susceptible to key-recovery attacks [LM21, CCP+24, LWF25], and while many security notions have been proposed in response [LM21, CSBB24b, CCP+24, BCD+25, BJSW25, LWF25], their relationships remain underexplored and it is unclear which suffice for on-the-fly MPC.

Challenge 2: Server Inputs and Circuit Privacy. Practical HE-based MPC protocols often delegate additional work to a *helper* server, e.g., evaluating a pre-trained model whose weights are known only to the server. Yet security analyses typically focus on client privacy against a corrupted server, whereas in real-world applications protecting the server's circuit [IP07, Gen09a, BdPMW16] or its private inputs [KVMGH24] is equally important.

Challenge 3: Groupwise Threshold Setting. While multi-key HE (MKHE) [LTV12] allows computation across independent keys, the emerging primitive of multi-*group* HE (MGHE) [AH19, Par21, CMS+23, KLSW24, CCKY25] unifies it with *threshold* HE (ThHE) [AJL+12, BGG+18, BS23], organizing participants into groups that act as individual logical parties in a multi-key framework. This mirrors hierarchical trust structures in federated or organizational settings [SBTP+22, XHX+22] and is among the approaches envisioned in the recent NIST threshold call [BP25]. We focus on MGHE as it retains the cross-entity flexibility of MKHE while leveraging the internal efficiency of ThHE, yet its security landscape remains fragmented. keys.

Challenge 4: Composability. Challenges 1–3 each call for a dedicated analysis, yet our aim is to capture all of them in a single functionality modeling (multi-group) HE. Moreover, MPC security is formalized in the *simulation* paradigm [CD05], as indistinguishability between a real execution and an ideal functionality, whereas the guarantees its compilers consume from the underlying HE are generally *game-based*, so the two must be bridged by hand in each analysis. The same tension appears when lifting (multi-party) HE to malicious security, which typically composes it with NIZKs/zkSNARKs proving honest computation [LTV12, MN24]: recent work [GKO+23, CF24] has significantly advanced the understanding of simulation-security for these prim-

itives and of the conditions for secure composition, but existing compilers [LTV12, AJL+12, MW16, MTBH21, HHK+25] still rely on older assumptions, leaving a gap in their analyses. Most of them [LTV12, MTBH21, HHK+25] also provide only *single-instance* security, covering neither concurrent executions of on-the-fly MPC nor co-execution with other protocols, which is a severe limitation, since such applications are typically deployed within larger systems, e.g., as part of blockchain execution.

1.1 Our Results

UC-Secure Multi-Group HE. Of independent interest, we introduce an ideal functionality $\mathcal{F}_{\mathsf{MGHE}}$ for multi-group HE (Fig. 1) in the *Universal Composability* (UC) framework [Can00]. To the best of our knowledge, $\mathcal{F}_{\mathsf{MGHE}}$ is the *first* ideal functionality for multi-group HE that simultaneously captures approximate and imperfect computations, client-side and server-side privacy, and security in a group-wise t-out-of-n threshold setting, against both a *semi-malicious* adversary [AJL+12], who may choose randomness adaptively during protocol execution, and a full malicious adversary. Throughout the execution, we assume the adversary corrupts a subset of clients or the server *non-adaptively*.

We then identify the following sufficient conditions under which an MGHE scheme UC-realizes $\mathcal{F}_{\mathsf{MGHE}}$ against a *semi-malicious* adversary.

- *Client Privacy*: IND-CPA^{pD} security (Fig. 3 and Definition 2), a notion that we introduce to generalize and strictly strengthen existing definitions from [LM21, BCD+25, BJSW25, LWF25]. IND-CPA^{pD} captures client privacy when semi-malicious clients and servers may co-exist. In Table 2, we summarize the existing patches that can be applied on concrete MGHE instantiations e.g., [KLSW24] and propose a plausible route to achieving IND-CPA^{pD} security against semi-malicious adversaries by combining these patches together.
- *Server Privacy*: simulation-based circuit privacy SIM-CIRC from [BdPMW16] for exact and perfect MGHE, where the circuit topology may be revealed to the adversary; and game-based circuit privacy IND-CIRC (Fig. 4 and Definition 4), which we extend from [KS25] to the multi-group setting in order to handle approximate computation and imperfect correctness.
- *Threshold Security*: since MGHE uses distributed decryption, where clients first publish *decryption shares* that are then combined to recover the final message, we require:
  (i) SIM-PDEC [BGG+18] (Definition 5), which guarantees that decryption shares are simulatable and leak no information about clients' secret shares. This notion, however, does not cover the bootstrapping attack as in [CCP+24]. Therefore, IND-CPA^{pD} security we introduce is not redundant and has to be guaranteed along with SIM-PDEC.
  (ii) A new *robustness* notion (Fig. 5 and Definition 6), extending [BGG+18, BS23] to ensure correctness of distributed decryption for *evaluated* ciphertexts, even when the adversary can inject maliciously generated decryption shares.

UC-Secure On-the-Fly MPC. Having established UC security for MGHE, we turn to on-the-fly MPC, for which we define the UC functionality $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ (Fig. 9, Section 3.1) and realize it in three adversarial settings.

- *Semi-Malicious Security* (Section 3.2 and Theorem 3). The conclusions of prior works [LTV12, MW16, KLSW24] extend directly: a UC-secure MGHE scheme immediately yields a UC-secure on-the-fly MPC protocol against a semi-malicious adversary that *non-adaptively* corrupts the parties. We revisit this from the perspective of composable security, realizing $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ in the $\mathcal{F}_{\mathsf{MGHE}}$-hybrid model.
- *Malicious Security from Extractability* (Section 3.3, Theorem 4). Our first compiler employs a zero-knowledge *argument of knowledge*, for which we show that *(straightline black-box) any-simulation-extractability* [DHLW10] is required for UC security, by adapting the ideal functionality of [CF24]. In particular, since $\mathcal{F}_{\mathsf{MGHE}}$'s simulator only learns the message length, simulated ciphertexts need not satisfy the relation: proofs must still be accepted for uncorrupted clients, while knowledge soundness via extraction must still hold for corrupted ones.
- *Malicious Security from Naor–Yung Paradigm* (Section 3.4, Theorem 5). Our second compiler follows Naor–Yung [NY90] with a zero-knowledge proof, as also used for IND-CCA1 and non-malleable HE schemes [BSW12, CRRV17]. Although less efficient, requiring twice as many encryptions, evaluations, and decryptions, it avoids an argument of knowledge for the "full" witness. Of independent interest, we introduce a *split-witness* ideal functionality $\mathcal{F}_{\mathsf{NIZK}}$ (Fig. 11, Section 3.4.1), inspired by [GOS06, CKS11], separating a knowledge component supplied by the ideal-process adversary from an existential component that only needs to "exist". We show that this relaxation remains sound thanks to the Naor–Yung structure, whose double-encryption relation binds the extracted plaintext to both ciphertext branches, so that the existential component is not free-standing.

2 Multi-Group Homomorphic Encryption

Our first building block is *Multi-Group Homomorphic Encryption* (MGHE), whose name is first given in [KLSW24], yet it has also been explored in many prior works [AH19, Par21, CMS+23, CCKY25] due to its nature that generalizes both multi-key and threshold homomorphic encryption. Parties in MGHE are split into groups. Parties within a group behave as a single entity in a classic multi-key HE scheme. Inside the group, a threshold decryption is carried out for a ciphertexts that is intended for the group. We define the interface of an MGHE scheme as follows.

Definition 1 (Multi-Group Homomorphic Encryption). *A* Multi-Group Homomorphic Encryption *scheme is a tuple of probabilistic polynomial-time*

(PPT) algorithms $\mathsf{MGHE} = (\mathtt{PGen}, \mathtt{KGen}, \mathtt{KJoin}, \mathtt{Enc}, \mathtt{Eval}, \mathtt{PDec}, \mathtt{Combine})$[4] *defined as follows:*

- $\mathsf{pp} \leftarrow \mathtt{PGen}(1^\lambda, 1^\kappa)$*: On input the security parameter* λ *and the bound* κ *on the circuit depth, the algorithm outputs public parameters* pp*.*
- $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathtt{KGen}(\mathsf{pp})$*: On input the public parameter* pp*, the algorithm outputs a key pair* $(\mathsf{pk}, \mathsf{sk})$*.*
- $\mathsf{jpk} \leftarrow \mathtt{KJoin}$[5]$((\mathsf{pk}_i, \mathsf{sk}_i)_{i\in I})$*: On input the key pairs* $(\mathsf{pk}_i, \mathsf{sk}_i)_{i\in I}$ *of parties indexed by* I*, the distributed protocol outputs a joint public key* jpk*.*
- $c \leftarrow \mathtt{Enc}(\mathsf{jpk}, m)$*: On input the joint public key* jpk *and a message* m*, the algorithm outputs a ciphertext* c*.*
- $\widehat{c} \leftarrow \mathtt{Eval}(f, (\mathsf{jpk}_j, c_j)_{j\in[\ell]})$*: On input a circuit* f *of depth at most* κ*, ciphertexts* $c_1, \ldots, c_\ell$*, and their associated joint public keys* $\mathsf{jpk}_1, \ldots, \mathsf{jpk}_\ell$*, the algorithm outputs an evaluated ciphertext* $\widehat{c}$*.*
- $m \leftarrow \mathtt{DistDec}(\{\mathsf{sk}_i\}_{i\in I}, c)$*: On input a ciphertext* c *and the secret keys* $\{\mathsf{sk}_i\}_{i\in I}$ *of parties in* I*, the distributed protocol outputs the decrypted message* m*. This is realized in two steps:*
  - $d \leftarrow \mathtt{PDec}(\mathsf{sk}_i, c)$*: On input a ciphertext* c *and the secret key* sk_i *of a party, the algorithm outputs the decryption share* d*.*
  - $m \leftarrow \mathtt{Combine}(c, \{d_i\}_{i\in I})$*: On input a ciphertext* c *and the decryption shares* $\{d_i\}_{i\in I}$ *of* c*, the algorithm outputs the message* m*.*

2.1 Functionality $\mathcal{F}_{\mathsf{MGHE}}$

We define an ideal functionality $\mathcal{F}_{\mathsf{MGHE}}$ for multi-group HE in the *Universal Composability* (UC) framework [Can00], which we show in Fig. 1. At a high level, $\mathcal{F}_{\mathsf{MGHE}}$ proceeds in three phases:

- *Input Phase*: a client $\mathcal{P}_{\mathsf{cli}}$ sends a message m to $\mathcal{F}_{\mathsf{MGHE}}$. Depending on the corruption state of the client and its group, $\mathcal{F}_{\mathsf{MGHE}}$ reveals either m or its length $|m|$ to the (ideal-process) adversary $\mathcal{S}$, who then simulates the corresponding ciphertext c.
- *Computation Phase*: the server $\mathcal{P}_{\mathsf{srv}}$ sends a circuit f to $\mathcal{F}_{\mathsf{MGHE}}$. The functionality asks $\mathcal{S}$ to simulate the evaluation of f on the ciphertexts provided by the clients. If the server is not corrupted, only the circuit depth $|f|$ is provided to $\mathcal{S}$ to model circuit privacy. We note that we also model *multi-hop* evaluation in this functionality where an evaluated result can further serve as the input of the next evaluation.

[4] Throughout the remainder of this work, unless an algorithm explicitly specifies a random coin (e.g., $\mathsf{MGHE}.\mathtt{Enc}(\mathsf{jpk}, m; \omega)$), it is assumed that the random coin is sampled honestly according to the distribution prescribed by the scheme.

[5] In our syntax, $\mathtt{KJoin}(\cdot)$ takes both public and secret keys as input, since in most existing works [AJL+12, Par21, MTPBH21], generating a group's public *evaluation key* requires access to the parties' secret keys and thus must be implemented via MPC. While Kwak et al. [KLSW24] recently showed that this step can be made fully *non-interactive*, we adopt the more conservative syntax for generality.

Functionality: $\mathcal{F}_{\mathsf{MGHE}}$

Participants: n clients $\mathcal{P}_{\mathsf{cli}_1}, \mathcal{P}_{\mathsf{cli}_2}, \ldots, \mathcal{P}_{\mathsf{cli}_n}$; Initially, subsets of clients $\{\mathcal{P}_{\mathsf{cli}_i}\}_{i\in I}$ with $I \subseteq [n]$ form k groups $\mathcal{P}_{\mathsf{grp}_j}$ with $\mathsf{grp}_j = \{\mathsf{cli}_i\}_{i\in I_j}$ with $I_j \subseteq I$ for $j \in [k]$; a server $\mathcal{P}_{\mathsf{srv}}$; an ideal-process adversary $\mathcal{S}$.

– $\langle \texttt{Encrypt} : \mathsf{grp}, \mathsf{sid}, \mathsf{jpk}, m ; \boxed{\tilde{\omega}} \rangle$ from $\mathcal{P}_{\mathsf{cli}}$:
1. Retrieve jpk' from $\mathcal{G}_{\mathsf{KRK}}$. If $\mathsf{jpk} \neq \mathsf{jpk}'$ and $\mathsf{cli} \notin \mathsf{Corrupt}$, abort.
2. If fewer than $\lceil \frac{|\mathsf{grp}|}{t} \rceil$ clients in grp are corrupted and $\mathsf{cli} \notin \mathsf{Corrupt}$, output $\langle \texttt{Encrypt} : \mathsf{grp}, \mathsf{sid}, \mathsf{jpk}, |m| \rangle$ to $\mathcal{S}$.
3. Else if $\mathsf{cli} \in \mathsf{Corrupt}$, output $\langle \texttt{Encrypt} : \mathsf{grp}, \mathsf{sid}, \mathsf{jpk}, m ; \boxed{\tilde{\omega}} \rangle$ to $\mathcal{S}$.
4. Otherwise, output $\langle \texttt{Encrypt} : \mathsf{grp}, \mathsf{sid}, \mathsf{jpk}, m \rangle$ to $\mathcal{S}$.
5. When $\mathcal{S}$ returns $(c ; \omega, \boxed{\mathtt{hon}})$:
   a. Abort if c has been recorded.
   b. Set $\delta := \mathsf{GetEncNoise}(\omega)$. //Track noise from encryption randomness.
   c. Record $(m ; \delta, c)$ for $(\mathsf{grp}, \mathsf{sid})$ if $\mathsf{cli} \notin \mathsf{Corrupt} \vee (\mathsf{cli} \in \mathsf{Corrupt} \wedge \mathtt{hon} = 1)$.
   d. Record $(m ; \omega, c)$ for $(\mathsf{grp}, \mathsf{sid})$ if $\mathsf{cli} \notin \mathsf{Corrupt} \vee (\mathsf{cli} \in \mathsf{Corrupt} \wedge \mathtt{hon} = 1)$.
6. Output $\{\texttt{"ct"} : \mathsf{grp}, \mathsf{sid}, c\}$ to $\mathcal{P}_{\mathsf{cli}}$.

– $\langle \texttt{Evaluate} : \mathsf{sid}, f, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{jpk}_j, c_j)_{j\in[\ell]} ; \boxed{\tilde{\sigma}} \rangle$ from $\mathcal{P}_{\mathsf{srv}}$:
1. For each grp_j, retrieve jpk'_j from $\mathcal{G}_{\mathsf{KRK}}$. If $\mathsf{jpk}_j \neq \mathsf{jpk}'_j$ for some $j \in [\ell]$ and $\mathsf{srv} \notin \mathsf{Corrupt}$, abort.
2. If $\mathsf{sid} \neq \mathsf{sid}_j$ for any $j \in [\ell]$, abort.
3. Set $\widehat{\mathsf{grp}} = \cup_{j\in[\ell]} \mathsf{grp}_j$.
4. If $\mathsf{srv} \notin \mathsf{Corrupt}$:
   a. If $\exists\, j \in [\ell]$ s.t. no $(m_j ; \delta_j, c_j)$ is stored, abort.
   b. Compute $\widehat{m} = f(m_1, \ldots, m_\ell)$.
   c. Output $\langle \texttt{Evaluate} : \mathsf{sid}, |f|, \widehat{m}, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{jpk}_j, c_j)_{j\in[\ell]} \rangle$ to $\mathcal{S}$.
5. If $\mathsf{srv} \in \mathsf{Corrupt}$:
   a. If $\exists\, j \in [\ell]$ s.t. no $(m_j ; \delta_j, c_j)$ is stored, $\boxed{\text{abort}}$ set $\widehat{m} := \perp$; otherwise compute $\widehat{m} = f(m_1, \ldots, m_\ell)$.
   b. Output $\langle \texttt{Evaluate} : \mathsf{sid}, f, \widehat{m}, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{jpk}_j, c_j)_{j\in[\ell]} ; \boxed{\tilde{\sigma}} \rangle$ to $\mathcal{S}$.
6. When $\mathcal{S}$ returns $(\widehat{c}, f^\star ; \sigma, \boxed{\mathtt{hon}})$:
   a. Abort if $\widehat{c}$ has been recorded for $(\widehat{\mathsf{grp}}, \mathsf{sid})$.
   b. Set $\hat{\delta} := \mathsf{GetEvalNoise}(f^\star, \{\delta_j\}_{j\in[\ell]}, \sigma)$ //Compute accumulated noise.
   c. Record $(\widehat{m} ; \hat{\delta}, \widehat{c})$ for $(\widehat{\mathsf{grp}}, \mathsf{sid})$ if $\mathsf{srv} \notin \mathsf{Corrupt} \vee (\mathsf{srv} \in \mathsf{Corrupt} \wedge \mathtt{hon} = 1 \wedge \widehat{m} \neq \perp)$.
   d. Record $(\widehat{m} ; \sigma, \widehat{c})$ for $(\widehat{\mathsf{grp}}, \mathsf{sid})$ if $\mathsf{srv} \notin \mathsf{Corrupt} \vee (\mathsf{srv} \in \mathsf{Corrupt} \wedge \mathtt{hon} = 1 \wedge \widehat{m} \neq \perp)$.
7. Output $\{\texttt{"ct"} : \widehat{\mathsf{grp}}, \mathsf{sid}, \widehat{c}\}$ to $\mathcal{P}_{\mathsf{srv}}$.

– $\left\langle \texttt{Share} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, c ; \boxed{\tilde{\rho}} \right\rangle$ from $\mathcal{P}_{\mathsf{cli}}$:
1. If $\mathsf{cli} \notin \mathsf{Corrupt}$:
   a. If $\mathsf{cli} \notin \mathsf{grp}$ or if there is no $(m ; \delta, c)$ stored, abort.
   b. Output $\langle \texttt{Share} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, c, m \rangle$ to $\mathcal{S}$.
2. If $\mathsf{cli} \in \mathsf{Corrupt}$, output $\left\langle \texttt{Share} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, c ; \boxed{\tilde{\rho}} \right\rangle$ to $\mathcal{S}$.
3. When $\mathcal{S}$ returns $(d ; \rho, \mathtt{hon})$, if $\mathsf{cli} \notin \mathsf{Corrupt} \vee (\mathsf{cli} \in \mathsf{Corrupt} \wedge \mathtt{hon} = 1)$, record $(c, m, d ; \rho)$ for $(\mathsf{cli}, \mathsf{sid})$ if there is a record (c, m) for $(\mathsf{grp}, \mathsf{sid})$.
4. Output $\{\texttt{"sh"} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, c, d\}$ to $\mathcal{P}_{\mathsf{cli}}$.

– $\langle \texttt{Combine} : \mathsf{grp}, \mathsf{sid}, c, \{d_j\}_{\mathsf{cli}_j\in\mathsf{grp}'} \rangle$ from $\mathcal{P}_{\mathsf{cli}}$.
1. If $\mathsf{cli} \notin \mathsf{Corrupt}$:
   a. If there is no $(m ; \delta, c)$ for $(\mathsf{grp}, \mathsf{sid})$ stored, abort.
   b. If there exist $(c, m, d'_i ; \rho_i)_{\mathsf{cli}_i\in\mathsf{grp}''}$ from distinct clients s.t. $\left|\{d'_i\}_{\mathsf{cli}_i\in\mathsf{grp}''} \cap \{d_j\}_{\mathsf{cli}_j\in\mathsf{grp}'}\right| \geq \lceil \frac{|\mathsf{grp}|}{t} \rceil$
      i. Set the noise $\hat{\delta} := \delta + \sum_{\mathsf{cli}_i\in\mathsf{grp}''\cap\mathsf{Corrupt}} \rho_i + \sum_{\mathsf{cli}_i\in\mathsf{grp}''\setminus\mathsf{Corrupt}} \rho_i - \delta$.
      ii. Output $\left\{\texttt{"pt"} : \mathsf{grp}, \mathsf{sid}, c, m \boxed{+\hat{\delta}}\right\}$ to $\mathcal{P}_{\mathsf{cli}}$.
   c. Otherwise, abort.
2. If $\mathsf{cli} \in \mathsf{Corrupt}$:
   a. If there is $(m ; \delta, c)$ and Condition 1.b holds, output $\left\langle \texttt{Combine} : \left[\mathsf{grp}, \mathsf{sid}, c, \{d_j\}_{\mathsf{cli}_j\in\mathsf{grp}'}, m \boxed{+\hat{\delta}}\right] \right\rangle$ to $\mathcal{S}$.
   b. Otherwise, output $\langle \texttt{Combine} : [\mathsf{grp}, \mathsf{sid}, c, \{d_j\}_{\mathsf{cli}_j\in\mathsf{grp}'}] \rangle$ to $\mathcal{S}$.
   c. Output $\{\texttt{"pt"} : \mathsf{grp}, \mathsf{sid}, c, m^\star\}$ to $\mathcal{P}_{\mathsf{cli}}$ when $\mathcal{S}$ returns $m^\star$.

– $\langle \texttt{Corrupt} : \mathsf{pid} \rangle$ from $\mathcal{S}$:
1. Label $\mathcal{P}_{\mathsf{pid}}$ as corrupted by setting $\mathsf{Corrupt} := \mathsf{Corrupt} \cup \{\mathsf{pid}\}$.

Fig. 1: Functionality $\mathcal{F}_{\mathsf{MGHE}}$ for multi-group HE. Highlighted components apply only when MGHE is approximate. Dot-boxed components appear exclusively when $\mathcal{S}$ simulates a *semi-malicious* adversary (for Section 2). Framed components appear exclusively when $\mathcal{S}$ simulates a *malicious* adversary (for Section 3).

– *Output Phase*: a client $\mathcal{P}_{\mathsf{cli}}$ requests its decryption share d of a ciphertext c from $\mathcal{F}_{\mathsf{MGHE}}$. Then $\mathcal{F}_{\mathsf{MGHE}}$ queries $\mathcal{S}$ with either c or the pair (c, m) (depending on corruption) to receive the simulated decryption share d. Once a sufficient number of decryption shares have been collected, the client submits them to $\mathcal{F}_{\mathsf{MGHE}}$ to reconstruct the message. We note that this reconstruction step does not involve $\mathcal{S}$, highlighting that it is a non-interactive process.

To the best of our knowledge, this is the *first* UC ideal functionality for homomorphic encryption.We see this formalization as contribution of independent interest. We outline key aspects of the functionality below.

UC with Global Subroutines. Our functionality is defined in the model of UC with *global subroutines* [BCH+20] (UCGS). In particular, we introduce a global key registry functionality $\mathcal{G}_{\mathsf{KRK}}$ (Appendix B.3, Fig. 14), which handles key registration and retrieval, as well as the special key-aggregation operation required in multi-group HE to generate a group public key jpk for a group $\mathsf{grp} = \{\mathsf{cli}_1, \ldots, \mathsf{cli}_n\}$ from individual key pairs $(\mathsf{pk}_i, \mathsf{sk}_i)_{i \in \mathsf{grp}}$ of clients in grp. Since this aggregation must itself be carried out via an MPC procedure, encapsulating it within a global subroutine yields a cleaner formalization than attempting to "bootstrap" an MPC workflow inside the functionality.

Corruptions. Following Canetti [Can20], we assume pid-*wise corruption* in this functionality and in all subsequent ones. In this model, a party $\mathsf{pid} \in \{\mathsf{cli}_1, \ldots, \mathsf{cli}_n, \mathsf{srv}\}$ is corrupted once the adversary sends a corruption message to the functionality; this message has no further effect beyond marking pid as corrupted. From that point onward, all inputs from and outputs to $\mathcal{P}_{\mathsf{pid}}$ are instead provided by, or delivered to, the adversary.

In $\mathcal{F}_{\mathsf{MGHE}}$, we assume that the adversary *non-adaptively* corrupts the clients and possibly the server. Adaptive corruption is non-trivial to achieve, as it would require MGHE, as an advanced variant of public-key encryption, to admit a form of *(receiver) non-committing encryption*, as noted in prior discussions of UC-secure PKE [CFGN96, DN00, CHK05, CLNS17]. Katz et al. [KTZ13] also show that it may call for two-move semi-honest oblivious transfer (OT), projective garbling schemes leaking only the circuit size, in addition to the receiver non-committing encryption, yielding evaluated ciphertexts whose size grows with the circuit and thus sacrificing compactness.

Adversary Models. We distinguish between a *semi-malicious* adversary [AJL+12], who follows the prescribed protocol but may choose the randomness used during execution from arbitrary distributions, and a fully *malicious* adversary, whose behavior is unrestricted. This distinction is reflected in how $\mathcal{F}_{\mathsf{MGHE}}$ treats responses from the ideal-process adversary $\mathcal{S}$: when $\mathcal{S}$ simulates a semi-malicious adversary $\mathcal{A}$, it may specify the coins used in encryption and evaluation, and since $\mathcal{A}$ still runs the prescribed algorithms, the resulting ciphertexts are treated as well-formed and their message–ciphertext pairs are recorded. This model is stronger than the usual semi-honest one and is necessary for HE-based protocols: while zkSNARKs/NIZKs can prove that coins lie in some syntactic range, proving that they were sampled honestly from a prescribed distribution, e.g., a Gaussian, is generally inefficient and may be infeasible.

We additionally allow a semi-malicious adversary to inject malformed decryption shares on behalf of corrupted clients. Such shares are not recorded by $\mathcal{F}_{\mathsf{MGHE}}$ unless declared honest, and are instead handled through its robustness requirement, which is an intentionally general choice, since robustness can be enforced either by proof-based mechanisms, as in Sections 3.3 and 3.4, or by other techniques such as error correction for Linear Secret-Sharing schemes [BS23].

In the *fully malicious* case, by contrast, ciphertexts returned by $\mathcal{S}_{\mathsf{MGHE}}$ are not recorded automatically: in the $\langle \texttt{Encrypt} : \cdot \rangle$ and $\langle \texttt{Evaluate} : \cdot \rangle$ interfaces, $\mathcal{F}_{\mathsf{MGHE}}$ lets $\mathcal{S}_{\mathsf{MGHE}}$ set a flag `hon` indicating whether the ciphertext is explainable, e.g., via extraction from zero-knowledge arguments as in Sections 3.3 and 3.4; ciphertexts accepted in this way become eligible inputs for later evaluation and distributed decryption. We stress that this explanation must be provided *online*: a message–ciphertext pair is recorded at the moment the ciphertext is accepted, and both $\langle \texttt{Evaluate} : \cdot \rangle$ and $\langle \texttt{Share} : \cdot \rangle$ abort on inputs lacking a record. Whether it could instead be deferred depends on when $\mathcal{F}_{\mathsf{MGHE}}$ must *consume* a recorded plaintext. Everywhere except evaluation under a corrupted server, where $\mathcal{S}_{\mathsf{MGHE}}$ supplies the evaluated ciphertext itself, the plaintext is consumed on the spot: with an uncorrupted server, $\widehat{m}$ is handed to $\mathcal{S}_{\mathsf{MGHE}}$ at evaluation time to simulate the evaluated ciphertext for circuit privacy; and for distributed decryption, when fewer than a threshold of clients in a group are corrupted, $\mathcal{F}_{\mathsf{MGHE}}$ determines the decrypted message from its records without consulting $\mathcal{S}_{\mathsf{MGHE}}$, while simulating an uncorrupted client's decryption share itself requires the underlying plaintext, e.g., as in [MW16, KLSW24]. The message must therefore be fixed *no later* than the release of the first decryption share; requiring every record to bind a plaintext upon creation extends this to evaluated ciphertexts $\widehat{m} = f(\{m_j\}_{j\in[\ell]})$, including multi-hop ones, and yields a single invariant across all interfaces.

Multi-Group Security. To capture the multi-group setting, we introduce a "virtual party" $\mathcal{P}_{\mathsf{grp}}$ representing each group. After the initialization of all n clients, we assume k initial groups $\mathcal{P}_{\mathsf{grp}_1}, \ldots, \mathcal{P}_{\mathsf{grp}_k}$ are formed, where $\mathsf{grp}_j = \{\mathsf{cli}_i\}_{i\in I_j}$ for $I_j \subseteq [n]$ and $j \in [k]$. New groups created during computation are modeled as unions of existing ones,[6] i.e., $\widehat{\mathsf{grp}} = \cup_{j\in[\ell]}\mathsf{grp}_j$, where each grp_j contributes an input to the computation of a circuit. Since groups may be formed dynamically and vary in size, reconstruction is parameterized by a *threshold factor* t fixed for all groups[7] i.e., for any group grp, at least $\lceil \frac{|\mathsf{grp}|}{t} \rceil$ members are required to reconstruct a secret.

[6] We remark that MGHE supports dynamic group formation via a standard *key-switching* operation [MTBH21]: a ciphertext, e.g., one obtained after a round of evaluation, can be transformed into a ciphertext under the joint public key of a newly formed group, so that the server can further evaluate it under the new key, while decryption is carried out by the members of the new group.

[7] For simplicity, we adopt a fixed threshold factor across all groups. Supporting group-specific thresholds or access structure could be achieved through inter-group consensus when they form a new group to generate a new group key and run *key-switching* operation on a ciphertext to switch to the new group key.

Approximate and Imperfect Computations. To account for approximate HE, the functionality also tracks the noise incurred during encryption, evaluation, and decryption share, along with the corresponding plaintext-ciphertext pairs. For this purpose, we require the MGHE scheme to come with the following efficiently computable functions. Intuitively, these functions abstract away a scheme-specific noise analysis, of which there is a broad variety in the literature.

- $\mathsf{GetEncNoise}(\omega) \to \delta$: on input the randomness ω used in encryption, output the corresponding noise δ.
- $\mathsf{GetEvalNoise}(f, \{\delta_j\}_{j\in[\ell]}, \sigma) \to \hat{\delta}$: on input a circuit f, the noise values $\{\delta_j\}_{j\in[\ell]}$ of input ciphertexts $\{c_j\}_{j\in[\ell]}$, and the server's additional noise σ (for circuit privacy), output the noise $\hat{\delta}$ of the evaluated ciphertext.

2.2 What Are the "Right" Security Notions for MGHE?

To identify sufficient conditions for realizing $\mathcal{F}_{\mathsf{MGHE}}$, we revisit and introduce notions that unpack its security goals. These notions apply to both approximate and exact schemes, with either imperfect or perfect correctness, and consider a *semi-malicious* adversary that *non-adaptively* corrupts a subset of clients and possibly the server, as in our UC functionality.

2.2.1 Client Privacy

The canonical security notion for multi-group HE is IND-CPA-security [KLSW24]. However, there is reason to believe that this notion is too weak in practice, as attacks against the semantic security of both approximate and exact (non-multi-party) HE schemes are known [AV21, CCP+24, GNSJ24, CSBB24a]. These attacks exploit the disclosure of (partial) decryptions, and may thus naturally transfer to the multi-party setting. Li and Micciancio introduce the stronger notion of IND-CPA^{D} security [LM21] to address this issue in the single-key setting.

We introduce the notion of *Indistinguishability of Chosen Plaintext Attack under Partial-Decryption* (IND-CPA^{pD}), which we believe is the natural analog of IND-CPA^{D} security for multi-group schemes. We formally define IND-CPA^{pD} in Fig. 3 and Definition 2. Since multi-group HE naturally generalizes both threshold and multi-key HE, our notion generalizes and captures existing security notions of threshold HE [BS23, CCP+24] and multi-key HE [KS25]. In Figure 2, we summarize the relationship between our IND-CPA^{pD}notion and existing $\{\mathrm{MK}, \mathrm{Th}, \mathrm{IND}\}$-CPA$^{(\mathrm{D})}$ notions for HE and their variants.

Definition 2 (IND-CPA^{pD} Security). *A multi-group homomorphic encryption scheme* MGHE *is said to satisfy* $(q_e, q_{e^\star}, q_v, q_d, t, \varepsilon)$*-indistinguishability under chosen plaintext attack with decryption share query (*IND-CPA^{pD}*), if for any adversary* $\mathcal{A}$ *that makes* q_e *queries to* $\mathcal{O}_{\mathsf{Enc}}$*,* $q_{e^\star}$ *queries to* $\mathcal{O}_{\mathsf{Enc}^\star}$*,* q_v *queries to* $\mathcal{O}_{\mathsf{Eval}}$*,* q_d *queries to* $\mathcal{O}_{\mathsf{Dec}}$*, and corrupts parties in a group up to the threshold with respect to the threshold factor* t*, the advantage* $\mathbf{Adv}^{\text{IND-CPA}^{\mathrm{pD}}}_{\mathsf{MGHE}}(\mathcal{A}) \leq \varepsilon$ *where*

$$\mathbf{Adv}^{\text{IND-CPA}^{\mathrm{pD}}}_{\mathsf{MGHE}}(\mathcal{A}) := \left|\Pr\left[\mathrm{G}^{\text{IND-CPA}^{\mathrm{pD}}\text{-}0}_{\mathsf{MGHE}}(\mathcal{A}) \Rightarrow 1\right] - \Pr\left[\mathrm{G}^{\text{IND-CPA}^{\mathrm{pD}}\text{-}1}_{\mathsf{MGHE}}(\mathcal{A}) \Rightarrow 1\right]\right| .$$

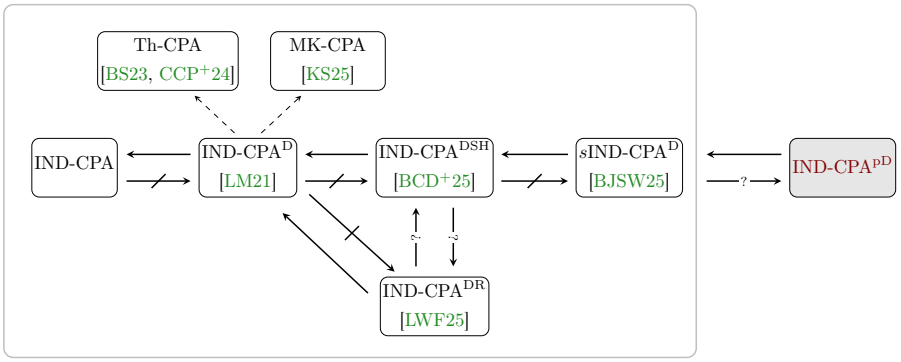


Fig. 2: Security notions for (multi-party) homomorphic encryption. Arrows denote relationships: A - - - ➝ B means that B is a generalization of A in multi-key or threshold setting; ⟶, ⟶/, and —?➝ denote implication, separation, and unknown relation, respectively. Our notion IND-CPA^{pD} generalizes and implies existing notions.

Game: $\mathrm{G}^{\text{IND-CPA}^{\mathrm{pD}}\text{-}b}_{\mathsf{MGHE}}$

Procedure INIT

$\mathsf{pp} \leftarrow \mathsf{MGHE.PGen}(1^\lambda, 1^\kappa)$
$(\{\gamma_i\}_{i\in I_\mathcal{A}}, I_\mathcal{A}) \leftarrow \mathcal{A}(\mathsf{pp})$ // $I_\mathcal{A} \subsetneq [n]$
for $i \in [n] \setminus I_\mathcal{A}$ **do**
  $(\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathsf{MGHE.KGen}(\mathsf{pp})$
for $i \in I_\mathcal{A}$ **do**
  $(\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathsf{MGHE.KGen}(\mathsf{pp}\,;\,\gamma_i)$
$\{I_1, \ldots, I_k\} \leftarrow \mathcal{A}(\{\mathsf{pk}_i\}_{i\in[n]}, \{\mathsf{sk}_i\}_{i\in I_\mathcal{A}})$
for $j \in [k]$ **do**
  $\mathsf{jpk}_j \leftarrow \mathsf{MGHE.KJoin}((\mathsf{pk}_i, \mathsf{sk}_i)_{i\in I_j})$
$T := []$

Oracle $\mathcal{O}_{\mathsf{Eval}}(f, L\,;\,\sigma)$

$\widehat{I} := \cup_{l\in L} T[l].I$
if $|\widehat{I} \cap I_\mathcal{A}| \geq \lceil |\widehat{I}|/t \rceil$ **then**
  return $\perp$
$\widehat{c} \leftarrow \mathsf{MGHE.Eval}(f, (T[l].\mathsf{pk}, T[l].c)_{l\in L}\,;\,\sigma)$
$\widehat{m}_0 := f((T[l].m_0)_{l\in L})$
$\widehat{m}_1 := f((T[l].m_1)_{l\in L})$
$\widehat{\mathsf{jpk}} \leftarrow \mathsf{MGHE.KJoin}((\mathsf{pk}_i, \mathsf{sk}_i)_{i\in\widehat{I}})$
$T := T \bowtie (\widehat{I}, \widehat{\mathsf{jpk}}, \widehat{m}_0, \widehat{m}_1, \widehat{c})$
return $\widehat{c}$

Oracle $\mathcal{O}_{\mathsf{Enc}}(\mathsf{jpk}_j, m_0, m_1)$

if $|I_j \cap I_\mathcal{A}| \geq \lceil |I_j|/t \rceil$ **then**
  return $\perp$
$c \leftarrow \mathsf{MGHE.Enc}(\mathsf{jpk}_j, m_b)$
$T := T \bowtie (I_j, \mathsf{jpk}_j, m_0, m_1, c)$
return c

Oracle $\mathcal{O}_{\mathsf{Enc}^\star}(\mathsf{jpk}_j, m\,;\,\omega)$

$c \leftarrow \mathsf{MGHE.Enc}(\mathsf{jpk}_j, m\,;\,\omega)$
$T := T \bowtie (I_j, \mathsf{jpk}_j, m, m, c)$
return c

Oracle $\mathcal{O}_{\mathsf{Dec}}(l)$

if $T[l].m_0 = T[l].m_1$ **then**
  for $i \in T[l].I$ **do**
    $d_i \leftarrow \mathsf{MGHE.PDec}(\mathsf{sk}_i, T[l].c)$
  return $\{d_i\}_{i\in T[l].I}$

Procedure FIN

$b' \leftarrow \mathcal{A}^{\mathcal{O}_{\{\mathsf{Enc},\mathsf{Enc}^\star,\mathsf{Eval},\mathsf{Dec}\}}}(\{\mathsf{pk}_i\}_{i\in[n]}, \{\mathsf{jpk}_j\}_{j\in[k]}, \{\mathsf{sk}_i\}_{i\in I_\mathcal{A}})$
return b'

Fig. 3: IND-CPA^{pD} game for a multi-group homomorphic encryption (MGHE) scheme. $\bowtie$ appends a tuple to a list.

Towards Semi-Malicious Security. We extend the notion of IND-CPA^{D} security [LM21] to the *semi-malicious* setting by introducing an oracle $\mathcal{O}_{\mathsf{Enc}^\star}$ that lets the adversary $\mathcal{A}$ choose the randomness used in encryptions of *non-challenge* messages, following Bernard et al. [BJSW25]. This additional control enables a refinement of the *noise-flooding* countermeasure of Li et al. [LMSS22]: encryption noise can be calibrated to the average case rather than scaled to a worst-case bound, mitigating attacks that exploit average-scale flooding [CCP+24, GNSJ24] while significantly improving efficiency . Beyond encryption, we allow adversarial control over the randomness of key generation and homomorphic evaluation through queries to $\mathcal{O}_{\mathsf{Eval}}$, thereby capturing settings in which the evaluation server is also corrupted, which is more realistic as also noted by [PS24]. This strictly strengthens the prior notion against a semi-honest server [LWF25], where the adversary merely *learns* honestly sampled randomness from $\mathcal{O}_{\mathsf{Eval}}$, whereas our model grants full control over randomness throughout the protocol execution.

Table 2: Candidate routes toward proving IND-CPA^{pD}. Here ● means that existing techniques plausibly cover the component up to scheme-level adaptation, ◎ denotes a plausible route requiring additional analysis, and ○ marks the main missing proof.

Interface	Current Techniques	Gap	Approx.&Imperfect	Exact&Perfect
$\mathcal{O}_{\mathsf{Enc}^\star}$ Semi-malicious encryption	Low-drift switching [BJSW25]	Lifting to multi-key setting	●	●
$\mathcal{O}_{\mathsf{Dec}}$ Partial decryption	Noise-flooding/smudging [MW16, BS23, KLSW24]	Interplay with other oracles	●	●
$\mathcal{O}_{\mathsf{Eval}}$ Semi-malicious evaluation	(Relaxed Functionality) Bootstrapping fixes [LWF25]	Accommodating semi-malicious randomness	○	●
All interfaces	Combine above	Combined analysis	◎	◎

On Proving IND-CPA^{pD}. In Table 2, we summarize how existing techniques map to the oracle interfaces of the IND-CPA^{pD} game. The semi-malicious encryption interface $\mathcal{O}_{\mathsf{Enc}^\star}$ is closely related to the adversarial-randomness setting handled by low-drift switching [BJSW25], covering both approximate(&imperfect) and exact(&perfect) schemes; the remaining gap is to lift the technique to multiple encryption keys. Leakage from partial decryption can plausibly be, and typically is, addressed by flooding/smudging from threshold and multi-party FHE [MW16, BS23], applicable to both regimes and already used in concrete MGHE instantiations, e.g., [KLSW24]. The least settled component is semi-malicious evaluation: [LWF25] considers leakage of honestly sampled evaluation randomness0, whereas IND-CPA^{pD} lets the adversary *choose* it. Since the randomness is

range-checked in our compilers (Sections 3.3 and 3.4), *sanitizing* the evaluated ciphertext resets its noise to a distribution independent of the adversarial choice (e.g., as in [DS16]), plausibly reducing this case to the semi-honest evaluator handled in [LWF25].

Applying these techniques to a concrete MGHE instantiation [AH19, Par21, CMS+23, KLSW24, CCKY25] thus gives a natural route towards IND-CPA^{pD}. We do not formulate this as a black-box theorem, since the required flooding bound, the treatment of evaluated ciphertexts, and the lifting to multiple joint keys depend on the concrete scheme. We treat IND-CPA^{pD} as the security notion needed to realize our UC functionality $\mathcal{F}_{\mathsf{MGHE}}$, and leave formally instantiating it with concrete HE schemes to future work.

2.2.2 Server Privacy

The notion of *circuit privacy* [Gen09a, IP07, BdPMW16] requires that evaluated ciphertexts are (statistically) indistinguishable from those generated by a simulator, thereby ensuring that no information about the evaluated circuit is leaked. Subsequent work refined this notion, either by tailoring the security to specific techniques (e.g. ciphertext sanitization, noise-flooding) for achieving circuit privacy [DS16, HMS25], by considering relaxed variants [BdPMW16, Gen09b] that allow partial disclosure of the circuit (e.g., its topology), or by considering scenarios where the server possesses a private input in addition to the circuit [KVMGH24].

Simulation-Based Circuit Privacy. For *exact* and *perfect* MGHE, we consider the *simulation-based security*. We adopt the relaxed notion SIM-CIRC (Definition 3) modified from [BdPMW16], in which the topology of the circuit is revealed to the adversary. This relaxation is justified by the assumption that the clients and the server agree in advance on how each client's input is assigned to positions within the circuit. As a result, an adversary controlling corrupted participants can already infer the circuit topology. We note that fully obscuring this topology would likely require fundamentally stronger primitives, such as iO-like obfuscation mechanisms or the adoption of a blind/oblivious execution model (e.g., via universal circuits or universal Turing machine [Val76]), which are well beyond the scope of this work.

Definition 3 (SIM-CIRC **Security).** *A multi-group homomorphic encryption scheme* MGHE *satisfies* circuit privacy *if* $\exists$ *a PPT simulator* $\mathtt{Sim}_{\mathrm{circ}}$ *s.t. for any ciphertexts* $c_1, \ldots, c_\ell$,

$$\mathtt{Sim}_{\mathrm{circ}}((\mathsf{jpk}_1, c_j)_{j\in[\ell]}, f(m_1, \ldots, m_\ell)) \overset{s}{\approx} \mathsf{MGHE}.\mathtt{Eval}(f, (\mathsf{jpk}_j, c_j)_{j\in[\ell]})$$

where $c_j \leftarrow \mathsf{MGHE}.\mathtt{Enc}(\mathsf{jpk}_j, m_i; \omega_i)$, $\mathsf{jpk}_j \leftarrow \mathsf{MGHE}.\mathtt{KJoin}((\mathsf{pk}_i, \mathsf{sk}_i)_{i\in I_j})$ *and* $(\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathsf{MGHE}.\mathtt{KGen}(\mathsf{pp}; \gamma_i)$ *with* $\mathsf{pp} \leftarrow \mathsf{MGHE}.\mathtt{PGen}(1^\lambda, 1^\kappa)$ *for some security parameter* λ *and circuit depth* κ*, where* ω_i *and* γ_i *are random coins that may be selected by an adversary from an arbitrary distribution.*

Game-Based Circuit Privacy. Simulation-based definitions which use the evaluated plaintext result for simulation may not be a suitable formalization for schemes using *approximate evaluation* or with *imperfect correctness*. For this, Kluczniak and Santato [KS25] proposed a relaxation of circuit privacy by introducing a game-based notion IND-CIRC which relaxes the security to computational indistinguishability.

In this work, we extend the notion IND-CIRC (Fig. 4 and Definition 4) introduced in [KS25] to the multi-group setting and to account for semi-malicious clients. Similarly as earlier, the oracles $\mathcal{O}_{\mathsf{Enc}}$ allows the adversary to select arbitrary randomness for encryption on behalf of corrupted client respectively. The adversary is given a left-or-right challenge evaluation oracle $\mathcal{O}_{\mathsf{Eval}}$ that takes two circuits and their respective input ciphertexts i.e., $(f_0, \{c_{0,1}, \ldots, c_{0,\ell}\}, f_1, \{c_{1,1}, \ldots, c_{1,\ell}\})$ where each input ciphertext is well-defined i.e., is output by either from $\mathcal{O}_{\mathsf{Enc}}$ or $\mathcal{O}_{\mathsf{Eval}}$ since we allow the disclosure of circuit topology.

Game: $\mathrm{G}^{\text{IND-CIRC-}b}_{\mathsf{MGHE}}$

Procedure INIT

$\mathsf{pp} \leftarrow \mathsf{MGHE.PGen}(1^\lambda, 1^\kappa)$
$(\{\gamma_i\}_{i \in I_{\mathcal{A}}}, I_{\mathcal{A}}) \leftarrow \mathcal{A}(\mathsf{pp})$ // $I_{\mathcal{A}} \subseteq [n]$
for $i \in [n] \setminus I_{\mathcal{A}}$ **do**
  $(\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathsf{MGHE.KGen}(\mathsf{pp})$
for $i \in I_{\mathcal{A}}$ **do**
  $(\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathsf{MGHE.KGen}(\mathsf{pp}\,;\, \gamma_i)$
$\{I_1, \ldots, I_k\} \leftarrow \mathcal{A}(\{\mathsf{pk}_i\}_{i \in [n]}, \{\mathsf{sk}_i\}_{i \in I_{\mathcal{A}}})$
for $j \in [k]$ **do**
  $\mathsf{jpk}_j \leftarrow \mathsf{MGHE.KJoin}((\mathsf{pk}_i, \mathsf{sk}_i)_{i \in I_j})$
$\mathcal{T} \coloneqq [\,]$

Oracle $\mathcal{O}_{\mathsf{Dec}}(l)$

for $i \in \mathcal{T}[l].I$ **do**
  $d_i \leftarrow \mathsf{MGHE.PDec}(\mathsf{sk}_i\ \mathcal{T}[l].c)$
return $\{d_i\}_{i \in \mathcal{T}[l].I}$

Oracle $\mathcal{O}_{\mathsf{Enc}}(\mathsf{jpk}_j, m\,;\, \omega)$

$c \leftarrow \mathsf{MGHE.Enc}(\mathsf{jpk}_j, m\,;\, \omega)$
$\mathcal{T} \coloneqq \mathcal{T} \bowtie (I_j, \mathsf{jpk}_j, m, c)$
return c

Oracle $\mathcal{O}_{\mathsf{Eval}}(f_0, f_1, L_0, L_1)$

$\widehat{I} \coloneqq \cup_{l \in L_b} \mathcal{T}[l].I$
$\widehat{m}_0 \coloneqq f_0((\mathcal{T}[l].m)_{l \in L_0})$
$\widehat{m}_1 \coloneqq f_1((\mathcal{T}[l].m)_{l \in L_1})$
if $\widehat{m}_0 \neq \widehat{m}_1$ **then**
  return $\perp$
$\widehat{c} \leftarrow \mathsf{MGHE.Eval}(f_b, (\mathcal{T}[l].\mathsf{pk}, \mathcal{T}[l].c)_{l \in L_b})$
$\mathcal{T} \coloneqq \mathcal{T} \bowtie (\widehat{I}, \widehat{m}_0, \widehat{c})$
return $\widehat{c}$

Procedure FIN

return $b' \leftarrow \mathcal{A}^{\mathcal{O}_{\{\mathsf{Enc}, \mathsf{Eval}, \mathsf{Dec}\}}}(\{\mathsf{pk}_i\}_{i \in [n]}, \{\mathsf{jpk}_j\}_{j \in [k]}, \{\mathsf{sk}_i\}_{i \in I_{\mathcal{A}}})$

Fig. 4: IND-CIRC game for a multi-group homomorphic encryption scheme. $\bowtie$ appends a tuple to a list.

We note that when querying $\mathcal{O}_{\mathsf{Eval}}$, the adversary is allowed to select two sets L_0 and L_1 of input ciphertexts. As noted by Li and Micciancio [LM21], if only a single set L were permitted, the circuit topology would be revealed to the adversary. Our formalization in Fig. 4 captures the more general case (i.e., it can be $L_0 \neq L_1$), yet we still define the adversary s.t. it only selects one set of input ciphertexts when building reductions (details in Section 2.3).

Definition 4 (IND-CIRC Security). *A multi-group homomorphic encryption scheme* MGHE *is said to satisfy* $(q_e, q_v, q_d, \varepsilon)$*-circuit indistinguishability*

*(*IND-CIRC*), if for any adversary $\mathcal{A}$ that makes q_e queries to $\mathcal{O}_{\mathsf{Enc}}$, q_v queries to $\mathcal{O}_{\mathsf{Eval}}$, q_d queries to $\mathcal{O}_{\mathsf{Dec}}$, the advantage $\mathbf{Adv}^{\text{IND-CIRC}}_{\mathsf{MGHE}}(\mathcal{A}) \leq \varepsilon$ where*

$$\mathbf{Adv}^{\text{IND-CIRC}}_{\mathsf{MGHE}}(\mathcal{A}) \coloneqq \left|\Pr\left[\mathrm{G}^{\text{IND-CIRC-0}}_{\mathsf{MGHE}}(\mathcal{A}) \Rightarrow 1\right] - \Pr\left[\mathrm{G}^{\text{IND-CIRC-1}}_{\mathsf{MGHE}}(\mathcal{A}) \Rightarrow 1\right]\right|.$$

2.2.3 Threshold Security

We generalize security guarantees beyond the restrictive (n, n)-threshold case, which is typically considered in existing works [LTV12, MW16, KLSW24, HHK+25], to more flexible (t, n)-threshold structure[8] which aligns more with our goal that not all parties have to be online at the same time. In this work, we assume that the decryption process is divided into two phases: 1) each party first "partially" decrypts the ciphertext c to a decryption share d; 2) these decryption shares are combined together to recover the message. For this, we consider two notions for threshold security: *simulatability of decryption shares* (SIM-PDEC) and *robustness*.

Partial-Decryption Simulatability. We assume that the multi-group HE supports decryption shares. This necessitates an additional security property ensuring that a decryption share does not leak any information about the (individual) secret keys of each client. *Simulatability of Decryption Share* (SIM-PDEC), first defined by Boneh et al. [BGG+18] for threshold HE, formalizes the security that decryption shares do not reveal the key shares of parties participating in the decryption. Intuitively, decryption shares are simulatable if there exists a simulator that simulates the decryption share of *not corrupted client* without the knowledge of their individual secret keys. In Definition 5, we restate the definition of simulatability of decryption shares in the multi-group setting.

Definition 5 (SIM-PDEC **Security).** *Let I be an index set representing all the clients in a group and let $I_{\mathcal{A}} \subsetneq I$ denote the index set of corrupted parties. A multi-group homomorphic encryption scheme* MGHE *satisfies* partial-decryption simulatability *(*SIM-PDEC*) if $\exists$ a PPT simulator $\mathtt{Sim}_{\mathrm{th}}$ s.t.*

$$\forall i \in I \setminus I_{\mathcal{A}},\ \mathtt{Sim}_{\mathrm{th}}(c, m, \{\mathsf{sk}_j\}_{j\in I_{\mathcal{A}}}, \{d_j\}_{j\in I_{\mathcal{A}}}) \overset{s}{\approx} \mathsf{MGHE.PDec}(\mathsf{sk}_i, c)$$

where $d_j \leftarrow \mathsf{MGHE.PDec}(\mathsf{sk}_j, c)$ for $j \in I_{\mathcal{A}}$ and $m \leftarrow \mathsf{MGHE.DistDec}(\{\mathsf{sk}_i\}_{i\in I}, c)$.

Remark 1 (Why IND-CPA^{pD} *and* SIM-PDEC *Are Both Needed).* We stress that IND-CPA^{pD} security remains necessary in addition to SIM-PDEC security. At first glance, the oracle $\mathcal{O}_{\mathsf{Dec}}$ may appear redundant once decryption shares are simulatable. Several works [MW16, KLSW24] have also combined SIM-PDEC with standard IND-CPA in their security proofs. However, IND-CPA + SIM-PDEC does *not* in general imply IND-CPA^{pD}. The reason is that SIM-PDEC is a share-simulation property: it ensures that individual honest decryption shares can be simulated, but it does not by itself rule out plaintext leakage through the

[8] We remark that it is trivial to adapt our notion to accommodate general *access structures* [CK93, RB07].

partial-decryption interface as a whole, especially after shares are combined or when correctness/failure behavior is observable.

A concrete example is the *bootstrapping failure* attack of Cheon et al. [CCP+24], where decryption failures are exploited after the decryption shares are combined. This shows that even if individual shares are simulatable, the behavior of combined decryption may still leak information correlated with the challenge plaintext. In particular, such attacks are not limited to approximate HE: they also apply to exact FHE schemes with imperfect correctness, including BFV/BGV-style constructions in practical parameter regimes [Bra12, FV12, CCP+24]. Thus, SIM-PDEC protects the secrecy with respect to individual decryption shares, while IND-CPA^{pD} protects client plaintexts against the full partial-decryption interface; neither assumption should be viewed as replacing the other.

Robustness. For a general (t, n)-threshold decryption setting, we introduce *robustness*, which ensures that the correct message[9] is reconstructed whenever sufficiently many valid decryption shares are available, regardless of malicious shares injected by the adversary [BBH06]. Boudgoust and Scholl [BS23] first formalized it for *threshold* HE, but only for freshly encrypted ciphertexts—a significant limitation, since evaluated ciphertexts are typically the ones decrypted in practice, e.g., to recover MPC outputs. We therefore define *robustness* (Fig. 5 and Definition 6) explicitly for evaluated ciphertexts, and, to model the co-presence of a semi-malicious server and clients, introduce an $\mathcal{O}_{\mathsf{Eval}}$ oracle that lets the server choose the randomness used during evaluation.

For approximate HE, we refine the definition to track noise growth via $\mathsf{GetEncNoise}(\cdot)$ and $\mathsf{GetEvalNoise}(\cdot)$ as in $\mathcal{F}_{\mathsf{MGHE}}$: the adversary wins if the reconstructed message deviates from the intended one by more than a *publicly computable* bound $B \leftarrow \mathsf{Estimate}(f, B_{\mathsf{Enc}}, B_{\mathsf{Eval}}, B_{\mathsf{Dec}})$, which depends on the circuit and the encryption, evaluation, and decryption error bounds, as in [KS25].

While in Sections 3.3 and 3.4, we use zero-knowledge arguments to filter out ill-formed decryption shares as a generic way to enforce robustness, we retain this abstraction for generality, since other methods exist, such as error correction for Linear Secret-Sharing (LSSS) schemes [BS23].

Definition 6 (Robustness). *A multi-group homomorphic encryption scheme* MGHE *is said to satisfy* $(q_e, q_v, q_d, t, \varepsilon)$*-robustness, if for any adversary* $\mathcal{A}$ *that makes* q_e *queries to* $\mathcal{O}_{\mathsf{Enc}}$, q_v *queries to* $\mathcal{O}_{\mathsf{Eval}}$, q_d *queries to* $\mathcal{O}_{\mathsf{Dec}}$, *and corrupts a group up to the threshold with respect to a threshold factor* t, *the advantage* $\mathbf{Adv}^{\mathrm{Rob}}_{\mathsf{MGHE}}(\mathcal{A}) \leq \varepsilon$ *where*

$$\mathbf{Adv}^{\mathrm{Rob}}_{\mathsf{MGHE}}(\mathcal{A}) \coloneqq \Pr\left[\mathrm{G}^{\mathrm{Rob}}_{\mathsf{MGHE}}(\mathcal{A}) \Rightarrow 1\right].$$

2.3 Realizing $\mathcal{F}_{\mathsf{MGHE}}$ against Semi-Malicious Adversaries

We show that the new security properties introduced in Section 2.2 are sufficient to UC-realize the ideal functionality $\mathcal{F}_{\mathsf{MGHE}}$. A multi-group homomorphic encryption

[9] Thus, this notion also captures correctness of an MGHE scheme.

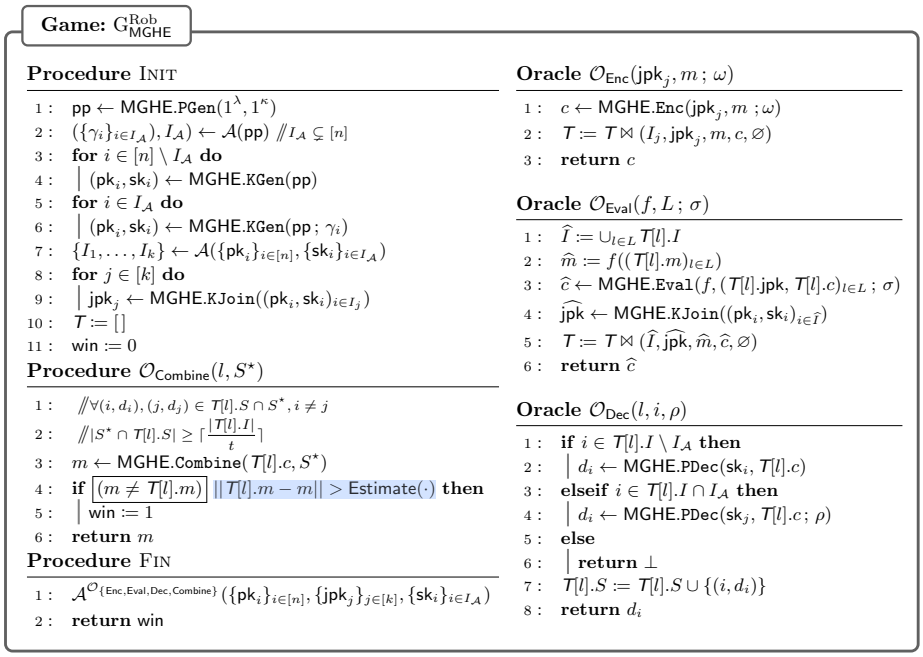


Fig. 5: Robustness game for a multi-group homomorphic encryption scheme. Framed components appear only when MGHE is exact and perfectly correct. Highlighted components appear only when MGHE is approximate or imperfectly correct. $\bowtie$ appends a tuple to a list.

scheme MGHE defines a corresponding protocol Π_{MGHE} (Fig. 6). If MGHE satisfies the required security properties, then Π_{MGHE} UC-realizes $\mathcal{F}_{MGHE}$ with respect to the ideal-process adversary $\mathcal{S}_{MGHE}$ against a *semi-malicious* adversary that *non-adaptively* corrupt a subset of clients and possibly the server. We split this result into two theorems, each addressing different corruption scenarios.

Corruption of Both Clients and Server. In Theorem 1, both the server and a subset of clients are corrupted, and only client-side confidentiality must be ensured.

Theorem 1. *Let* MGHE *be a multi-group homomorphic encryption scheme with threshold factor t. The protocol Π_{MGHE} UC-realizes the functionality $\mathcal{F}_{MGHE}$ in the presence of a global subroutine $\mathcal{G}_{KRK}$ against a semi-malicious adversary that non-adaptively corrupts the clients and the server, and injects malformed decryption shares on behalf of corrupted clients, if:*

- MGHE *satisfies robustness, and the adversary corrupts no more than* $\lceil \frac{|grp|}{t} \rceil$ *clients in any group.*
- MGHE *is* IND-CPA^{pD}*-secure, and there exists at least one not corrupted client who provides input to the computation, s.t. the group* grp *under which it provides input contains no more than* $\lceil \frac{|grp|}{t} \rceil$ *corrupted clients.*
- MGHE *satisfies* SIM-PDEC *security with respect to a simulator* Sim_{th}.

Protocol: Π_{MGHE}

Participants: n clients $\mathcal{P}_{\mathsf{cli}_1}, \mathcal{P}_{\mathsf{cli}_2}, \ldots, \mathcal{P}_{\mathsf{cli}_n}$; Initially, subsets of clients $\{\mathcal{P}_{\mathsf{cli}_i}\}_{i\in I}$ with $I \subseteq [n]$ form k groups $\mathcal{P}_{\mathsf{grp}_j}$ with $\mathsf{grp}_j = \{\mathsf{cli}_i\}_{i\in I_j}$ with $I_j \subseteq I$ for $j \in [k]$; a server $\mathcal{P}_{\mathsf{srv}}$.

- $\langle \texttt{RegParam} : \mathsf{sid}\rangle$ to $\mathcal{P}_{\mathsf{cli}}$: Forward this input to $\mathcal{G}_{\mathsf{KRK}}$.
- $\langle \texttt{RegKey} : \mathsf{sid}\rangle$ to $\mathcal{P}_{\mathsf{cli}}$: Forward this input to $\mathcal{G}_{\mathsf{KRK}}$.
- $\langle \texttt{JoinKey} : \mathsf{sid}\rangle$ to $\mathcal{P}_{\mathsf{grp}}$: Forward this input to $\mathcal{G}_{\mathsf{KRK}}$.
- $\langle \texttt{Encrypt} : \mathsf{grp}, \mathsf{sid}, \mathsf{jpk}, m\rangle$ to $\mathcal{P}_{\mathsf{cli}}$:
  1. Retrieve jpk' for grp from $\mathcal{G}_{\mathsf{KRK}}$. If $\mathsf{jpk}' \neq \mathsf{jpk}$, abort.
  2. Otherwise, compute $c \leftarrow \mathsf{MGHE}.\texttt{Enc}(\mathsf{jpk}, m)$ and output $\{\texttt{"ct"} : \mathsf{grp}, \mathsf{sid}, c\}$.
- $\langle \texttt{Evaluate} : \mathsf{sid}, f, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{jpk}_j, c_j)_{j\in[\ell]}\rangle$ to $\mathcal{P}_{\mathsf{srv}}$:
  1. Retrieve jpk'_j for each grp_j. If there is $j \in [\ell]$ s.t. $\mathsf{jpk}'_j \neq \mathsf{jpk}_j$, abort.
  2. If $\mathsf{sid} \neq \mathsf{sid}_j$ for any $j \in [\ell]$, abort.
  3. Set $\widehat{\mathsf{grp}} = \cup_{i\in[\ell]}\mathsf{grp}_j$.
  4. Compute $\widehat{c} \leftarrow \mathsf{MGHE}.\texttt{Eval}(f, (\mathsf{jpk}_j, c_j)_{j\in[\ell]})$
  5. Output $\{\texttt{"ct"} : \widehat{\mathsf{grp}}, \mathsf{sid}, \widehat{c}\}$.
- $\langle \texttt{Share} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, c\rangle$ to $\mathcal{P}_{\mathsf{cli}}$:
  1. If $\mathsf{cli} \notin \mathsf{grp}$, abort.
  2. Retrieve $(\mathsf{pk}, \mathsf{sk})$ from $\mathcal{G}_{\mathsf{KRK}}$.
  3. Compute $d \leftarrow \mathsf{MGHE}.\texttt{PDec}(\mathsf{sk}, c)$ and output $\{\texttt{"sh"} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, c, d\}$.
- $\langle \texttt{Combine} : [\mathsf{grp}, \mathsf{sid}, c, \{d_i\}_{\mathsf{cli}_i\in\mathsf{grp}'}]\rangle$ to $\mathcal{P}_{\mathsf{cli}}$:
  1. If $\mathsf{grp}' \not\subseteq \mathsf{grp}$ or $|\mathsf{grp}' \cap \mathsf{grp}| < \lceil \frac{|\mathsf{grp}|}{t} \rceil$, abort.
  2. Otherwise, compute $m \leftarrow \mathsf{MGHE}.\texttt{Combine}(c, \{d_i\}_{\mathsf{cli}_i\in\mathsf{grp}'})$
  3. Output $\{\texttt{"pt"} : \mathsf{grp}, \mathsf{sid}, c, m\}$.

Fig. 6: Protocol Π_{MGHE} for multi-group homomorphic encryption interacting with $\mathcal{P}_{\mathsf{cli}_i}, \ldots \mathcal{P}_{\mathsf{cli}_n}$, a server $\mathcal{P}_{\mathsf{srv}}$ and a real-world *semi-malicious* adversary $\mathcal{A}$.

We give a full proof of Theorem 1 in Appendix D.1.

Corruption of Only Clients. In Theorem 2 and Corollary 1, we consider the semi-malicious setting where only a subset of clients is corrupted and the server remains not corrupted. In this case, beyond client side, we must also establish server-side privacy, which we capture via an additional hybrid game reducible to circuit privacy.

For approximate/imperfect HE, in Theorem 2, we first build the reduction to the game-based circuit privacy IND-CIRC.

Theorem 2. *The protocol Π_{MGHE} UC-realizes the functionality $\mathcal{F}_{\mathsf{MGHE}}$ in the presence of a global subroutine $\mathcal{G}_{\mathsf{KRK}}$ against a semi-malicious adversary that non-adaptively corrupts only clients and injects malformed decryption shares on behalf of corrupted clients if an approximate* MGHE *is additionally* IND-CIRC *secure.*

We give a full proof of Theorem 2 in Appendix D.2.

We then consider a stronger guarantee by requiring statistical indistinguishability for circuit privacy in the case of exact and perfect HE schemes. In Corollary 1, we establish server-side confidentiality under SIM-CIRC security.

Corollary 1. *The protocol Π_{MGHE} UC-realizes the functionality $\mathcal{F}_{\mathsf{MGHE}}$ in the presence of a global subroutine $\mathcal{G}_{\mathsf{KRK}}$ against a semi-malicious adversary that*

Simulator: $\mathcal{S}_{\mathsf{MGHE}}$

- $\langle \texttt{Encrypt} : \mathsf{grp}, \mathsf{sid}, \mathsf{jpk}, |m| \rangle$ from $\mathcal{F}_{\mathsf{MGHE}}$:
  1. Sample a random coin ω from the default distribution $\mathcal{D}_\omega$
  2. Compute $c \leftarrow \mathsf{MGHE}.\texttt{Enc}(\mathsf{jpk}, 0^{|m|}\,;\,\omega)$.
  3. Return $(c\,;\,\omega)$ to $\mathcal{F}_{\mathsf{MGHE}}$.
- $\langle \texttt{Encrypt} : \mathsf{grp}, \mathsf{sid}, \mathsf{jpk}, m \rangle$ from $\mathcal{F}_{\mathsf{MGHE}}$:
  1. Sample a random coin ω from the default distribution $\mathcal{D}_\omega$.
  2. Compute $c \leftarrow \mathsf{MGHE}.\texttt{Enc}(\mathsf{jpk}, m\,;\,\omega)$.
  3. Return $(c\,;\,\omega)$ to $\mathcal{F}_{\mathsf{MGHE}}$.
- $\langle \texttt{Encrypt} : \mathsf{grp}, \mathsf{sid}, \mathsf{jpk}, m\,;\,\omega \rangle$ from $\mathcal{F}_{\mathsf{MGHE}}$:
  1. Compute $c \leftarrow \mathsf{MGHE}.\texttt{Enc}(\mathsf{jpk}, m\,;\,\omega)$.
  2. Return $(c\,;\,\omega)$ to $\mathcal{F}_{\mathsf{MGHE}}$.
- $\langle \texttt{Evaluate} : \widehat{\mathsf{grp}}, \mathsf{sid}, |f|, \widehat{m}, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{jpk}_j, c_j)_{j\in[\ell]} \rangle$ from $\mathcal{F}_{\mathsf{MGHE}}$:
  1. Compute $(\widehat{c}, \sigma) \leftarrow \mathtt{Sim}_{\mathrm{circ}}(\{\mathsf{jpk}_j\}_{j\in[\ell]}, \widehat{m})$. // $\mathtt{Sim}_{\mathrm{circ}}$ also returns random coins σ.
  2. Sample a circuit $f^\star$ s.t. $|f^\star| = |f|$ and sample a random coin $\sigma \leftarrow \mathcal{D}_\sigma$.
  3. Compute $\widehat{c} \leftarrow \mathsf{MGHE}.\texttt{Eval}(f^\star, (\mathsf{jpk}_j, c_j)_{j\in[\ell]}\,;\,\sigma)$.
  4. Return $(\widehat{c},\ f^\star\,;\,\sigma)$ to $\mathcal{F}_{\mathsf{MGHE}}$.
- $\langle \texttt{Evaluate} : \widehat{\mathsf{grp}}, \mathsf{sid}, f, \widehat{m}, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{jpk}_j, c_j)_{j\in[\ell]}\,;\,\sigma \rangle$ from $\mathcal{F}_{\mathsf{MGHE}}$:
  1. Compute $\widehat{c} \leftarrow \mathsf{MGHE}.\texttt{Eval}(f, (\mathsf{jpk}_j, c_j)_{j\in[\ell]}\,;\,\sigma)$.
  2. Return $(\widehat{c},\ f\ ;\,\sigma)$ to $\mathcal{F}_{\mathsf{MGHE}}$.
- $\langle \texttt{Share} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, c, m \rangle$ from $\mathcal{F}_{\mathsf{MGHE}}$:
  1. Compute $(d\,;\,\rho) \leftarrow \mathtt{Sim}_{\mathrm{th}}(c, m, \{\mathsf{sk}_i\}_{\mathsf{cli}_i \in \mathsf{Corrupt}\cap\mathsf{grp}})$
  2. Return $(d\,;\,\rho)$ to $\mathcal{F}_{\mathsf{MGHE}}$.
- $\langle \texttt{Share} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, c\,;\,\rho \rangle$ from $\mathcal{F}_{\mathsf{MGHE}}$:
  1. Retrieve sk of $\mathcal{P}_{\mathsf{cli}}$ from $\mathcal{G}_{\mathsf{KRK}}$.
  2. Compute $d \leftarrow \mathsf{MGHE}.\texttt{PDec}(\mathsf{sk}, c\,;\,\rho)$
  3. Return $(d\,;\,\rho)$ to $\mathcal{F}_{\mathsf{MGHE}}$.
- $\left\langle \texttt{Combine} : [\mathsf{grp}, \mathsf{sid}, c, \{d_j\}_{\mathsf{cli}_j\in\mathsf{grp}'}, m{+}\hat{\delta}] \right\rangle$ from $\mathcal{F}_{\mathsf{MGHE}}$:
  1. Output $\left\{ \texttt{"pt"} : \mathsf{grp}, \mathsf{sid}, c, m{+}\hat{\delta} \right\}$ to $\mathcal{F}_{\mathsf{MGHE}}$.
- $\langle \texttt{Combine} : [\mathsf{grp}, \mathsf{sid}, c, \{d_j\}_{\mathsf{cli}_j\in\mathsf{grp}'}] \rangle$ from $\mathcal{F}_{\mathsf{MGHE}}$:
  1. Abort.

Fig. 7: Simulator $\mathcal{S}_{\mathsf{MGHE}}$ for the functionality $\mathcal{F}_{\mathsf{MGHE}}$ simulating a *semi-malicious* adversary. Framed components apply only when MGHE is exact and perfectly correct. Highlighted components apply only when MGHE is approximate or imperfectly correc.

non-adaptively corrupts only clients and injects malformed decryption shares on behalf of corrupted clients if an exact MGHE *is additionally* SIM-CIRC *secure.*

We give a full proof of Corollary 1 in Appendix D.3.

Proof Sketches. For the sake of space, we postpone the full proofs of Theorem 1, Theorem 2 and Corollary 1 to Appendix D.1, D.2 and D.3, respectively. The proof proceeds through a series of hybrid games, each transitioning closer to the ideal world. The transitions between these hybrids rely on the security properties of the multi-group homomorphic encryption scheme MGHE.

We give the intuition of the proof for Theorem 2 and Corollary 1 for better generality. The first hybrid transition from the **REAL** to **HYB**^{1} leverages the

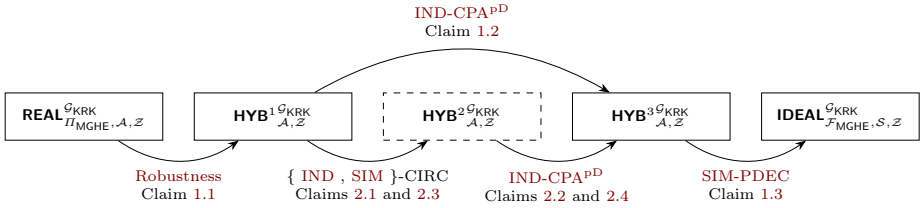


Fig. 8: Hybrid games and reduction in the security proofs for $\mathcal{F}_{\mathsf{MGHE}}$ realization. The dot-boxed hybrid game is exclusively for Theorem 2 and Corollary 1 where the server is not corrupted.

robustness property of MGHE to handle malformed decryption shares. The second transition to $\mathbf{HYB}^2$ employs either IND-CIRC or SIM-CIRC security to ensure server-side privacy. The third transition to $\mathbf{HYB}^3$ utilizes the IND-CPA^{pD} security of MGHE to maintain client-side privacy. Finally, the transition to **IDEAL** is justified by the SIM-PDEC security property, ensuring that the decryption shares do not disclose individual secret keys in the distributed decryption process.

In Fig. 8, we illustrate these hybrids and reductions involved in the proofs of Theorem 1, Theorem 2, and Corollary 1.

3 On-the-Fly MPC

Armed with a solid understanding of UC-secure multi-group homomorphic encryption, we now turn our attention to *on-the-fly multi-party computation* (OtF-MPC). In Section 3.1 We first introduce a UC ideal functionality $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ for OtF-MPC that captures its defining characteristics, including dynamic client participation and computation delegation to an untrusted server. We then show how to UC-realize $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ in various adversarial models.

3.1 UC Functionality for On-the-Fly MPC

We define an ideal functionality $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ (Fig. 9) for on-the-fly multi-party computation, which we show in Fig. 9. In contrast to standard MPC models [CD05] used in several prior works including [LTV12, MW16], $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ incorporates two defining characteristics: (1) *dynamic client participation* and (2) *computation delegation* to an untrusted server.

Under this model, a client joins a computation task *asynchronously* by submitting its input x to $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$, which serves as a secure registry that stores a pair (uid, x) for a unique identifier (uid). The server can subsequently evaluate a circuit f over any subset of stored inputs by specifying their respective uid's and the circuit f to $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$. This addresses a critical gap in between on-the-fly and existing (standard) MPC functionalities, which typically assume a fixed set of participants and treat the circuit as a fixed parameter of the functionality. By explicitly modeling the circuit as a server-provided input, $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ formalizes

Functionality: $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$

Participants: n clients $\mathcal{P}_{\mathsf{cli}_1}, \mathcal{P}_{\mathsf{cli}_2}, \ldots, \mathcal{P}_{\mathsf{cli}_n}$; Initially, subsets of clients $\{\mathcal{P}_{\mathsf{cli}_i}\}_{i\in I}$ with $I \subseteq [n]$ form k groups $\mathcal{P}_{\mathsf{grp}_j}$ with $\mathsf{grp}_j = \{\mathsf{cli}_i\}_{i\in I_j}$ with $I_j \subseteq I$ for $j \in [k]$; a server $\mathcal{P}_{\mathsf{srv}}$; an ideal-process adversary $\mathcal{S}$.

- $\langle \texttt{Upload} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, m\rangle$ from $\mathcal{P}_{\mathsf{cli}}$:
  1. If $\mathsf{cli} \notin \mathsf{Corrupt}$ and fewer than $\lceil \frac{|\mathsf{grp}|}{t} \rceil$ clients in grp have been corrupted, output $\{\texttt{"up"} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, |m|\}$ to $\mathcal{S}$.
  2. Otherwise, output $\{\texttt{"up"} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, m\}$ to $\mathcal{S}$.
  3. If there is a record under $(\mathsf{grp}, \mathsf{sid}, \mathsf{uid})$, abort.
  4. Otherwise, record $\{\texttt{"up"} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, m\}$.
- $\left\langle \texttt{Evaluate} : \mathsf{srv}, \mathsf{sid}, \mathsf{uid}, f, \boxed{\widehat{m}^\star}, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)_{j\in[\ell]} \right\rangle$ from $\mathcal{P}_{\mathsf{srv}}$:
  1. Retrieve $\{\texttt{"}\cdot\texttt{"} : \cdot, \mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j, m_j\}$ under $(\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)$ for $j \in [\ell]$.
  2. If there exist $m_j = \perp$ for any $j \in [\ell]$, abort.
  3. If $\mathsf{sid} \neq \mathsf{sid}_j$ for any $j \in [\ell]$, abort.
  4. Set $\widehat{m} = f(\{m_j\}_{j\in[\ell]})$ and $\widehat{\mathsf{grp}} = \cup_{j\in[\ell]}\mathsf{grp}_j$.
  5. If $\mathsf{srv} \notin \mathsf{Corrupt}$:
     a. If there is a record under $(\widehat{\mathsf{grp}}, \mathsf{sid}, \mathsf{uid})$, abort.
     b. Otherwise, record $\{\texttt{"eval"} : \mathsf{srv}, \widehat{\mathsf{grp}}, \mathsf{sid}, \mathsf{uid}, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)_{j\in[\ell]}, \widehat{m}\}$ under $(\mathsf{grp}, \mathsf{sid}, \mathsf{uid})$.
     c. Output $\{\texttt{"eval"} : \mathsf{srv}, \widehat{\mathsf{grp}}, \mathsf{sid}, \mathsf{uid}, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)_{j\in[\ell]}, \widehat{m}\}$ at $\mathcal{S}$.
  6. If $\mathsf{srv} \in \mathsf{Corrupt}$:
     a. If there is a record under $(\widehat{\mathsf{grp}}, \mathsf{sid}, \mathsf{uid})$, abort.
     b. If $\widehat{m}^\star$ is not provided:
        i. Record $\{\texttt{"eval"} : \mathsf{srv}, \widehat{\mathsf{grp}}, \mathsf{sid}, \mathsf{uid}, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)_{j\in[\ell]}, \widehat{m}\}$ under $(\mathsf{grp}, \mathsf{sid}, \mathsf{uid})$.
        ii. Output $\{\texttt{"eval"} : \mathsf{srv}, \widehat{\mathsf{grp}}, \mathsf{sid}, \mathsf{uid}, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)_{j\in[\ell]}, \widehat{m}\}$ at $\mathcal{S}$.
     c. Otherwise, record $\{\texttt{"eval"} : \mathsf{srv}, \widehat{\mathsf{grp}}, \mathsf{sid}, \mathsf{uid}, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)_{j\in[\ell]}, \widehat{m}^\star\}$ under $(\mathsf{grp}, \mathsf{sid}, \mathsf{uid})$.
- $\langle \texttt{Retrieve} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}\rangle$ from $\mathcal{P}_{\mathsf{cli}}$:
  1. Retrieve message $\{\mathsf{typ} : \cdot, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, \cdot, m\}$ under $(\mathsf{grp}, \mathsf{sid}, \mathsf{uid})$.
  2. If $\mathsf{cli} \notin \mathsf{Corrupt}$:
     a. If $\mathsf{cli} \notin \mathsf{grp}$, abort.
     b. If $\mathsf{typ} = \texttt{"up"}$, then abort.
     c. If $\mathsf{typ} = \texttt{"eval"}$:
        i. If fewer than $\lceil \frac{|\mathsf{grp}|}{t} \rceil$ clients in grp are corrupted, output $\{\texttt{"ret"} : \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, m\}$ to $\mathcal{P}_{\mathsf{cli}}$.
        ii. Otherwise, call $\mathcal{S}$ and output $\{\texttt{"ret"} : \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, m^\star\}$ to $\mathcal{P}_{\mathsf{cli}}$ when $\mathcal{S}$ returns $m^\star$.
  3. If $\mathsf{cli} \in \mathsf{Corrupt}$:
     a. If $\mathsf{typ} = \texttt{"up"}$:
        i. If more than $\lceil \frac{|\mathsf{grp}|}{t} \rceil$ clients in grp are corrupted, output $\langle \texttt{Retrieve} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, m\rangle$ to $\mathcal{S}$.
        ii. Otherwise, output $\langle \texttt{Retrieve} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}\rangle$ to $\mathcal{S}$.
     b. If $\mathsf{typ} = \texttt{"eval"}$, output $\langle \texttt{Retrieve} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, m\rangle$ to $\mathcal{S}$.
     c. Output $\{\texttt{"ret"} : \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, m^\star\}$ to $\mathcal{P}_{\mathsf{cli}}$ when $\mathcal{S}$ returns $m^\star$.
- $\langle \texttt{Corrupt} : \mathsf{pid}\rangle$ from $\mathcal{S}$:
  1. Label $\mathcal{P}_{\mathsf{pid}}$ as corrupted by setting $\mathsf{Corrupt} := \mathsf{Corrupt} \cup \{\mathsf{pid}\}$.

Fig. 9: Functionality $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ for on-the-fly MPC in group-wise setting. Framed inputs are exclusively from the corrupted party (on behalf of the adversary).

the server's role and ensures server-side privacy, which is a requirement that existing models often fail to capture.

We observe that the functionality $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ captures *exact* secure multiparty computation. This functionality can be naturally extended to support *secure approximation*, in the spirit of the $\mathcal{F}_{\mathsf{MGHE}}$ functionality, by explicitly tracking the error incurred at each step and aggregating it over the course of the computation. This approach aligns with the notion of *liberal security* for approximate

computation, as introduced by Feigenbaum et al. [FIM+01]. To achieve *standard security*, Hwang et al. [HHK+25] proposed a formalism that fixes the total error prior to protocol execution and distributes "shares of" noise to each party. We refer the reader to these works for further technical details.

3.2 Realizing $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ against Semi-Malicious Adversary

Leveraging $\mathcal{F}_{\mathsf{MGHE}}$ as a building block, we revisit the established claim [LTV12, MW16, KLSW24] that multi-party HE alone suffices to construct on-the-fly MPC protocols secure against a *semi-malicious* (SM) adversary that *non-adaptively* corrupt the clients and possibly the server.

We show this from the perspective of composable security by presenting a protocol in the $\mathcal{F}_{\mathsf{MGHE}}$-hybrid model, which makes calls to our UC functionality for MGHE, to realize our functionality $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ against a semi-malicious adversary.

Theorem 3. *The protocol $\Pi^{\mathsf{SM}}_{\mathsf{OtF\text{-}MPC}}$ UC-realizes $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ in $\mathcal{F}_{\mathsf{MGHE}}$-hybrid model in the presence of a global subroutine $\mathcal{G}_{\mathsf{KRK}}$ against a semi-malicious adversary that non-adaptively corrupts a group up to the threshold and possibly the server.*

We give the full description of the protocol $\Pi^{\mathsf{SM}}_{\mathsf{OtF\text{-}MPC}}$[10] in Appendix C and we further provide the full proof of Theorem 3 in Appendix D.4 for the realization of $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ in $\mathcal{F}_{\mathsf{MGHE}}$-hybrid model.

3.3 Security against Malicious Adversary from Extractability

We formalize a compiler that leverages a (semi-malicious-secure) MGHE protocol and a zero-knowledge succinct non-interactive argument of knowledge (zkSNARK) to achieve on-the-fly-MPC against *fully malicious* adversaries that can arbitrarily deviate from the protocol and non-adaptively corrupt parties.

The Protocol. Slightly more formally, in the $[\mathcal{F}_{\mathsf{MGHE}}, \mathcal{F}_{\mathsf{zkSNARK}}, \mathcal{F}_{\mathsf{AUTH}}]$-hybrid model, the protocol adopts a single-pass structure (1P) and proceeds in three phases:

– *Input Phase*: A client $\mathcal{P}_{\mathsf{cli}}$ invokes $\mathcal{F}_{\mathsf{MGHE}}$ to encrypt its input m to c with a group public key jpk, then invokes $\mathcal{F}^{R_{\mathsf{Enc}}}_{\mathsf{zkSNARK}}$ with (jpk, c) as instance and (m, ω) as witness, where ω is the encryption randomness, to get an argument π to prove the relation R_{Enc} such that $c \leftarrow \mathsf{MGHE.Enc}(\mathsf{pk}, m\,;\,\omega)$ where

$$R_{\mathsf{Enc}} = \left\{ ((\mathsf{jpk}, c), (m, \omega)) \;\middle|\; \begin{array}{l} c = \mathsf{MGHE.Enc}(\mathsf{jpk}, m\,;\,\omega) \\ \wedge\, -B_{\mathsf{Enc}} \le \omega \le B_{\mathsf{Enc}} \end{array} \right\}. \tag{1}$$

[10] We assume, in $\Pi^{\mathsf{SM}}_{\mathsf{OtF\text{-}MPC}}$ and all subsequent protocols, each group entity $\mathcal{P}_{\mathsf{grp}}$ invokes $\mathcal{G}_{\mathsf{KRK}}$ to combine the keys of its members into a group public key jpk upon formation of the group. For simplicity, this step is omitted from the protocol descriptions.

– *Computation Phase*: The server $\mathcal{P}_{\mathsf{srv}}$ invokes $\mathcal{F}_{\mathsf{MGHE}}$ with its circuit f and input ciphertext $(c_1, \ldots, c_\ell)$ and their respective group public keys $(\mathsf{jpk}_1, \ldots, \mathsf{jpk}_\ell)$ from clients to get an evaluated ciphertext $\widehat{c}$. Then $\mathcal{P}_{\mathsf{srv}}$ invokes $\mathcal{F}_{\mathsf{zkSNARK}}^{R_{\mathsf{Eval}}}$ with $(\mathsf{jpk}_j, c_j)_{j\in[\ell]}$ and $\widehat{c}$ as instance, and the circuit f [11] and σ as witness where σ is evaluation randomness, to get an argument π to prove the relation R_{Eval} such that $\widehat{c} \leftarrow \mathsf{MGHE.Eval}(f, (\mathsf{pk}_j, c_j)_{j\in[\ell]}\,;\,\sigma)$ where

$$R_{\mathsf{Eval}} = \left\{ \left(\left(\widehat{c}, (\mathsf{jpk}_j, c_j)_{j\in[\ell]}\right), (f, \sigma) \right) \;\middle|\; \begin{array}{l} \widehat{c} = \mathsf{MGHE.Eval}(f, (\mathsf{jpk}j, c_j)_{j\in[\ell]}\,;\,\sigma) \\ \wedge\; -B_{\mathsf{Eval}} \le \sigma \le B_{\mathsf{Eval}} \end{array} \right\}. \tag{2}$$

– *Output Phase*: Each client $\mathcal{P}_{\mathsf{cli}_i}$ for $i \in \mathsf{grp}$, where grp is a group, first invokes $\mathcal{F}_{\mathsf{MGHE}}$ to get its decryption share d for some ciphertext c, then invokes $\mathcal{F}_{\mathsf{zkSNARK}}^{R_{\mathsf{Dec}}}$ with (c, d) as instance, and its secret key sk [12] and a decryption randomness ρ as witness, to obtain an argument π to prove that $d \leftarrow \mathsf{MGHE.PDec}(\mathsf{sk}, c\,;\,\rho)$ where

$$R_{\mathsf{Dec}} = \left\{ ((\mathsf{pp}, \mathsf{pk}, c, d), (\mathsf{sk}, \gamma, \rho)) \;\middle|\; \begin{array}{l} d = \mathsf{MGHE.PDec}(\mathsf{sk}, c\,;\,\rho) \\ \wedge\; \mathsf{pk} = \mathsf{MGHE.PKGen}(\mathsf{pp}, \mathsf{sk}\,;\,\gamma) \\ \wedge\; -B_{\mathsf{Dec}} \le \rho \le B_{\mathsf{Dec}} \end{array} \right\}. \tag{3}$$

After verifying the validity of each d through $\mathcal{F}_{\mathsf{zkSNARK}}^{R_{\mathsf{Dec}}}$ and collecting enough valid d's, each client $\mathcal{P}_{\mathsf{cli}_i}$ invokes $\mathcal{F}_{\mathsf{MGHE}}$ to combine these decryption shares together to recover the final result m.

Throughout, ciphertexts, decryption shares, and arguments are broadcasted over an *authenticated* channel $\mathcal{F}_{\mathsf{AUTH}}$ (Fig. 15, Appendix B.4),following standard assumptions in the literature.

Theorem 4. *The protocol $\Pi_{\mathsf{OtF\text{-}MPC}}^{\mathsf{SE\text{-}1P}}$ UC-realizes $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ in $[\mathcal{F}_{\mathsf{MGHE}}, \mathcal{F}_{\mathsf{zkSNARK}}, \mathcal{F}_{\mathsf{AUTH}}]$-hybrid model in the presence of global subroutines $\mathcal{G}_{\mathsf{KRK}}$ and $\mathcal{G}_{\mathsf{RO}}$ against a full malicious adversary that non-adaptively corrupts a group up to the threshold and possibly the server.*

We give the full description of $\Pi_{\mathsf{OtF\text{-}MPC}}^{\mathsf{SE\text{-}1P}}$ in Appendix C and we give a full proof of Theorem 4 in Appendix D.5.

Non-Malleability from Any-Simulation-Extractability. We use (a modification of) the functionality $\mathcal{F}_{\mathsf{zkSNARK}}$ (Fig. 11) defined by Chiesa and Fenzi

[11] In practice, this may not necessarily be the full circuit but rather certain intermediate values during computation for compactness and efficiency, as noted in [KVMGH24].

[12] We assume each public key pk is derived from its secret key sk via $\mathsf{MGHE.PKGen}(\cdot)$ using randomness γ. Equivalently, one may commit directly to sk. UC-secure commitment schemes are known in several models, including the CRS model [CF01], ROM [HM04], stateless-token model, and PUF model [DS13]. Extending our analysis to incorporate such commitments is left for future work.

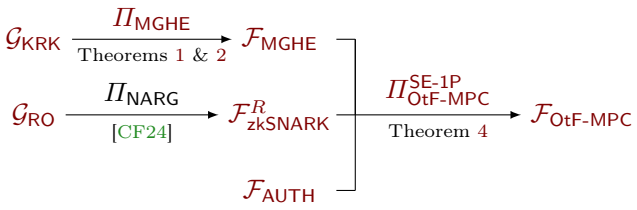


Fig. 10: Protocol $\Pi^{\mathsf{SE\text{-}1P}}_{\mathsf{OtF\text{-}MPC}}$ UC-realizes $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ against a (full) malicious adversary in the $[\mathcal{F}_{\mathsf{MGHE}}, \mathcal{F}_{\mathsf{zkSNARK}}, \mathcal{F}_{\mathsf{AUTH}}]$-hybrid model in the presence of a global key registry $\mathcal{G}_{\mathsf{KRK}}$ and a global random oracle $\mathcal{G}_{\mathsf{RO}}$.

[CF24], which is realizable in the presence of a global programmable random oracle $\mathcal{G}_{\mathsf{RO}}$ [CDG+18] (Fig. 13, Appendix B.2). We modify the functionality $\mathcal{F}_{\mathsf{zkSNARK}}$ to require *any simulation security* [DHLW10] due to its composition with $\mathcal{F}_{\mathsf{MGHE}}$. That is, for an *uncorrupted* prover, the simulator $\pi \leftarrow \mathtt{Sim}_{\mathsf{zk}}(x)$ must be able to simulate the argument π even if $(x, w) \notin R$ for some instance-witness pair (x, w) and a relation R, such that the *completeness* must be satisfied i.e., (x, π) must also be accepted. Indeed, since the ciphertexts and decryption shares are simulated, it is not necessary that they hold for a specific relation R. For example, $\mathcal{S}_{\mathsf{MGHE}}$ for $\mathcal{F}_{\mathsf{MGHE}}$ only learns the message length $|m|$, thus the simulated ciphertext c may not necessarily satisfy the relation R_{Enc}.

On the other hand, when the prover is corrupted, *knowledge soundness via extractability* must still hold. Concretely, for any statement–proof pair (x, π) produced by a corrupted prover and accepted by the verifier, there must exist an efficient (straightline blackbox) extractor $\mathtt{Ext}$ that outputs a witness $w \leftarrow \mathtt{Ext}(x, \pi)$ such that $(x, w) \in R$. This extractability property constitutes a core security requirement: informally, it guarantees that any accepting prover indeed "knows" the underlying witness. From the proof perspective, extractability enables the ideal-process adversary $\mathcal{S}_{\mathsf{OtF\text{-}MPC}}$ for $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ to extract a message m from a ciphertext c submitted by an adversary $\mathcal{A}$ interacting with $\mathcal{F}_{\mathsf{MGHE}}$, $\mathcal{F}_{\mathsf{zkSNARK}}$, and $\mathcal{F}_{\mathsf{AUTH}}$. The extracted message m can then be (retroactively) provided to $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ to allow the simulation.

3.4 Relaxing Extractability with Existential Witness

The protocol with an any-simulation-extractable zkSNARK leads to a straightforward construction, but it is a relatively strong property to require from a zkSNARK. In practice, however, it often suffices to guarantee merely the *existence* of a valid witness certifying that a ciphertext is well-formed. This motivates relaxing the assumption of arguments of knowledge.[13]

[13] We write $\mathcal{F}_{\mathsf{NIZK}}$ to stress that it is not an argument of knowledge. For efficiency, one may instantiate it with a succinct non-interactive argument (SNARG) that also provides zero knowledge.

Functionality: $\mathcal{F}^R_{\mathsf{zkSNARK}}$ $\mathcal{F}^R_{\mathsf{NIZK}}$

Parameter: a binary relation R, an instance bound n.
Participants: a prover $\mathcal{P}_{\mathsf{pvr}}$, a verifier $\mathcal{P}_{\mathsf{vfr}}$, and an ideal-process adversary $\mathcal{S}$.

– $\langle \texttt{Setup} : [\mathsf{sid}] \rangle$ from $\mathcal{P}_{\mathsf{pid}}$:
  1. If Algo has been set for sid or pid $\in$ Corrupt, abort.
  2. Otherwise, output $\langle \texttt{Setup} : [\mathsf{sid}] \rangle$ to $\mathcal{S}$. Upon receiving a tuple of algorithms $(\texttt{Sim}, \texttt{Ext}, \texttt{Vfy})$ from $\mathcal{S}$. Set Algo as this tuple for sid.

– $\langle \texttt{Prove} : \mathsf{sid}, x, (w, w^\star) \rangle$ from the prover $\mathcal{P}_{\mathsf{pvr}}$:
  1. If Algo has not been set, or $|x| > n$, or pvr $\in$ Corrupt, abort.
  2. Output $\langle \texttt{Observe} : \mathsf{sid} \rangle$ to $\mathcal{S}$ and obtain the illegitimate query trace $\{\texttt{"obv"} : \mathsf{sid}, \mathsf{tr}_z\}$. Append tr_z to the trace list ExtTr for session sid.
  3. Call $(\pi, \mathsf{pg}) \leftarrow \texttt{Sim}^{\mathcal{G}_{\mathsf{RO}}[\mathsf{sid}]}(x)$.
  4. For each entry (x, v) in pg, call $\mathcal{S}$ to program $\mathcal{G}_{\mathsf{RO}}$.
  5. Append (x, v) to the list of proved argument Proved in session sid.
  6. Append pg to the list of honestly programmed points HProg in session sid.
  7. Append x to list of proved instance Inst.
  8. Output $\{\texttt{"pf"} : \mathsf{sid}, x, \pi\}$ to $\mathcal{P}_{\mathsf{pvr}}$.

– $\langle \texttt{Verify} : \mathsf{sid}, x, \pi \rangle$ from the verifier $\mathcal{P}_{\mathsf{vfr}}$:
  1. If Algo has not been set, or $|x| > n$, or vfr $\in$ Corrupt, abort.
  2. If $(x, \pi) \in$ Proved, output $\{\texttt{"vf"} : \mathsf{sid}, x, \pi, 1\}$ to $\mathcal{P}_{\mathsf{vfr}}$ and halt.
  3. Call $b \xleftarrow{\mathsf{tr}_v} \texttt{Vfy}^{\mathcal{G}_{\mathsf{RO}}[\mathsf{sid}]}(x, \pi)$. If $b = 0$, output $\{\texttt{"vf"} : \mathsf{sid}, x, \pi, 0\}$ to $\mathcal{P}_{\mathsf{vfr}}$ and halt.
  4. For each $(x, v) \in \mathsf{tr}_v \setminus \mathsf{HProg}[\mathsf{sid}]$, query $\mathcal{G}_{\mathsf{RO}}$ with $\langle \texttt{IsProg} : (\mathsf{sid}, x) \rangle$ to get the response $\{\texttt{"chk"} : b\}$. If $b = 1$ for any (x, v), output $\{\texttt{"vf"} : \mathsf{sid}, x, \pi, 0\}$ to $\mathcal{P}_{\mathsf{vfr}}$ and halt.
  5. If $x \notin \mathsf{Inst}[\mathsf{sid}]$:
     a. Output $\langle \texttt{Observe} : \mathsf{sid} \rangle$ to $\mathcal{S}$ and obtain the illegitimate query trace $\{\texttt{"obv"} : \mathsf{sid}, \mathsf{tr}_z\}$. Append tr_z to the trace list ExtTr for session sid.
     b. For each $(x, v) \in \mathsf{ExtTr}[\mathsf{sid}]$, query $\mathcal{G}_{\mathsf{RO}}$ with $\langle \texttt{IsProg} : (\mathsf{sid}, x) \rangle$. If x is programmed and $(x, v) \in \mathsf{HProg}[\mathsf{sid}]$, append (x, v) to trace tr'_z.
     c. Call $w \leftarrow \texttt{Ext}(x, \pi, \mathsf{tr}'_z)$.
     d. If $(x, w) \notin R$ and output $\{\texttt{"vf"} : \mathsf{sid}, x, \pi, \perp\}$ and halt.
     a. Output $\langle \texttt{Witness} : \mathsf{sid}, x, \pi \rangle$ to $\mathcal{S}$ and get $\{\texttt{"wit"} : \mathsf{sid}, x, \pi, w\}$ as response.
     b. If $\nexists\, w^\star : (x, (w, w^\star)) \in R$, output $\{\texttt{"vf"} : \mathsf{sid}, x, \pi, \perp\}$ and halt.
  6. Output $\{\texttt{"vf"} : \mathsf{sid}, x, \pi, 1\}$ and halt.

– $\langle \texttt{Corrupt} : \mathsf{pid} \rangle$ from $\mathcal{S}$:
  1. Label $\mathcal{P}_{\mathsf{pid}}$ as corrupted by setting Corrupt $\coloneqq$ Corrupt $\cup \{\mathsf{pid}\}$.

Fig. 11: Ideal functionality $\mathcal{F}_{\mathsf{zkSNARK}}$ for an any-simulation-extractable zk-SNARK [CF24] and ideal functionality $\mathcal{F}_{\mathsf{NIZK}}$ with split witness; framed part is exclusively for $\boxed{\mathcal{F}_{\mathsf{zkSNARK}}}$; the highlighted part is exclusively for $\mathcal{F}_{\mathsf{NIZK}}$.

At a high level, this compiler follows the Naor–Yung double encryption paradigm [NY90]. This compiler [BSW12, CRRV17] and some variants [MN24, BCF+25] of it has also been used to achieve CCA-security of HE.

3.4.1 NIZK Functionality with Split Witness

Before presenting the actual protocol, we introduce our ideal functionality $\mathcal{F}_{\mathsf{NIZK}}$ (Fig. 11). Our functionality builds on $\mathcal{F}_{\mathsf{zkSNARK}}$ of [CF24], but slightly relaxes the extractability requirement. Inspired by their treatment of extraction failure, by the *gullible* zero-knowledge functionality of Camenisch et al. [CKS11], and by the functionality of Groth et al. [GOS06], we model witnesses as pairs $(w, w^\star)$. The

component w is subject to knowledge soundness, whereas $w^\star$ is required only to exist.

More concretely, during verification the ideal-process adversary provides the extractable component w. The functionality $\mathcal{F}_{\mathsf{NIZK}}$ then checks whether there exists a corresponding $w^\star$ such that $(w, w^\star)$ is a valid witness for the instance x. If the verifier `Vfy` accepts the argument but no such $w^\star$ exists, then $\mathcal{F}_{\mathsf{NIZK}}$ treats this as a violation of (simulation-based) soundness and returns $\perp$. As in the previous functionalities, $\mathcal{F}_{\mathsf{NIZK}}$ also supports *any-simulation* security [DHLW10]. Therefore, we view *any simulation soundness* of NIZK as the natural candidate condition for realizing this split-witness functionality, and leave a formal realization theorem to future work.

We stress that the existence of $w^\star$ is not meant to be established by exhaustive search. Rather, in a *hybrid-model* simulation, the ideal-process adversary may obtain auxiliary state through an external channel that allows it to certify the existence of such a witness. For example, a $\mathcal{S}_{\mathsf{OtF\text{-}MPC}}$ for $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ that internally simulates $\mathcal{A}$'s interactions in $[\mathcal{F}_{\mathsf{MGHE}}, \mathcal{F}_{\mathsf{NIZK}}]$-hybrid model and thus may obtain the relevant random coins from $\mathcal{F}_{\mathsf{MGHE}}$. These coins can then serve as the existential component $w^\star$ when checking the statement inside $\mathcal{F}_{\mathsf{NIZK}}$.

3.4.2 Naor-Yung Compiler with Split-Witness NIZK

Informally, we replace the any-simulation-extractable zkSNARK with a NIZK and follow a *two-pass* (2P) structure in each phase such that two calls are made to $\mathcal{F}_{\mathsf{MGHE}}$ in two sessions with one call to $\mathcal{F}_{\mathsf{NIZK}}$ to prove the honest execution.

– *Input Phase*: A client $\mathcal{P}_{\mathsf{cli}}$ invokes $\mathcal{F}_{\mathsf{MGHE}}$ two times in $(\mathsf{sid}, 1)$ and $(\mathsf{sid}, 2)$ to encrypt its input m to c_1 and c_2 with two independent random coins ω_1 and ω_2. Then $\mathcal{P}_{\mathsf{cli}}$ invokes $\mathcal{F}_{\mathsf{NIZK}}^{R_{\mathsf{NY}}}$ with $(\mathsf{jpk}_1, c_1, \mathsf{jpk}_2, c_2)$ as instance, m as knowledge witness, and (ω_1, ω_2) as existential witness to get an argument π for the relation

$$R_{\mathsf{NY}} = \left\{ \left(\begin{pmatrix} \mathsf{jpk}_1, c_1, \\ \mathsf{jpk}_2, c_2 \end{pmatrix}, (m, (\omega_1, \omega_2)) \right) \;\middle|\; \begin{array}{l} \forall u \in \{1,2\} : \\ \quad c_u = \mathsf{MGHE.Enc}(\mathsf{jpk}_u, m; \omega_u) \\ \quad \wedge -B_{\mathsf{Enc}} \le \omega_u \le B_{\mathsf{Enc}} \end{array} \right\}. \tag{4}$$

– *Computation phase*: The server $\mathcal{P}_{\mathsf{srv}}$ invokes $\mathcal{F}_{\mathsf{MGHE}}$ with its circuit f and input ciphertext $(c_{u,1}, \ldots, c_{u,\ell})$ and respective group public keys $(\mathsf{jpk}_{u,1}, \ldots, \mathsf{jpk}_{u,\ell})$ for $u \in \{1,2\}$ from clients to get an evaluated ciphertext $\widehat{c}_1$ and $\widehat{c}_2$ in sessions $(\mathsf{sid}, 1)$ and $(\mathsf{sid}, 2)$. Then $\mathcal{P}_{\mathsf{srv}}$ invokes $\mathcal{F}_{\mathsf{NIZK}}^{R_{\mathsf{Eval}}}$ with $(\mathsf{jpk}_{u,j}, c_{u,j})_{u \in \{1,2\}, j \in [\ell]}$, $\widehat{c}_1$ and $\widehat{c}_2$ as instance, and (σ_1, σ_2) as existential witness where σ_1 and σ_2 are evaluation randomness in sessions $(\mathsf{sid}, 1)$ and $(\mathsf{sid}, 2)$, to get an argument π for the relation

$$R_{\mathsf{Eval}} = \left\{ \left(\begin{pmatrix} \widehat{c}_1, (\mathsf{jpk}_{j,1}, c_{j,1})_{j \in [\ell]}, \\ \widehat{c}_2, (\mathsf{jpk}_{j,2}, c_{j,2})_{j \in [\ell]} \end{pmatrix}, \left(f, \begin{pmatrix} \sigma_1, \\ \sigma_2 \end{pmatrix} \right) \right) \;\middle|\; \begin{array}{l} \forall u \in \{1,2\} : \\ \quad \widehat{c}_u = \mathsf{MGHE.Eval}(f, (\mathsf{jpk}_{j,u}, c_{j,u})_{j \in [\ell]}; \sigma_u) \\ \quad \wedge -B_{\mathsf{Eval}} \le \sigma_u \le B_{\mathsf{Eval}} \end{array} \right\}. \tag{5}$$

– *Output phase*: Each client $\mathcal{P}_{\mathsf{cli}_i}$ for $i \in \mathsf{grp}$ invokes $\mathcal{F}_{\mathsf{MGHE}}$ to get its decryption shares d_1 and d_2 for ciphertexts c_1 and c_2 in sessions $(\mathsf{sid}, 1)$ and $(\mathsf{sid}, 2)$, then invokes $\mathcal{F}_{\mathsf{NIZK}}^{R_{\mathsf{Dec}}}$ with (c_1, d_1, c_2, d_2) as instance, and its secret key sk_1 and sk_2 as knowledge witness and two decryption randomness ρ_1 and ρ_2 as existential witness, to obtain an argument π for the relation

$$R_{\mathsf{Dec}} = \left\{ \left(\begin{pmatrix} \mathsf{pp}_1, \mathsf{pk}_1, c_1, d_1, \\ \mathsf{pp}_2, \mathsf{pk}_2, c_2, d_2 \end{pmatrix}, \begin{pmatrix} \mathsf{sk}_1, (\gamma_1, \rho_1), \\ \mathsf{sk}_2, (\gamma_2, \rho_2) \end{pmatrix} \right) \middle| \begin{array}{l} \forall u \in \{1,2\} : \\ \quad d_u = \mathsf{MGHE.PDec}(\mathsf{sk}_u, c_u; \rho_u) \\ \quad \wedge\ \mathsf{pk}_u = \mathtt{PKGen}(\mathsf{pp}_u, \mathsf{sk}_u; \gamma_u) \\ \quad \wedge\ -B_{\mathsf{Dec}} \leq \rho_u \leq B_{\mathsf{Dec}} \end{array} \right\}. \tag{6}$$

After verifying the validity of each d_1 and d_2 through $\mathcal{F}_{\mathsf{NIZK}}^{R_{\mathsf{Dec}}}$ and collecting enough valid d_1's and d_2, each client $\mathcal{P}_{\mathsf{cli}_i}$ invokes $\mathcal{F}_{\mathsf{MGHE}}$ to combine these decryption shares together to recover two final results m_1 and m_2. The client checks for the equivalence of m_1 and m_2 (or if the error between m_1 and m_2 is within the estimated range). If yes, the client output either message; otherwise, it aborts the protocol and outputs nothing.

Theorem 5. *The protocol $\Pi_{\mathsf{OtF\text{-}MPC}}^{\mathsf{SS\text{-}2P}}$ UC-realizes $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ in $[\mathcal{F}_{\mathsf{MGHE}}, \mathcal{F}_{\mathsf{NIZK}}, \mathcal{F}_{\mathsf{AUTH}}]$-hybrid model in the presence of global subroutines $\mathcal{G}_{\mathsf{KRK}}$ and $\mathcal{G}_{\mathsf{RO}}$ against a full malicious adversary that non-adaptively corrupts a group up to the threshold and possibly the server.*

We give the full description of the protocol $\Pi_{\mathsf{OtF\text{-}MPC}}^{\mathsf{SS\text{-}2P}}$ in Appendix C and we give the full proof of Theorem 5 in Appendix D.6.

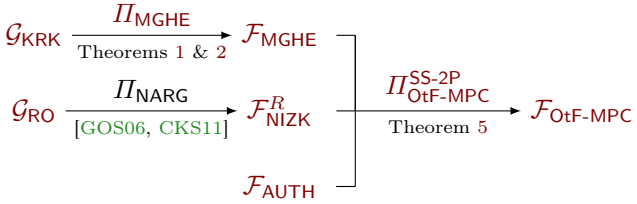


Fig. 12: Protocol $\Pi_{\mathsf{OtF\text{-}MPC}}^{\mathsf{SS\text{-}2P}}$ UC-realizes $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ against a (full) malicious adversary in the $[\mathcal{F}_{\mathsf{MGHE}}, \mathcal{F}_{\mathsf{NIZK}}, \mathcal{F}_{\mathsf{AUTH}}]$-hybrid model in the presence of global subroutines $\mathcal{G}_{\mathsf{KRK}}$ and $\mathcal{G}_{\mathsf{RO}}$.

Non-Malleability from Naor–Yung Paradigm. The split between knowledge and existential witnesses is meaningful only because $\Pi_{\mathsf{OtF\text{-}MPC}}^{\mathsf{SS\text{-}2P}}$ uses the Naor–Yung double-encryption structure. Since $\mathcal{F}_{\mathsf{NIZK}}$ requires the ideal-process adversary $\mathcal{S}_{\mathsf{NIZK}}$ to provide only the knowledge witness w, the simulation may continue with any claimed m supplied by $\mathcal{S}_{\mathsf{NIZK}}$, as long as this value can be extended by some existential witness $w^\star$ satisfying the relation R. Thus we must ensure that an arbitrary or inconsistent m can be accepted without being bound to the submitted ciphertext tuple. The Naor–Yung relation rules out this possibility. For

an instance $(\mathsf{jpk}_1, c_1, \mathsf{jpk}_2, c_2)$, the claimed knowledge witness m is accepted only if there exist two randomnesses ω_1, ω_2 such that $c_1 = \mathsf{MGHE.Enc}(\mathsf{jpk}_1, m\,;\,\omega_1)$ and $c_2 = \mathsf{MGHE.Enc}(\mathsf{jpk}_2, m\,;\,\omega_2)$. Thus, the existential witness is not free-standing; it certifies that the claimed knowledge witness m is indeed a common plaintext of the two branches, which is exactly the consistency mechanism underlying the original Naor–Yung paradigm [NY90].

Handling Approximate/Imperfect Computations. For approximate MGHE, we have the clients verify whether the difference between m_1 and m_2 lies within an admissible error range specified by a function $\mathsf{Estimate}(\cdot)$. Thanks to the split witness mechanism, the ideal-process adversary $\mathcal{S}_{\mathsf{OtF\text{-}MPC}}$ for $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ simulating an adversary $\mathcal{A}$ in the $[\mathcal{F}_{\mathsf{MGHE}}, \mathcal{F}_{\mathsf{NIZK}}, \mathcal{F}_{\mathsf{AUTH}}]$-hybrid model can still obtain the message m and explain to $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ while also retaining the consistency guaranteed by the Naor–Yung paradigm even for approximate MGHE.

The final outputs in sessions $(\mathsf{sid}, 1)$ and $(\mathsf{sid}, 2)$ are $m + \delta_1$ and $m + \delta_2$, respectively, for accumulated errors δ_1 and δ_2 in these two sessions. If the same circuit f is evaluated in both sessions, then the difference $\delta_1 - \delta_2$ should lie within the admissible range. Moreover, by the definition of $\mathcal{F}_{\mathsf{MGHE}}$, both δ_1 and δ_2 are (statistically) indistinguishable from a noise term δ generated by $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ (under the liberal security as in [HHK+25]). Consequently, the value $m + \delta$ output by a client in the ideal world is indistinguishable from both $m + \delta_1$ and $m + \delta_2$ produced in the real world. We stress, however, that the function $\mathsf{Estimate}(\cdot)$ must be carefully defined to ensure a secure admissible error range.

Finally, we remark that Brzuska et al. [BCF+25] suggest that the construction by Manulis and Nguyen [MN24], which first follows the Naor–Yung paradigm for encryption and runs a SNARK during evaluation, may not work to construct their (v)CCA secure HE. We emphasize that our observations do *not* imply that the Naor–Yung paradigm generically yields CCA security when applied to approximate HE. We leave a formal investigation of this question as a direction for future work.

Remark 2 (Verification for input ciphertexts). Verifying an evaluated ciphertext also requires that all its input ciphertexts be valid, i.e., each satisfies R_{Enc} or R_{Eval} for some earlier circuit f. In $\Pi^{\mathsf{SE\text{-}1P}}_{\mathsf{OtF\text{-}MPC}}$ and $\Pi^{\mathsf{SS\text{-}2P}}_{\mathsf{OtF\text{-}MPC}}$ we take the simplest approach: clients and the server record invalid ciphertexts and reject later evaluations using them. Alternatively, a *chain of arguments* [BSW12, MN24, BCF+25] would have the relation itself certify input validity, yielding a recursive relation that calls for a *relativized* zkSNARK [CCG+23, BCG24] due to the RO. As the state of the art provides only knowledge soundness while our UC analysis requires (at least) simulation security, we leave UC-secure relativized zkSNARKs to future work.

Acknowledgment. Ganyuan Cao is supported by the ANR-23-CE39-0009 TRUST project. Part of this work was done while Ganyuan Cao was affiliated with EPFL. We thank the anonymous reviewers of Asiacrypt 2026 for their careful reading and constructive feedback, which helped improve this work.

References

AABA+26. Andreea Alexandru, Ahmad Al Badawi, Daniel Apon, Jean-Philippe Bossuat, Sylvain Chatel, Ben Fisch, Nicholas Genise, Shai Halevi, Loïs Huguenin-Dumittan, Guy Itzhaki, Andrey Kim, Yongwoo Lee, Zeyu Liu, Janmajaya Mall, Christian Mouchet, Carlo Pascoe, Chris Peikert, Kabir Peshawaria, Yuriy Polyakov, R.V. Saraswathy, Sarabjeet Singh, Yongsoo Song, Eran Tromer, Vinod Vaikuntanathan, Vincent Zucca, and Guy Zyskind. PANTHERIA: Threshold FHE for RLWE-based cryptosystems. *Preview writeup to NIST-IR8214C*, 2026. https://csrc.nist.gov/presentations/2026/mpts2026-2b5.

ABPS24. Shahla Atapoor, Karim Baghery, Hilder V. L. Pereira, and Jannik Spiessens. Verifiable FHE via lattice-based SNARKs. *CiC*, 1(1):24, 2024. doi:10.62056/a6ksdkp10.

AH19. Asma Aloufi and Peizhao Hu. Collaborative homomorphic computation on data encrypted under multiple keys. *arXiv preprint arXiv:1911.04101*, 2019.

AJL+12. Gilad Asharov, Abhishek Jain, Adriana López-Alt, Eran Tromer, Vinod Vaikuntanathan, and Daniel Wichs. Multiparty computation with low communication, computation and interaction via threshold FHE. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 483–501. Springer, Berlin, Heidelberg, April 2012. doi:10.1007/978-3-642-29011-4_29.

AV21. Adi Akavia and Margarita Vald. On the privacy of protocols based on CPA-secure homomorphic encryption. Cryptology ePrint Archive, Report 2021/803, 2021. URL: https://eprint.iacr.org/2021/803.

BBC+26. Mathieu Ballandras, Carl Bootland, Kelong Cong, Ben Curtis, Daniel Demmler, Tore Kasper Frederiksen, Marc Joye, Benoit Libert, Jean-Baptiste Orfila, Nigel P. Smart, Titouan Tanguy, Samuel Tap, and Michael Walter. TFHE, ZHEnith and Nexus. *Preview writeup to NIST-IR8214C*, 2026. https://csrc.nist.gov/presentations/2026/mpts2026-2b3.

BBH06. Dan Boneh, Xavier Boyen, and Shai Halevi. Chosen ciphertext secure public key threshold encryption without random oracles. In David Pointcheval, editor, *CT-RSA 2006*, volume 3860 of *LNCS*, pages 226–243. Springer, Berlin, Heidelberg, February 2006. doi:10.1007/11605805_15.

BCD+25. Flávio Bergamaschi, Anamaria Costache, Dana Dachman-Soled, Hunter Kippen, Lucas LaBuff, and Rui Tang. Revisiting the security of approximate FHE with noise-flooding countermeasures. In Tibor Jager and Jiaxin Pan, editors, *PKC 2025, Part V*, volume 15678 of *LNCS*, pages 102–134. Springer, Cham, May 2025. doi:10.1007/978-3-031-91832-2_4.

BCF+25. Chris Brzuska, Sébastien Canard, Caroline Fontaine, Duong Hieu Phan, David Pointcheval, Marc Renard, and Renaud Sirdey. Relations among new CCA security notions for approximate FHE. *CiC*, 2(1):20, 2025. doi:10.62056/aee0iv7sf.

BCG24. Annalisa Barbara, Alessandro Chiesa, and Ziyi Guan. Relativized succinct arguments in the ROM do not exist. Cryptology ePrint Archive, Report 2024/728, 2024. URL: https://eprint.iacr.org/2024/728.

BCH+20. Christian Badertscher, Ran Canetti, Julia Hesse, Björn Tackmann, and Vassilis Zikas. Universal composition with global subroutines: Capturing global setup within plain UC. In Rafael Pass and Krzysztof Pietrzak,

editors, *TCC 2020, Part III*, volume 12552 of *LNCS*, pages 1–30. Springer, Cham, November 2020. doi:10.1007/978-3-030-64381-2_1.

BCNP04. Boaz Barak, Ran Canetti, Jesper Buus Nielsen, and Rafael Pass. Universally composable protocols with relaxed set-up assumptions. In *45th FOCS*, pages 186–195. IEEE Computer Society Press, October 2004. doi:10.1109/FOCS.2004.71.

BdPMW16. Florian Bourse, Rafaël del Pino, Michele Minelli, and Hoeteck Wee. FHE circuit privacy almost for free. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part II*, volume 9815 of *LNCS*, pages 62–89. Springer, Berlin, Heidelberg, August 2016. doi:10.1007/978-3-662-53008-5_3.

BGG+18. Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter M. R. Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part I*, volume 10991 of *LNCS*, pages 565–596. Springer, Cham, August 2018. doi:10.1007/978-3-319-96884-1_19.

BJSW25. Olivier Bernard, Marc Joye, Nigel P. Smart, and Michael Walter. Drifting towards better error probabilities in fully homomorphic encryption schemes. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part VIII*, volume 15608 of *LNCS*, pages 181–211. Springer, Cham, May 2025. doi:10.1007/978-3-031-91101-9_7.

BP25. Luís T. A. N. Brandão and René Peralta. Nist first call for multi-party threshold schemes. (National Institute of Standards and Technology) NIST Internal Report (NISTIR) 8214C (Second Public Draft), 3 2025. doi:10.6028/NIST.IR.8214C.2pd.

Bra12. Zvika Brakerski. Fully homomorphic encryption without modulus switching from classical GapSVP. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 868–886. Springer, Berlin, Heidelberg, August 2012. doi:10.1007/978-3-642-32009-5_50.

BS23. Katharina Boudgoust and Peter Scholl. Simple threshold (fully homomorphic) encryption from LWE with polynomial modulus. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part I*, volume 14438 of *LNCS*, pages 371–404. Springer, Singapore, December 2023. doi:10.1007/978-981-99-8721-4_12.

BSW12. Dan Boneh, Gil Segev, and Brent Waters. Targeted malleability: homomorphic encryption for restricted computations. In Shafi Goldwasser, editor, *ITCS 2012*, pages 350–366. ACM, January 2012. doi:10.1145/2090236.2090264.

Can00. Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. Cryptology ePrint Archive, Report 2000/067, 2000. URL: https://eprint.iacr.org/2000/067.

Can20. Ran Canetti. Universally composable security. *J. ACM*, 67(5), September 2020. doi:10.1145/3402457.

CCG+23. Megan Chen, Alessandro Chiesa, Tom Gur, Jack O'Connor, and Nicholas Spooner. Proof-carrying data from arithmetized random oracles. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part II*, volume 14005 of *LNCS*, pages 379–404. Springer, Cham, April 2023. doi:10.1007/978-3-031-30617-4_13.

CCKY25. Jung Hee Cheon, Hyeongmin Choe, Seunghong Kim, and Yongdong Yeo. Multi-party homomorphic encryption with dynamicity and ciphertext reusability. Cryptology ePrint Archive, Report 2025/581, 2025. URL: https://eprint.iacr.org/2025/581.

CCP+24. Jung Hee Cheon, Hyeongmin Choe, Alain Passelègue, Damien Stehlé, and Elias Suvanto. Attacks against the IND-CPA^{D} security of exact FHE schemes. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 2505–2519. ACM Press, October 2024. doi:10.1145/3658644.3690341.

CD05. Ronald Cramer and Ivan Damgård. *Multiparty Computation, an Introduction*, pages 41–87. Birkhäuser Basel, Basel, 2005. doi:10.1007/3-7643-7394-6_2.

CDG+18. Jan Camenisch, Manu Drijvers, Tommaso Gagliardoni, Anja Lehmann, and Gregory Neven. The wonderful world of global random oracles. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part I*, volume 10820 of *LNCS*, pages 280–312. Springer, Cham, April / May 2018. doi:10.1007/978-3-319-78381-9_11.

CDPW07. Ran Canetti, Yevgeniy Dodis, Rafael Pass, and Shabsi Walfish. Universally composable security with global setup. In Salil P. Vadhan, editor, *TCC 2007*, volume 4392 of *LNCS*, pages 61–85. Springer, Berlin, Heidelberg, February 2007. doi:10.1007/978-3-540-70936-7_4.

CF01. Ran Canetti and Marc Fischlin. Universally composable commitments. In Joe Kilian, editor, *CRYPTO 2001*, volume 2139 of *LNCS*, pages 19–40. Springer, Berlin, Heidelberg, August 2001. doi:10.1007/3-540-44647-8_2.

CF24. Alessandro Chiesa and Giacomo Fenzi. zkSNARKs in the ROM with unconditional UC-security. In Elette Boyle and Mohammad Mahmoody, editors, *TCC 2024, Part I*, volume 15364 of *LNCS*, pages 67–89. Springer, Cham, December 2024. doi:10.1007/978-3-031-78011-0_3.

CFGN96. Ran Canetti, Uriel Feige, Oded Goldreich, and Moni Naor. Adaptively secure multi-party computation. In *28th ACM STOC*, pages 639–648. ACM Press, May 1996. doi:10.1145/237814.238015.

CHK05. Ran Canetti, Shai Halevi, and Jonathan Katz. Adaptively-secure, non-interactive public-key encryption. In Joe Kilian, editor, *TCC 2005*, volume 3378 of *LNCS*, pages 150–168. Springer, Berlin, Heidelberg, February 2005. doi:10.1007/978-3-540-30576-7_9.

CK93. Benny Chor and Eyal Kushilevitz. Secret sharing over infinite domains. *Journal of Cryptology*, 6(2):87–95, June 1993. doi:10.1007/BF02620136.

CKKS17. Jung Hee Cheon, Andrey Kim, Miran Kim, and Yong Soo Song. Homomorphic encryption for arithmetic of approximate numbers. In Tsuyoshi Takagi and Thomas Peyrin, editors, *ASIACRYPT 2017, Part I*, volume 10624 of *LNCS*, pages 409–437. Springer, Cham, December 2017. doi:10.1007/978-3-319-70694-8_15.

CKS11. Jan Camenisch, Stephan Krenn, and Victor Shoup. A framework for practical universally composable zero-knowledge protocols. In Dong Hoon Lee and Xiaoyun Wang, editors, *ASIACRYPT 2011*, volume 7073 of *LNCS*, pages 449–467. Springer, Berlin, Heidelberg, December 2011. doi:10.1007/978-3-642-25385-0_24.

CLNS17. Jan Camenisch, Anja Lehmann, Gregory Neven, and Kai Samelin. UC-secure non-interactive public-key encryption. In Boris Köpf and Steve

Chong, editors, *CSF 2017 Computer Security Foundations Symposium*, pages 217–233. IEEE Computer Society Press, 2017. doi:10.1109/CSF.2017.14.

CMS+23. Sylvain Chatel, Christian Mouchet, Ali Utkan Sahin, Apostolos Pyrgelis, Carmela Troncoso, and Jean-Pierre Hubaux. PELTA - shielding multiparty-FHE against malicious adversaries. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *ACM CCS 2023*, pages 711–725. ACM Press, November 2023. doi:10.1145/3576915.3623139.

CRRV17. Ran Canetti, Srinivasan Raghuraman, Silas Richelson, and Vinod Vaikuntanathan. Chosen-ciphertext secure fully homomorphic encryption. In Serge Fehr, editor, *PKC 2017, Part II*, volume 10175 of *LNCS*, pages 213–240. Springer, Berlin, Heidelberg, March 2017. doi:10.1007/978-3-662-54388-7_8.

CSBB24a. Marina Checri, Renaud Sirdey, Aymen Boudguiga, and Jean-Paul Bultel. On the practical CPAD security of "exact" and threshold FHE schemes and libraries. Cryptology ePrint Archive, Report 2024/116, 2024. URL: https://eprint.iacr.org/2024/116.

CSBB24b. Marina Checri, Renaud Sirdey, Aymen Boudguiga, and Jean-Paul Bultel. On the practical CPA^D security of "exact" and threshold FHE schemes and libraries. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part III*, volume 14922 of *LNCS*, pages 3–33. Springer, Cham, August 2024. doi:10.1007/978-3-031-68382-4_1.

DHLW10. Yevgeniy Dodis, Kristiyan Haralambiev, Adriana López-Alt, and Daniel Wichs. Efficient public-key cryptography in the presence of key leakage. In Masayuki Abe, editor, *ASIACRYPT 2010*, volume 6477 of *LNCS*, pages 613–631. Springer, Berlin, Heidelberg, December 2010. doi:10.1007/978-3-642-17373-8_35.

DN00. Ivan Damgård and Jesper Buus Nielsen. Improved non-committing encryption schemes based on a general complexity assumption. In Mihir Bellare, editor, *CRYPTO 2000*, volume 1880 of *LNCS*, pages 432–450. Springer, Berlin, Heidelberg, August 2000. doi:10.1007/3-540-44598-6_27.

DS13. Ivan Damgård and Alessandra Scafuro. Unconditionally secure and universally composable commitments from physical assumptions. In Kazue Sako and Palash Sarkar, editors, *ASIACRYPT 2013, Part II*, volume 8270 of *LNCS*, pages 100–119. Springer, Berlin, Heidelberg, December 2013. doi:10.1007/978-3-642-42045-0_6.

DS16. Léo Ducas and Damien Stehlé. Sanitization of FHE ciphertexts. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part I*, volume 9665 of *LNCS*, pages 294–310. Springer, Berlin, Heidelberg, May 2016. doi:10.1007/978-3-662-49890-3_12.

FIM+01. Joan Feigenbaum, Yuval Ishai, Tal Malkin, Kobbi Nissim, Martin Strauss, and Rebecca N. Wright. Secure multiparty computation of approximations. In Fernando Orejas, Paul G. Spirakis, and Jan van Leeuwen, editors, *ICALP 2001*, volume 2076 of *LNCS*, pages 927–938. Springer, Berlin, Heidelberg, July 2001. doi:10.1007/3-540-48224-5_75.

FV12. Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Report 2012/144, 2012. URL: https://eprint.iacr.org/2012/144.

Gen09a. Craig Gentry. *A fully homomorphic encryption scheme*. PhD thesis, Stanford University, 2009. crypto.stanford.edu/craig.

Gen09b. Craig Gentry. Fully homomorphic encryption using ideal lattices. In Michael Mitzenmacher, editor, *41st ACM STOC*, pages 169–178. ACM Press, May / June 2009. doi:10.1145/1536414.1536440.

GKO+23. Chaya Ganesh, Yashvanth Kondi, Claudio Orlandi, Mahak Pancholi, Akira Takahashi, and Daniel Tschudi. Witness-succinct universally-composable SNARKs. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part II*, volume 14005 of *LNCS*, pages 315–346. Springer, Cham, April 2023. doi:10.1007/978-3-031-30617-4_11.

GNSJ24. Qian Guo, Denis Nabokov, Elias Suvanto, and Thomas Johansson. Key recovery attacks on approximate homomorphic encryption with non-worst-case noise flooding countermeasures. In Davide Balzarotti and Wenyuan Xu, editors, *USENIX Security 2024*. USENIX Association, August 2024. URL: https://www.usenix.org/conference/usenixsecurity24/presentation/guo-qian.

GOS06. Jens Groth, Rafail Ostrovsky, and Amit Sahai. Perfect non-interactive zero knowledge for NP. In Serge Vaudenay, editor, *EUROCRYPT 2006*, volume 4004 of *LNCS*, pages 339–358. Springer, Berlin, Heidelberg, May / June 2006. doi:10.1007/11761679_21.

HHK+25. Intak Hwang, Yisol Hwang, Miran Kim, Dongwon Lee, and Yongsoo Song. Provably secure approximate computation protocols from CKKS. Cryptology ePrint Archive, Report 2025/395, 2025. URL: https://eprint.iacr.org/2025/395.

HM04. Dennis Hofheinz and Jörn Müller-Quade. Universally composable commitments using random oracles. In Moni Naor, editor, *TCC 2004*, volume 2951 of *LNCS*, pages 58–76. Springer, Berlin, Heidelberg, February 2004. doi:10.1007/978-3-540-24638-1_4.

HMS25. Intak Hwang, Seonhong Min, and Yongsoo Song. Ciphertext-simulatable HE from BFV with randomized evaluation. Cryptology ePrint Archive, Report 2025/203, 2025. URL: https://eprint.iacr.org/2025/203.

HS15. Shai Halevi and Victor Shoup. Bootstrapping for HElib. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part I*, volume 9056 of *LNCS*, pages 641–670. Springer, Berlin, Heidelberg, April 2015. doi:10.1007/978-3-662-46800-5_25.

IP07. Yuval Ishai and Anat Paskin. Evaluating branching programs on encrypted data. In Salil P. Vadhan, editor, *TCC 2007*, volume 4392 of *LNCS*, pages 575–594. Springer, Berlin, Heidelberg, February 2007. doi:10.1007/978-3-540-70936-7_31.

JNO16. Thomas P. Jakobsen, Jesper Buus Nielsen, and Claudio Orlandi. A framework for outsourcing of secure computation. Cryptology ePrint Archive, Report 2016/037, 2016. URL: https://eprint.iacr.org/2016/037.

KLSW24. Hyesun Kwak, Dongwon Lee, Yongsoo Song, and Sameer Wagh. A general framework of homomorphic encryption for multiple parties with non-interactive key-aggregation. In Christina Pöpper and Lejla Batina, editors, *ACNS 2024, Part II*, volume 14584 of *LNCS*, pages 403–430. Springer, Cham, March 2024. doi:10.1007/978-3-031-54773-7_16.

KS25. Kamil Kluczniak and Giacomo Santato. On circuit private, multikey and threshold approximate homomorphic encryption. *CiC*, 2(1):9, 2025. doi:10.62056/a69qgyl7s.

KTZ13. Jonathan Katz, Aishwarya Thiruvengadam, and Hong-Sheng Zhou. Feasibility and infeasibility of adaptively secure fully homomorphic encryption. In Kaoru Kurosawa and Goichiro Hanaoka, editors, *PKC 2013*, volume 7778 of *LNCS*, pages 14–31. Springer, Berlin, Heidelberg, February / March 2013. doi:10.1007/978-3-642-36362-7_2.

KVMGH24. Christian Knabenhans, Alexander Viand, Antonio Merino-Gallardo, and Anwar Hithnawi. VFHE: Verifiable fully homomorphic encryption. In *12th Workshop on Encrypted Computing & Applied Homomorphic Cryptography (WAHC)*, 2024.

LM21. Baiyu Li and Daniele Micciancio. On the security of homomorphic encryption on approximate numbers. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 648–677. Springer, Cham, October 2021. doi:10.1007/978-3-030-77870-5_23.

LMSS22. Baiyu Li, Daniele Micciancio, Mark Schultz, and Jessica Sorrell. Securing approximate homomorphic encryption using differential privacy. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 560–589. Springer, Cham, August 2022. doi:10.1007/978-3-031-15802-5_20.

LTV12. Adriana López-Alt, Eran Tromer, and Vinod Vaikuntanathan. On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption. In Howard J. Karloff and Toniann Pitassi, editors, *44th ACM STOC*, pages 1219–1234. ACM Press, May 2012. doi:10.1145/2213977.2214086.

LWF25. Zeyu Liu, Yunhao Wang, and Ben Fisch. IND-CPA-D of relaxed functional bootstrapping: A new attack, A general fix, and A stronger model. In Chun-Ying Huang, Jyh-Cheng Chen, Shiuh-Pyng Shieh, David Lie, and Véronique Cortier, editors, *ACM CCS 2025*, pages 2907–2921. ACM Press, October 2025. doi:10.1145/3719027.3765170.

MN24. Mark Manulis and Jérôme Nguyen. Fully homomorphic encryption beyond IND-CCA1 security: Integrity through verifiability. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part II*, volume 14652 of *LNCS*, pages 63–93. Springer, Cham, May 2024. doi:10.1007/978-3-031-58723-8_3.

MTBH21. Christian Mouchet, Juan Ramón Troncoso-Pastoriza, Jean-Philippe Bossuat, and Jean-Pierre Hubaux. Multiparty homomorphic encryption from ring-learning-with-errors. *PoPETs*, 2021(4):291–311, October 2021. doi:10.2478/popets-2021-0071.

MTPBH21. Christian Mouchet, Juan Troncoso-Pastoriza, Jean-Philippe Bossuat, and Jean-Pierre Hubaux. Multiparty homomorphic encryption from ring-learning-with-errors. *Proceedings on Privacy Enhancing Technologies*, 2021(4):291–311, 2021.

MW16. Pratyay Mukherjee and Daniel Wichs. Two round multiparty computation via multi-key FHE. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 735–763. Springer, Berlin, Heidelberg, May 2016. doi:10.1007/978-3-662-49896-5_26.

NY90. Moni Naor and Moti Yung. Public-key cryptosystems provably secure against chosen ciphertext attacks. In *22nd ACM STOC*, pages 427–437. ACM Press, May 1990. doi:10.1145/100216.100273.

Par21. Jeongeun Park. Homomorphic encryption for multiple users with less communications. Cryptology ePrint Archive, Report 2021/1085, 2021. URL: https://eprint.iacr.org/2021/1085.

PS24. Alain Passelègue and Damien Stehlé. Low communication threshold fully homomorphic encryption. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part I*, volume 15484 of *LNCS*, pages 297–329. Springer, Singapore, December 2024. doi:10.1007/978-981-96-0875-1_10.

RB07. Phillip Rogaway and Mihir Bellare. Robust computational secret sharing and a unified account of classical secret-sharing goals. In Peng Ning, Sabrina De Capitani di Vimercati, and Paul F. Syverson, editors, *ACM CCS 2007*, pages 172–184. ACM Press, October 2007. doi:10.1145/1315245.1315268.

SBTP+22. Sinem Sav, Jean-Philippe Bossuat, Juan R Troncoso-Pastoriza, Manfred Claassen, and Jean-Pierre Hubaux. Privacy-preserving federated neural network learning for disease-associated cell classification. *Patterns*, 3(5), 2022.

SVdV16. Berry Schoenmakers, Meilof Veeningen, and Niels de Vreede. Trinocchio: Privacy-preserving outsourcing by distributed verifiable computation. In Mark Manulis, Ahmad-Reza Sadeghi, and Steve Schneider, editors, *ACNS 2016*, volume 9696 of *LNCS*, pages 346–366. Springer, Cham, June 2016. doi:10.1007/978-3-319-39555-5_19.

TC16. V Thangam and K Chandrasekaran. Elliptic curve based secure outsourced computation in multi-party cloud environment. In *Security in Computing and Communications: 4th International Symposium, SSCC 2016, Jaipur, India, September 21-24, 2016, Proceedings 4*, pages 199–212. Springer, 2016.

Val76. Leslie G Valiant. Universal circuits (preliminary report). In *Proceedings of the eighth annual ACM symposium on Theory of computing*, pages 196–203, 1976.

Via23. Alexander Viand. *Useable Fully Homomorphic Encryption*. PhD thesis, ETH Zurich, 2023.

XHX+22. Guowen Xu, Xingshuo Han, Shengmin Xu, Tianwei Zhang, Hongwei Li, Xinyi Huang, and Robert H Deng. Hercules: Boosting the performance of privacy-preserving federated learning. *IEEE Transactions on Dependable and Secure Computing*, 20(5):4418–4433, 2022.

A Related Work

HE-based MPC. López-Alt et al. [LTV12] initiated the study on on-the-fly MPC by using multi-key HE that enables the computation on data encrypted under different keys. They show that their construction is secure against a *semi-malicious* adversary who is allowed to use arbitrarily chosen random coins during the execution of the protocol if the underlying MKHE scheme satisfies *semantic security*. Later, Mukherjee and Wichs [MW16] extended the analysis by considering the setting involving *decryption share* which is generated by each participant using their secret key. These decryption shares can then be combined to reconstruct the message without requiring every participant to be online at the same time, thereby improving practicality in distributed settings. To improve the efficiency of MKHE-based MPC, Mouchet et al. [MTBH21] proposed an MPC protocol by adding the dynamic inclusion property of MKHE to threshold HE through a *key-switching* operation. This technique is later generalized to the definition of *multi-group HE* defined by Kwak et al. [KLSW24] by clustering participants into groups. More recently, Cheon et al. [CCKY25] further improved the efficiency by making the ciphertext size independent of the number of participants. Still, we note that the conclusion from these later works is that, with HE alone, one can construct an MPC protocol that is secure against a semi-malicious or a semi-honest adversary (also known as honest-but-curious adversary), without further discussing the security against a fully malicious adversary. Additionally, these works typically focus on the privacy of inputs provided by the clients and assume that the server evaluating the circuit is corrupted by default without further discussing the privacy of the inputs provided by the server.

Composition of HE and Zero-Knowledge Arguments. The composition of (zk)SNARKs or NIZKs with HE has been studied in the literature multiple times, primarily to achieve *chosen-ciphertext* (CCA) security for HE. Boneh et al. [BSW12] employ the Naor–Yung paradigm [NY90] using a *two-pass* structure: the same message is encrypted twice under independent randomness, and a NIZK proof is appended to attest to the consistency of the two ciphertexts. The evaluation similarly follows a two-pass structure, where the evaluation algorithm is applied separately to each component of the ciphertext pair from Naor–Yung paradigm, with a SNARG proof attesting to the correctness of the evaluation. Later, Canetti et al. [CRRV17] proposed a similar construction based on the Naor–Yung approach, but replaced the NIZK with a zkSNARK to improve efficiency. Both works prove that their transforms yield CCA1-secure HE schemes. Viand et al. [Via23, KVMGH24] introduced the notion of *verifiable FHE*, where a CPA-secure FHE scheme is combined with a SNARK that proves only the correctness of the evaluation. However, their construction offers weaker guarantees than standard CCA1 security, as it does not prevent malleability of ciphertexts prior to evaluation. More recently, Manulis and Nguyen [MN24] revisited the Naor–Yung-based approach [BSW12, CRRV17], but used a *single-pass* evaluation strategy instead of two. They showed that this approach achieves the vCCA

security notion, which is stronger than CCA1 but slightly weaker than full CCA2. Atapoor et al. [ABPS24] proposed a verifiable HE construction using lattice-based SNARKs. Although they do not provide a formal proof of security, they argue that their scheme inherits security from the CPA-security of the underlying HE and the knowledge soundness of the SNARK.

B Preliminaries

B.1 Universal Composability (UC) with Global Subroutines

In this work, we prove security using the *Universal Composability* (UC) framework introduced by Canetti [Can00]. The main advantage of the UC framework is that protocols are guaranteed to maintain their security in any context, even in the presence of arbitrarily many protocol sessions that run concurrently in an adversarially controlled manner.

B.1.1 System of ITMs

The UC framework is a *simulation-based* framework where security is defined in terms of a comparison between a "real world" versus an "ideal world". In the real world, a multi-party protocol Π is executed among *parties* $\mathcal{P}_1, \ldots, \mathcal{P}_n$ who are modeled as *Interactive Turing Machines* (ITM). An adversary $\mathcal{A}$, who is also modeled as an ITM, carries attacks on the protocol Π. The adversary can corrupt parties, gaining access to their current and past states as well as the contents of all their tapes. Once a party is corrupted, the adversary gains full control over its future actions.

In the ideal world, a trusted *functionality* $\mathcal{F}$, modeled as an ITM and representing the desired security property, interacts with (dummy) parties $\mathcal{P}_1, \ldots, \mathcal{P}_n$. These dummy parties can only forward inputs to them to $\mathcal{F}$ and outputs responses from $\mathcal{F}$ to them as their outputs. The protocol in which $\mathcal{F}$ interacts with the dummy parties is referred as an *ideal protocol* $\mathrm{I}_{\mathcal{F}}$. Additionally, there is an *ideal-process adversary* $\mathcal{S}$, also commonly called the *simulator*. The simulator's capabilities are restricted to corrupting parties, delaying messages, or suppressing messages sent from $\mathcal{F}$ to a party.

An *environment* $\mathcal{Z}$, who is sometimes referred as a *distinguisher* and also modeled as an ITM, chooses protocol inputs to all parties and may read all outputs locally made by the parties and communicate with the adversary. Informally, we say a protocol is UC-secure if there is no environment that can distinguishes between whether it is interacting with the real adversary $\mathcal{A}$ or the ideal-process adversary $\mathcal{S}$.

B.1.2 Global Subroutines

For many protocols, a global setup is needed, for example, a common reference string (CRS) or a random oracle. For this purpose, Badertscher et al. [BCH+20] proposed UC with *global subroutine* (UCGS) by using a transformation named

management protocol [BCH+20, Section 3.1] to allow plain UC framework capture the global setup. A management protocol $\mathsf{Mgmt}[\Pi, \mathcal{G}]$ for a protocol Π and a global subroutine $\mathcal{G}$ is a UC-protocol, such that one instance of $\mathsf{Mgmt}[\Pi, \mathcal{G}]$ behaves like a single instance of Π along with one or more instances of $\mathcal{G}$. Specifically, the incoming communication to $\mathsf{Mgmt}[\Pi, \mathcal{G}]$ specifies a session sid, plus a session identifier for either Π or a specific instance of $\mathcal{G}$. The input is then passed to Π or the respective instance of $\mathcal{G}$.

UCGS can actually be viewed as a hybrid approach between basic UC and *generalized* UC [CDPW07]. Namely, the environment $\mathcal{Z}$ is allowed to invoke a single instance of the challenge protocol Π while it is allowed to invoke arbitrarily many instances of the global subroutine $\mathcal{G}$. This also allows us better flexibility when defining the global subroutine since we no longer need to fix the global functionality for the lifetime of a party.

B.1.3 UC Emulation and Realization

Informally, a protocol Π is said to *UC-emulate* another protocol Ψ in the UC framework if, for any adversary $\mathcal{A}$ attacking it, there exists a simulator $\mathcal{S}$ "attacking" Ψ, such that no *environment* $\mathcal{Z}$ can distinguish between these two executions. Note that we can consider two types of protocols. In the first case, the communication between the parties in this protocol is over an insecure channel. We also refer this as a *bare* model. In the second case, the protocol runs (multiple sessions of) an ideal functionality $\mathcal{F}$ as a subroutine. We refer such a protocol as a *$\mathcal{F}$-hybrid protocol*. For the completeness, in Definitions 7 and 8, we consider a $\mathcal{F}$-hybrid model. Essentially, a bare model can be considered as a $\mathcal{F}$-hybrid model with $\mathcal{F}$ that simply forwards the message.

Definition 7 (UC-Emulation with Global Subroutine [BCH+20, Definition 3.1]). *Let Π, Ψ be protocols and let $\mathcal{G}$ be a global subroutine. Let* Mgmt *be a management protocol. If* $\mathsf{Mgmt}[\Pi, \mathcal{G}]$ *UC-emulates* $\mathsf{Mgmt}[\Psi, \mathcal{G}]$*, then we say that Π UC-emulates Ψ in presence of a global subroutine $\mathcal{G}$. We use the notation*

$$\Delta_{\mathcal{Z}}\left(\{\mathbf{EXEC}^{\mathcal{G}}_{\mathcal{F},\Pi,\mathcal{A},\mathcal{Z}}(\mathbf{x},\lambda,z)\}_{\mathbf{x},\lambda,z};\{\mathbf{EXEC}^{\mathcal{G}}_{\Psi,\mathcal{S},\mathcal{Z}}(\mathbf{x},\lambda,z)\}_{\mathbf{x},\lambda,z}\}\right) \leq \varepsilon(\lambda)$$

to denote the advantage of the environment in distinguishing between these two execution in presence of a global subroutine $\mathcal{G}$.

However, the emulation of a protocol alone may not adequately capture its security. For instance, by trivially setting $\Pi \coloneqq \Psi$ and $\mathcal{S} \coloneqq \mathcal{A}$, the environment $\mathcal{Z}$ is unable to distinguish between the two executions but it does not imply any security. Thus it is essential to consider the *ideal protocol* $\mathrm{I}_{\mathcal{F}}$. We say that a protocol Π *UC-realizes* a functionality $\mathcal{F}$ if the protocol Π UC-emulates the ideal protocol $\mathrm{I}_{\mathcal{F}}$. In Definition 8, we formally define UC-realization for an ideal functionality $\mathcal{F}$.

Definition 8 (UC-realization with Global Subroutine). *Let Π be protocols. Let $\mathcal{F}$ be an ideal functionality. Let $\mathcal{G}$ be a global subroutine. Let* Mgmt *be a*

management protocol. If $\mathsf{Mgmt}[\Pi, \mathcal{G}]$ *UC-emulates* $\mathsf{Mgmt}[\mathrm{I}_{\mathcal{F}}, \mathcal{G}]$ *where* $\mathrm{I}_{\mathcal{F}}$ *is the ideal protocol of* $\mathcal{F}$*, then we say that* Π *UC-realizes* $\mathcal{F}$ *in presence of a global subroutine* $\mathcal{G}$*. We use the notation*

$$\Delta_{\mathcal{Z}}\left(\{\mathbf{REAL}^{\mathcal{G}}_{\mathcal{F},\Pi,\mathcal{A},\mathcal{Z}}(\mathbf{x},\lambda,z)\}_{\mathbf{x},\lambda,z};\{\mathbf{IDEAL}^{\mathcal{G}}_{\mathcal{F}',\mathcal{S},\mathcal{Z}}(\mathbf{x},\lambda,z)\}_{\mathbf{x},\lambda,z}\right) \leq \varepsilon(\lambda)$$

to denote the advantage of the environment $\mathcal{Z}$ *in distinguishing between these two executions in presence of a global subroutine* $\mathcal{G}$*.*

B.1.4 UC Composition Theorem

The UC framework allows us to show the security when multiple protocols are composed for another protocol. We let the protocol Φ be a $\mathcal{F}$-hybrid protocol and let Π be a protocol that UC-realizes an ideal functionality $\mathcal{F}$. The intuition is that we can then construct a *composed protocol* $\Phi^{\mathcal{F}\to\Pi}$ by changing Φ's calls to a session of $\mathcal{F}$ to a session of Π. At a high level, the composition theorem states that running the protocol $\Phi^{\mathcal{F}\to\Pi}$ is *at least as secure* as running the original protocol Φ who invokes the ideal functionality $\mathcal{F}$.

We describe some properties of the protocols described by Canetti [Can00] for the composition theorem to hold. Informally, we say that a protocol Φ is (Π,Ψ)*-compliant*[14] [Can00, Section 6.1] if it does not invokes both protocol Π and Ψ with the same session identifier. We say a protocol is *subroutine-respecting* [Can20, Definition 19] if the subprotocols in it communicate only with parties outside their session through the main protocol. We say a protocol is *subroutine-exposing* [Can20, Definition 21] if the adversary can obtain a list of subroutines invoked by the protocol.

For composition theorem involving global subroutines, we need two additional properties of the protocols. We say that a protocol Π is $\mathcal{G}$*-subroutine-respecting* if its subprotocols are allowed to pass and receive output from $\mathcal{G}$. We say that $\mathcal{G}$ is Π*-regular* if $\mathcal{G}$ is not allowed to spawn new ITMs and from using Π as a subroutine. Note in Corollary 2, we directly state the theorem for UC-realization since it is more needed than emulation.

Corollary 2 (UC Composition with Global Subroutine [BCH+20, Theorem 3.5]). *Let* $\mathcal{G}$ *be a global subroutine. Let* Mgmt *be a management protocol. Let* Φ*,* Ψ *and* Π *be protocols. Assume that*

1. Φ *is subroutine-exposing,* (Π,Ψ)*-compliant,* $(\Pi,\mathsf{Mgmt}[\Pi,\mathcal{G}])$*-compliant, and* $(\Pi,\mathsf{Mgmt}[\Psi,\mathcal{G}])$*-compliant.*
2. $\mathcal{G}$ *is subroutine-respecting and* Π*-regular.*
3. Π *and* Ψ *are* $\mathcal{G}$*-subroutine-respecting.*

If $\mathsf{Mgmt}[\Pi,\mathcal{G}]$ *UC-emulates* $\mathsf{Mgmt}[\Psi,\mathcal{G}]$*, then the composed protocol* $\Phi^{\Psi\to\Pi}$ *UC-emulates protocol* Φ*.*

[14] In the original definition, an addition identity-bound predicate ξ is also introduced. To be a compliant protocol, the extended identities in an extended session of protocol Φ must also satisfy the predicate ξ. This is less related to this thesisso we omit the discussion on this part.

B.2 Global Random Oracle $\mathcal{G}_{\mathsf{RO}}$

In Fig. 13, we depict the *restricted programmable and observable random oracle* (rpoRO) introduced by Camenisch et al. [CDG+18]. The oracle can be queried both by a party ITI $(\mathcal{P}_{\mathsf{pid}}, \mathsf{sid})$ and by the adversary (either the real-world adversary $\mathcal{A}$ or the ideal-world adversary $\mathcal{S}$). The random oracle is *domain-separated*, meaning that every query must be of the form (sid, x). Queries issued by an out-of-session ITI or directly by the adversary are deemed *illegitimate*. The (ideal-process) adversary $\mathcal{S}$ can thus observe the queries made by the real-world adversary $\mathcal{A}$ in other sessions for the purpose of simulation. The adversary may also *program* the random oracle by prescribing specific query-answer pairs. Moreover, parties can check whether a query has been programmed. Intuitively, this feature allows a party to abort the protocol if a (maliciously) programmed query occurs during (real-world) execution. Importantly, in the ideal world, since the parties are dummy, and all their interaction with the random oracle is mediated by the simulator. Thus the environment can issue checks only through corrupted or out-of-session honest parties, which means the simulator fully controls the interface by which the environment interacts with the oracle to check for such programmed queries in the challenge session. Thus this grants $\mathcal{S}$ the ability to program the oracle undetectably in the challenge session, thereby endowing it with greater power than the real-world adversary $\mathcal{A}$, making the functionality useful for proof.

B.3 Global Key Registry $\mathcal{G}_{\mathsf{KRK}}$

Inspired by the *key registration with knowledge* (KRK) setup in [BCNP04, CDPW07], we define a global subroutine $\mathcal{G}_{\mathsf{KRK}}$[15] for the registration and retrieval of public parameters and keys in a multi-group setting, as illustrated in Fig. 14.

To model the multi-group setting, we let a group $\mathcal{P}_{\mathsf{grp}}$ provide inputs to $\mathcal{G}_{\mathsf{KRK}}$ for key aggregation. If non-interactive key aggregation is available (as in Kwak et al. [KLSW24]), a client $\mathcal{P}_{\mathsf{cli}}$ or server $\mathcal{P}_{\mathsf{srv}}$ can provide its input directly, without relying on a group entity. Since key aggregation generally requires the secret keys, this step may need to be executed via an MPC protocol. Indeed, this "sub-functionality" of $\mathcal{G}_{\mathsf{KRK}}$ closely resembles the simple MPC functionality $\mathcal{F}_{\mathsf{MPC}}$ studied in prior works [CD05, LTV12].

We remark that $\mathcal{G}_{\mathsf{KRK}}$ can be viewed as a trusted party for public parameter and key generation, where the adversary's influence is restricted to choosing malicious randomness for corrupted parties. Thus, the adversary remains *semi-malicious* during the key-generation process throughout this work. To further

[15] We note that $\mathcal{G}_{\mathsf{KRK}}$ is Π*-regular*, meaning it communicates only with the calling parties and the subroutines explicitly declared in the protocol, and *subroutine-respecting* with respect to the global subroutine, meaning it never interacts outside its session or beyond the interface defined by the global subroutine. Thus $\mathcal{G}_{\mathsf{KRK}}$ satisfies the preconditions required by the UC composition theorem.

Functionality: $\mathcal{G}_{\mathsf{RO}}$

Parameter: output length λ.
Participants: a party instance $(\mathcal{P}_{\mathsf{pid}}, \mathsf{sid})$ and an adversary.

– $\langle \texttt{Query} : x \rangle$ from a machine $(\mathcal{P}_{\mathsf{pid}}, \mathsf{sid})$ or the adversary:
  1. If there does not exist v such that (x, v) is in the query list Qry.
     (a) Sample v from $\{0,1\}^{\lambda}$.
     (b) Append (x, v) to Qry.
  2. Parse x as (sid', x').
  3. If $\mathsf{sid} \neq \mathsf{sid}'$ or the query is from the adversary, append $((\mathsf{sid}', x'), v)$ to illegitimate query list $\mathsf{Illegal}$ under the label sid'.
  4. Output $\{\texttt{"val"} : v\}$ to the caller.

– $\langle \texttt{Observe} : \mathsf{sid} \rangle$ from the adversary:
  1. Get the illegal trace tr from $\mathsf{Illegal}$ under the label sid.
  2. Output $\{\texttt{"obv"} : \mathsf{sid}, \mathsf{tr}\}$ to the adversary.

– $\langle \texttt{ProgRO} : x, v \rangle$ from the adversary:
  1. If there does not exists v' such that (x, v') in Qry and $v' \neq v$:
     (a) Append (x, v) to Qry.
     (b) Append x to the list of programmed queries Prog.
     (c) Output $\{\texttt{"cfm"} : 1\}$ to the adversary.
  2. Otherwise, output $\{\texttt{"cfm"} : 0\}$ to the adversary.

– $\langle \texttt{IsProg} : x \rangle$ from a machine $(\mathcal{P}_{\mathsf{pid}}, \mathsf{sid})$ or the adversary:
  1. Parse x as (sid', x').
  2. If it is from $(\mathcal{P}_{\mathsf{pid}}, \mathsf{sid})$ and $\mathsf{sid} \neq \mathsf{sid}'$, abort.
  3. Set $b := x \in_? \mathsf{Prog}$.
  4. Output $\{\texttt{"chk"} : b\}$ to $(\mathcal{P}_{\mathsf{pid}}, \mathsf{sid})$ or the adversary.

Fig. 13: A restricted programmable and observable global random oracle (rpoRO) $\mathcal{G}_{\mathsf{RO}}$ described by Camenisch et al. [CDG+18].

capture a *malicious* adversary also in the key-generation process, one may additionally require a zkSNARK proof to ensure well-formed keys, beyond those considered in Sections 3.3 and 3.4. Our definition of $\mathcal{G}_{\mathsf{KRK}}$ is intentionally streamlined to limit the number of hybrid games in the proofs, thereby improving readability.

We assume that $\mathcal{G}_{\mathsf{KRK}}$ follows the pid-*wise corruption* model of [Can20]. During key retrieval, if a party $\mathcal{P}_{\mathsf{pid}}$ is corrupted, the adversary may obtain its secret key for any session by specifying the session identifier sid. In this case, the functionality treats the request as if it were sent by the corrupted party.

Finally, we note that the ideal-process adversary $\mathcal{S}$ can obtain the public key of any uncorrupted party directly from $\mathcal{G}_{\mathsf{KRK}}$, without going through a corrupted party. This reflects the real-world scenario in which public keys are publicly available and not restricted to members of a particular group.

Functionality: $\mathcal{G}_{\mathsf{KRK}}$

Participants: n clients $\mathcal{P}_{\mathsf{cli}_1}, \mathcal{P}_{\mathsf{cli}_2}, \ldots, \mathcal{P}_{\mathsf{cli}_n}$; Initially, subsets of clients $\{\mathcal{P}_{\mathsf{cli}_i}\}_{i \in I}$ with $I \subseteq [n]$ form k groups $\mathcal{P}_{\mathsf{grp}_j}$ with $\mathsf{grp}_j = \{\mathsf{cli}_i\}_{i \in I_j}$ with $I_j \subseteq I$ for $j \in [k]$; a server $\mathcal{P}_{\mathsf{srv}}$; an adversary.

– $\langle \texttt{RegParam} : \mathsf{sid}, \lambda, \kappa \rangle$ from $\mathcal{P}_{\mathsf{cli}}$ or $\mathcal{P}_{\mathsf{srv}}$:
  1. If a parameter has been set for sid, abort.
  2. Otherwise, run $\mathsf{pp} \leftarrow \mathsf{MGHE}.\texttt{PGen}(1^\lambda, 1^\kappa)$ and store pp under label sid.
– $\langle \texttt{RetParam} : \mathsf{sid} \rangle$ from $\mathcal{P}_{\mathsf{cli}}$:
  1. If a parameter pp has not been set for sid, abort.
  2. Otherwise, output $\{\texttt{"param"} : \mathsf{sid}, \mathsf{pp}\}$ at $\mathcal{P}_{\mathsf{cli}}$.
– $\left\langle \texttt{RegKey} : \mathsf{cli}, \mathsf{sid}, \mathsf{pp} \,;\boxed{\gamma} \right\rangle$ from $\mathcal{P}_{\mathsf{cli}}$:
  1. If pp does not match with sid, abort.
  2. If $\mathcal{P}_{\mathsf{cli}}$ is not corrupted, run $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{MGHE}.\texttt{KGen}(\mathsf{pp})$.
  3. If $\mathcal{P}_{\mathsf{cli}}$ is corrupted, run $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{MGHE}.\texttt{KGen}(\mathsf{pp} \,;\gamma)$.
  4. Store $(\mathsf{pk}, \mathsf{sk})$ under label $(\mathsf{cli}, \mathsf{sid})$.
– $\langle \texttt{RetKey} : \mathsf{pid}', \mathsf{sid} \rangle$ from $\mathcal{P}_{\mathsf{pid}}$ or the adversary.
  1. If no key under label $(\mathsf{pid}', \mathsf{sid})$, abort.
  2. If $\mathsf{pid} = \mathsf{pid}'$, output $\{\texttt{"key"} : \mathsf{pid}', \mathsf{sid}, (\mathsf{pk}, \mathsf{sk})\}$ at $\mathcal{P}_{\mathsf{pid}}$.
  3. Otherwise, output $\{\texttt{"key"} : \mathsf{pid}', \mathsf{sid}, \mathsf{pk}\}$ at $\mathcal{P}_{\mathsf{pid}}$ or the adversary.
– $\langle \texttt{JoinKey} : \mathsf{grp}, \mathsf{sid} \rangle$ from $\mathcal{P}_{\mathsf{grp}}$:
  1. If there are $(\mathsf{pk}_i, \mathsf{sk}_i)$ stored under label $(\mathsf{cli}_i, \mathsf{sid})$ for all $\mathsf{cli}_i \in \mathsf{grp}$:
     (a) Run $\mathsf{jpk} \leftarrow \mathsf{MGHE}.\texttt{KJoin}((\mathsf{pk}_i, \mathsf{sk}_i)_{\mathsf{cli}_i \in \mathsf{grp}})$.
     (b) Store jpk under label $(\mathsf{grp}, \mathsf{sid})$.
  2. Otherwise, abort.
– $\langle \texttt{Corrupt} : \mathsf{pid} \rangle$ from the adversary:
  1. Label $\mathcal{P}_{\mathsf{pid}}$ as corrupted.

Fig. 14: Global subroutine $\mathcal{G}_{\mathsf{KRK}}$ with public parameter and key registry functionality. Dot-boxed inputs originate exclusively from the corrupted party (on behalf of the adversary).

B.4 Authenticated Channel $\mathcal{F}_{\mathsf{AUTH}}$

Functionality: $\mathcal{F}_{\mathsf{AUTH}}$

Participants: a party $\mathcal{P}_{\mathsf{pid}}$ and an ideal-process adversary $\mathcal{S}$.

– $\langle \texttt{RegMsg} : \mathsf{msg} \rangle$ from $\mathcal{P}_{\mathsf{pid}}$:
  1. Parse msg as $\{\texttt{"} \cdot \texttt{"} : \mathsf{src}, \mathsf{dst}, \mathsf{sid}, \mathsf{uid}, \cdot\}$.
  2. Register the message msg under the label $(\mathsf{src}, \mathsf{dst}, \mathsf{sid}, \mathsf{uid})$.
– $\langle \texttt{RetMsg} : \mathsf{src}, \mathsf{dst}, \mathsf{sid}, \mathsf{uid} \rangle$ from $\mathcal{P}_{\mathsf{pid}}$:
  1. Find the message msg under the label $(\mathsf{src}, \mathsf{dst}, \mathsf{sid}, \mathsf{uid})$.
  2. If no such message exists, abort.
  3. Otherwise, output msg to $\mathcal{P}_{\mathsf{pid}}$.

Fig. 15: An authenticated channel $\mathcal{F}_{\mathsf{AUTH}}$.

In Fig. 15, we illustrate an authenticated channel $\mathcal{F}_{\mathsf{AUTH}}$ through which parties exchange messages, as a pre-assumption commonly made in existing works on MPC. In contrast to the standard authenticated channel defined by Canetti et al. [Can00], which models point-to-point message delivery, our ideal functionality

$\mathcal{F}_{\mathsf{AUTH}}$ is designed as a registry where parties post and retrieve messages. This abstraction more accurately captures the broadcast-style communication typically used in multi-party settings.

We recall that under pid-wise corruption [Can20], once a party $\mathcal{P}_{\mathsf{pid}}$ is corrupted, the functionality thereafter accepts inputs directly from the adversary, treating them as if they originated from $\mathcal{P}_{\mathsf{pid}}$. In particular, when a party $\mathcal{P}_{\mathsf{pid}}$ submits an input to register a message, the functionality simply logs it under the identifier pid, regardless of whether it was sent by an uncorrupted party or by the adversary on behalf of a corrupted one.

C Full Description of Protocols

Protocol: $\Pi^{\mathsf{SM}}_{\mathsf{OtF\text{-}MPC}}$

Participants: n clients $\mathcal{P}_{\mathsf{cli}_1}, \mathcal{P}_{\mathsf{cli}_2}, \ldots, \mathcal{P}_{\mathsf{cli}_n}$. At a state, a subset of clients $\{\mathcal{P}_{\mathsf{cli}_i}\}_{i\in I}$ with $I \subseteq [n]$ form ℓ groups $\mathcal{P}_{\mathsf{grp}_j}$ with $\mathsf{grp}_j = \{\mathsf{cli}_i\}_{i\in I_j}$ with $I_j \subseteq I$ for $j \in [\ell]$. A server $\mathcal{P}_{\mathsf{srv}}$.

– $\langle \mathtt{Upload} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, m\rangle$ to $\mathcal{P}_{\mathsf{cli}}$:
  1. Call $\mathcal{G}_{\mathsf{KRK}}$ in sid to get jpk of grp.
  2. Call $\mathcal{F}_{\mathsf{MGHE}}$ in sid with m to obtain ciphertext c.
  3. Output $\{\mathtt{"up"} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, c\}$.

– $\big\langle \mathtt{Evaluate} : \mathsf{srv}, \mathsf{sid}, \mathsf{uid}, f, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)_{j\in[\ell]}\big\rangle$ to $\mathcal{P}_{\mathsf{srv}}$:
  1. If $\mathsf{sid} \neq \mathsf{sid}_j$ for any $j \in [\ell]$, abort.
  2. For $j \in [\ell]$ do:
     (a) Call $\mathcal{G}_{\mathsf{KRK}}$ in sid to get jpk_j.
     (b) Retrieve c_j recorded under $(\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)$.
  3. Set $\hat{\mathsf{grp}} = \cup_{i\in[\ell]}\mathsf{grp}_j$.
  4. Call $\mathcal{F}_{\mathsf{MGHE}}$ in sid with $(\mathsf{jpk}_j, c_j)_{j\in[\ell]}$ to obtain the evaluated ciphertext $\hat{c}$.
  5. Output $\big\{\mathtt{"eval"} : \mathsf{srv}, \hat{\mathsf{grp}}, \mathsf{sid}, \mathsf{uid}, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)_{j\in[\ell]}, \hat{c}\big\}$.

– $\langle \mathtt{Retrieve} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}\rangle$ to $\mathcal{P}_{\mathsf{cli}}$:
  1. If $\mathsf{cli} \notin \mathsf{grp}$, abort.
  2. Retrieve message $\{\mathsf{typ} : \cdot, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, \cdot, c\}$.
  3. If $\mathsf{typ} = \mathtt{"up"}$, abort. // Partially decrypt evaluated ctxt only.
  4. Call $\mathcal{F}_{\mathsf{MGHE}}$ in session sid to obtain decryption share d.
  5. Output $\{\mathtt{"sh"} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, d\}$.
  6. Set $S_{\mathtt{sh}} := \varnothing$.
  7. For $\mathsf{cli}' \in \mathsf{grp} \setminus \{\mathsf{cli}\}$ do:
     (a) Receive message $\big\{\mathtt{"sh"} : \mathsf{cli}', \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, d\big\}$.
     (b) Set $S_{\mathtt{sh}} := S_{\mathtt{sh}} \cup \{d\}$.
  8. If $|S_{\mathtt{sh}}| \geq \lceil \frac{|\mathsf{grp}|}{t} \rceil$:
     (a) Call $\mathcal{F}_{\mathsf{MGHE}}$ in session sid with $S_{\mathtt{sh}}$ to obtain a message m.
     (b) Output $\{\mathtt{"ret"} : \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, m\}$.
  9. Otherwise, abort.

Fig. 16: Protocol $\Pi^{\mathsf{SM}}_{\mathsf{OtF\text{-}MPC}}$ in $\mathcal{F}_{\mathsf{MGHE}}$-hybrid model that is secure against a semi-malicious adversary described in Section 3.2.

Protocol: $\Pi^{\mathsf{SE\text{-}1P}}_{\mathsf{OtF\text{-}MPC}}$

Participants: n clients $\mathcal{P}_{\mathsf{cli}_1}, \mathcal{P}_{\mathsf{cli}_2}, \ldots, \mathcal{P}_{\mathsf{cli}_n}$. At a state, a subset of clients $\{\mathcal{P}_{\mathsf{cli}_i}\}_{i\in I}$ with $I \subseteq [n]$ form ℓ groups $\mathcal{P}_{\mathsf{grp}_j}$ with $\mathsf{grp}_j = \{\mathsf{cli}_i\}_{i\in I_j}$ with $I_j \subseteq I$ for $j \in [\ell]$. A server $\mathcal{P}_{\mathsf{srv}}$.

– $\langle \texttt{Upload} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, m\rangle$ to $\mathcal{P}_{\mathsf{cli}}$:
  1. Call $\mathcal{G}_{\mathsf{KRK}}$ in sid with grp to get jpk.
  2. Call $\mathcal{F}_{\mathsf{MGHE}}$ in sid with m to get ciphertext c.
  3. Call $\mathcal{F}_{\mathsf{MGHE}}$ in sid to get randomness ω.
  4. Call $\mathcal{F}^{R_{\mathsf{Enc}}}_{\mathsf{zkSNARK}}$ in sid with instance (jpk, c) and witness (m, ω) to get π
  5. Output $\{\texttt{"up"} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, (c, \pi)\}$ to $\mathcal{F}_{\mathsf{AUTH}}$.

– $\langle \texttt{Evaluate} : \mathsf{srv}, \mathsf{sid}, \mathsf{uid}, f, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)_{j\in[\ell]}\rangle$ to $\mathcal{P}_{\mathsf{srv}}$:
  1. If $\mathsf{sid} \neq \mathsf{sid}_j$ for any $j \in [\ell]$, abort.
  2. For $j \in [\ell]$ do:
     (a) Get the message msg under label $(\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)$ from $\mathcal{F}_{\mathsf{AUTH}}$
     (b) If $\mathsf{msg} = \{\texttt{"up"} : \mathsf{cli}, \mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j, (c_j, \pi_j)\}$.
        (i) Call $\mathcal{G}_{\mathsf{KRK}}$ in sid to get jpk_j of grp_j.
        (ii) Call $\mathcal{F}^{R_{\mathsf{Enc}}}_{\mathsf{zkSNARK}}$ to verify π_j with instance (jpk_j, c_j). Abort if verification fails.
     (c) Else if $\mathsf{msg} = \{\texttt{"eval"} : \mathsf{srv}, \mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j, (\mathsf{grp}_{j'}, \mathsf{sid}_{j'}, \mathsf{uid}_{j'})_{j'\in[\ell']}, (c_j, \pi_j)\}$.
        (i) Call $\mathcal{G}_{\mathsf{KRK}}$ in sid to get $\mathsf{jpk}_{j'}$ of $\mathsf{grp}_{j'}$ for $j' \in [\ell']$.
        (ii) Call $\mathcal{F}^{R_{\mathsf{Eval}}}_{\mathsf{zkSNARK}}$ to verify π_j with instance $((\mathsf{jpk}_{j'}, c_{j'})_{j'\in[\ell']}, c_j)$.
        (iii) Abort if verification fails or any of $c_{j'}$ for $j' \in [\ell']$ is invalid.
  3. Set $\hat{\mathsf{grp}} = \cup_{i\in[\ell]}\mathsf{grp}_j$.
  4. Call $\mathcal{F}_{\mathsf{MGHE}}$ in sid with f and $(\mathsf{jpk}_j, c_j)_{j\in[\ell]}$ to obtain $\hat{c}$.
  5. Call $\mathcal{F}_{\mathsf{MGHE}}$ in sid to obtain randomness σ.
  6. Call $\mathcal{F}^{R_{\mathsf{Eval}}}_{\mathsf{zkSNARK}}$ in sid with instance $((\mathsf{jpk}_j, c_j)_{j\in[\ell]}, \hat{c})$ and witness σ to get π.
  7. Output $\{\texttt{"eval"} : \mathsf{srv}, \hat{\mathsf{grp}}, \mathsf{sid}, \mathsf{uid}, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)_{j\in[\ell]}, \hat{c}, \pi\}$ to $\mathcal{F}_{\mathsf{AUTH}}$.

– $\langle \texttt{Retrieve} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}\rangle$ to $\mathcal{P}_{\mathsf{cli}}$:
  1. If $\mathsf{cli} \notin \mathsf{grp}$, abort.
  2. Get the message msg under label $(\mathsf{grp}, \mathsf{sid}, \mathsf{uid})$ from $\mathcal{F}_{\mathsf{AUTH}}$
  3. Abort if $\mathsf{msg} = \{\texttt{"up"} : \mathsf{cli}', \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, (c, \pi)\}$.
     //Partially decrypt evaluated ciphertext only.
  4. Parse msg as $\{\texttt{"eval"} : \mathsf{srv}, \hat{\mathsf{grp}}, \mathsf{sid}, \mathsf{uid}, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)_{j\in[\ell]}, \hat{c}, \pi\}$.
  5. Call $\mathcal{G}_{\mathsf{KRK}}$ in sid to get jpk_j of grp_j for $j \in [\ell]$.
  6. Call $\mathcal{F}^{R_{\mathsf{Eval}}}_{\mathsf{zkSNARK}}$ in sid to verify π with instance $((\mathsf{jpk}_j, c_j)_{j\in[\ell]}, \hat{c})$.
  7. Abort if verification fails or any input ciphertext c_j is invalid.
  8. Call $\mathcal{G}_{\mathsf{KRK}}$ in sid with cli to get sk.
  9. Call $\mathcal{F}_{\mathsf{MGHE}}$ in sid to get decryption share d.
  10. Call $\mathcal{F}_{\mathsf{MGHE}}$ in sid to get randomness ρ.
  11. Call $\mathcal{F}^{R_{\mathsf{Dec}}}_{\mathsf{zkSNARK}}$ in sid with instance $(\hat{c}, d)$ and witness (sk, ρ) to get π.
  12. Output $\{\texttt{"sh"} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, d, \pi\}$ to $\mathcal{F}_{\mathsf{AUTH}}$.
  13. Set $\mathcal{S}_{\mathtt{sh}} := \varnothing$.
  14. For $\mathsf{cli}' \in \mathsf{grp} \setminus \{\mathsf{cli}\}$ do:
     (a) Get message $\{\texttt{"sh"} : \mathsf{cli}', \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, d, \pi\}$ from $\mathcal{F}_{\mathsf{AUTH}}$.
     (b) Call $\mathcal{F}^{R_{\mathsf{Dec}}}_{\mathsf{zkSNARK}}$ in session sid with (d, π).
     (c) If π is valid, set $\mathcal{S}_{\mathtt{sh}} := \mathcal{S}_{\mathtt{sh}} \cup \{d\}$.
  15. If $|\mathcal{S}_{\mathtt{sh}}| \geq \lceil \frac{|\mathsf{grp}|}{t} \rceil$:
     (a) Call $\mathcal{F}_{\mathsf{MGHE}}$ in sid with $\mathcal{S}_{\mathtt{sh}}$ to get message m.
     (b) Output $\{\texttt{"ret"} : \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, m\}$.
  16. Otherwise, abort.

Fig. 17: Protocol $\Pi^{\mathsf{SE\text{-}1P}}_{\mathsf{OtF\text{-}MPC}}$ in $[\mathcal{F}_{\mathsf{MGHE}}, \mathcal{F}_{\mathsf{zkSNARK}}, \mathcal{F}_{\mathsf{AUTH}}]$-hybrid model that is secure against a (full) malicious adversary described in Section 3.3.

Protocol: $\Pi^{\mathsf{SS\text{-}2P}}_{\mathsf{OtF\text{-}MPC}}$

Participants: n clients $\mathcal{P}_{\mathsf{cli}_1}, \mathcal{P}_{\mathsf{cli}_2}, \ldots, \mathcal{P}_{\mathsf{cli}_n}$. At a state, a subset of clients $\{\mathcal{P}_{\mathsf{cli}_i}\}_{i\in I}$ with $I \subseteq [n]$ form ℓ groups $\mathcal{P}_{\mathsf{grp}_j}$ with $\mathsf{grp}_j = \{\mathsf{cli}_i\}_{i\in I_j}$ with $I_j \subseteq I$ for $j \in [\ell]$. A server $\mathcal{P}_{\mathsf{srv}}$.

– $\langle \texttt{Upload} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, m\rangle$ to $\mathcal{P}_{\mathsf{cli}}$:
1. Call $\mathcal{G}_{\mathsf{KRK}}$ in $(\mathsf{sid}, 1)$ and $(\mathsf{sid}, 2)$ with grp to get jpk_1 and jpk_2.
2. Call $\mathcal{F}_{\mathsf{MGHE}}$ in $(\mathsf{sid}, 1)$ and $(\mathsf{sid}, 2)$ with m to get ciphertexts c_1 and c_2.
3. Call $\mathcal{F}_{\mathsf{MGHE}}$ in $(\mathsf{sid}, 1)$ and $(\mathsf{sid}, 2)$ to get randomness ω_1 and ω_2.
4. Call $\mathcal{F}^{R_{\mathsf{Enc}}}_{\mathsf{NIZK}}$ in sid with instance $(\mathsf{jpk}_1, \mathsf{jpk}_2, c_1, c_2)$ and witness (m, ω_1, ω_2) to get π.
5. Output $\{\texttt{"up"} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, (c_1, c_2, \pi)\}$ to $\mathcal{F}_{\mathsf{AUTH}}$.

– $\langle \texttt{Evaluate} : \mathsf{srv}, \mathsf{sid}, \mathsf{uid}, f, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)_{j\in[\ell]}\rangle$ to $\mathcal{P}_{\mathsf{srv}}$:
1. If $\mathsf{sid} \neq \mathsf{sid}_j$ for any $j \in [\ell]$, abort.
2. For $j \in [\ell]$:
   (a) Get the message msg under label $(\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)$ from $\mathcal{F}_{\mathsf{AUTH}}$
   (b) If $\mathsf{msg} = \{\texttt{"up"} : \mathsf{cli}, \mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j, (c_{j,1}, c_{j,2}, \pi_j)\}$.
      (i) Call $\mathcal{G}_{\mathsf{KRK}}$ in $(\mathsf{sid}, 1)$ and $(\mathsf{sid}, 2)$ to get $\mathsf{jpk}_{j,1}$ and $\mathsf{jpk}_{j,2}$ of grp_j.
      (ii) Call $\mathcal{F}^{R_{\mathsf{Enc}}}_{\mathsf{NIZK}}$ to verify π_j with instance $(\mathsf{jpk}_j, c_{j,1}, c_{j,2})$. Abort if verification fails.
   (c) Else if $\mathsf{msg} = \{\texttt{"eval"} : \mathsf{srv}, \mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j, (\mathsf{grp}_{j'}, \mathsf{sid}_{j'}, \mathsf{uid}_{j'})_{j'\in[\ell']}, (c_{j,1}, c_{j,2}, \pi_j)\}$.
      (i) Call $\mathcal{G}_{\mathsf{KRK}}$ in $(\mathsf{sid}, 1)$ and $(\mathsf{sid}, 2)$ to get $\mathsf{jpk}_{j',1}$ and $\mathsf{jpk}_{j',2}$ of $\mathsf{grp}_{j'}$ for $j' \in [\ell']$.
      (ii) Call $\mathcal{F}^{R_{\mathsf{Eval}}}_{\mathsf{NIZK}}$ to verify π_j with instance $((\mathsf{jpk}_{j',1}, \mathsf{jpk}_{j',2}, c_{j',1}, c_{j'2})_{j'\in[\ell']}, c_{j,1}, c_{j,2})$.
      (iii) Abort if verification fails or any of $c_{j',1}$ or $c_{j',2}$ for $j' \in [\ell']$ is invalid.
3. Set $\hat{\mathsf{grp}} = \cup_{i\in[\ell]}\mathsf{grp}_j$.
4. For $k \in \{1, 2\}$: Call $\mathcal{F}_{\mathsf{MGHE}}$ in (sid, k) with f and $(\mathsf{jpk}_{j,k}, c_{j,k})$ to get $\hat{c}_k$.
5. Call $\mathcal{F}_{\mathsf{MGHE}}$ in $(\mathsf{sid}, 1)$ and $(\mathsf{sid}, 2)$ to get randomness σ_1 and σ_2.
6. Call $\mathcal{F}^{R_{\mathsf{Eval}}}_{\mathsf{NIZK}}$ in sid with $((\mathsf{jpk}_{j,1}, c_{j,1}, \mathsf{jpk}_{j,2}, c_{j,2})_{j\in[\ell]}, \hat{c}_1, \hat{c}_2)$ and (σ_1, σ_2) to get π.
7. Output $\{\texttt{"eval"} : \mathsf{srv}, \hat{\mathsf{grp}}, \mathsf{sid}, \mathsf{uid}, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)_{j\in[\ell]}, (\hat{c}_1, \hat{c}_2, \pi)\}$ to $\mathcal{F}_{\mathsf{AUTH}}$.

– $\langle \texttt{Retrieve} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}\rangle$ to $\mathcal{P}_{\mathsf{cli}}$:
1. If $\mathsf{cli} \notin \mathsf{grp}$, abort.
2. Get the message msg under label $(\mathsf{grp}, \mathsf{sid}, \mathsf{uid})$ from $\mathcal{F}_{\mathsf{AUTH}}$
3. Abort if $\mathsf{msg} = \{\texttt{"up"} : \mathsf{cli}', \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, (c, \pi)\}$.
   //Partially decrypt evaluated ciphertext only.
4. Parse msg as $\{\texttt{"eval"} : \mathsf{srv}, \hat{\mathsf{grp}}, \mathsf{sid}, \mathsf{uid}, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)_{j\in[\ell]}, (\hat{c}_1, \hat{c}_2, \pi)\}$.
5. Call $\mathcal{G}_{\mathsf{KRK}}$ in $(\mathsf{sid}, 1)$ and $(\mathsf{sid}, 2)$ to get $\mathsf{jpk}_{j,1}$ and $\mathsf{jpk}_{j,2}$ of grp_j for $j \in [\ell]$.
6. Call $\mathcal{F}^{R_{\mathsf{Eval}}}_{\mathsf{NIZK}}$ in session sid to verify π with instance $((\mathsf{jpk}_{j,1}, \mathsf{jpk}_{j,2}, c_{j,1}, c_{j,2})_{j\in[\ell]}, \hat{c}_1, \hat{c}_2)$.
7. Abort if verification fails, or any input ciphertexts $(c_{j,1}, c_{j,2}, \pi_j)$ is invalid.
8. Call $\mathcal{G}_{\mathsf{KRK}}$ in $(\mathsf{sid}, 1)$ and $(\mathsf{sid}, 2)$ with cli to get the secret key sk_1 and sk_2.
9. Call $\mathcal{F}_{\mathsf{MGHE}}$ in session $(\mathsf{sid}, 1)$ and $(\mathsf{sid}, 2)$ to obtain decryption shares d_1 and d_2.
10. Call $\mathcal{F}_{\mathsf{MGHE}}$ in session $(\mathsf{sid}, 1)$ and $(\mathsf{sid}, 2)$ to obtain the randomness ρ_1 and ρ_2.
11. Call $\mathcal{F}^{R_{\mathsf{Dec}}}_{\mathsf{NIZK}}$ in session sid with $(\hat{c}_1, \hat{c}_2, d_1, d_2)$ and $(\mathsf{sk}_1, \mathsf{sk}_2, \rho_1, \rho_2)$ to get π
12. Output $\{\texttt{"sh"} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, d_1, d_2, \pi\}$ to $\mathcal{F}_{\mathsf{AUTH}}$.
13. Set $S_{\mathtt{sh}_1} := \varnothing$ and $S_{\mathtt{sh}_2} := \varnothing$.
14. For $\mathsf{cli}' \in \mathsf{grp} \setminus \{\mathsf{cli}\}$ do:
   (a) Receive message $\{\texttt{"sh"} : \mathsf{cli}', \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, d_1, d_2, \pi\}$ from $\mathcal{F}_{\mathsf{AUTH}}$.
   (b) Call $\mathcal{F}^{R_{\mathsf{Dec}}}_{\mathsf{NIZK}}$ in session sid with (d_1, d_2, π).
   (c) If π is valid, set $S_{\mathtt{sh}_1} := S_{\mathtt{sh}_1} \cup \{d_1\}$ and $S_{\mathtt{sh}_2} := S_{\mathtt{sh}_2} \cup \{d_2\}$.
15. If $|S_{\mathtt{sh}_1}| < \lceil \frac{|\mathsf{grp}|}{t} \rceil$, abort. Otherwise:
   (a) For $u \in \{1, 2\}$: Call $\mathcal{F}_{\mathsf{MGHE}}$ in session (sid, u) with $S_{\mathtt{sh}_u}$ to obtain a message m_u.
   (b) If $\boxed{m_1 \neq m_2}$ $|m_1 - m_2| \leq \mathsf{Estimate}(\cdot)$, output $\{\texttt{"ret"} : \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, m\}$, and abort otherwise.

Fig. 18: Protocol $\Pi^{\mathsf{SS\text{-}2P}}_{\mathsf{OtF\text{-}MPC}}$ in the $[\mathcal{F}_{\mathsf{MGHE}}, \mathcal{F}_{\mathsf{NIZK}}, \mathcal{F}_{\mathsf{AUTH}}]$-hybrid model, secure against a (full) malicious adversary described in Section 3.4. Framed parts appear exclusively when MGHE is exact and perfectly correct. Highlighted parts appear only when MGHE is approximate or imperfectly correct.

D Full Proofs of Theorems

D.1 Full proof of Theorem 1

Theorem 1. *Let* MGHE *be a multi-group homomorphic encryption scheme with threshold factor t. Then, the protocol Π_{MGHE} UC-realizes the functionality $\mathcal{F}_{\mathsf{MGHE}}$ in the presence of a global key registry $\mathcal{G}_{\mathsf{KRK}}$ against a partial-decryption manipulating semi-malicious adversary that non-adaptively corrupts the clients and the server, provided that:*

- MGHE *satisfies robustness (*Rob*), and the adversary corrupts fewer than $\frac{|\mathsf{grp}|}{t}$ clients in any group.*
- MGHE *is* IND-CPA^{pD}*-secure, and there exists at least one uncorrupted client who provides input to the computation, such that the group* grp *under which it provides input contains no more than $\frac{|\mathsf{grp}|}{t}$ corrupted clients.*
- MGHE *satisfies* SIM-PDEC *security with respect to a simulator* $\mathtt{Sim}_{th}$*.*

Proof. We consider n clients $\mathcal{P}_{\mathsf{cli}_1}, \ldots, \mathcal{P}_{\mathsf{cli}_n}$ and a server $\mathcal{P}_{\mathsf{srv}}$ in the system. In the setup phase, we let a client $\mathcal{P}_{\mathsf{cli}_i}$, for some $i \in [n]$, be first activated with the input $\langle \mathtt{RegParam} : \mathsf{sid} \rangle$ to initialize the system's public parameter at $\mathcal{G}_{\mathsf{KRK}}$. Following this, each client $\mathcal{P}_{\mathsf{cli}_i}$ receives the input $\langle \mathtt{RegKey} : \mathsf{cli}, \mathsf{sid} \rangle$ to generate their respective key pairs $(\mathsf{pk}_i, \mathsf{sk}_i)$ at $\mathcal{G}_{\mathsf{KRK}}$.

Next, we form k initial groups $\mathsf{grp}_1, \ldots, \mathsf{grp}_k$, where each group $\mathsf{grp}_j = \{\mathsf{cli}_i\}_{i \in I_j}$ includes a subset of clients indexed by $I_j \subseteq [n]$ for $j \in [k]$. For every group, the corresponding "virtual" group entity $\mathcal{P}_{\mathsf{grp}_j}$ is provided with input $\langle \mathtt{JoinKey} : \mathsf{grp}_j, \mathsf{sid} \rangle$ to compute joint public key jpk_j at $\mathcal{G}_{\mathsf{KRK}}$.

Then the adversary corrupts a subset of clients by sending backdoor messages and queries $\mathcal{G}_{\mathsf{KRK}}$ to access the secret keys sk_i of the corrupted clients. Notably, during this setup phase, $\mathcal{G}_{\mathsf{KRK}}$ does these operations for both of the real and ideal worlds. Thus these two worlds remain indistinguishable to the environment $\mathcal{Z}$ at this stage.

To prove the security of the protocol, we then define the following sequence of hybrids:

- $\mathbf{REAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\Pi_{\mathsf{MGHE}}, \mathcal{A}, \mathcal{Z}}$: This represents the execution of the protocol Π_{MGHE} in the real world and an adversary $\mathcal{A}$ attacking the protocol.
- $\mathbf{HYB}^{\mathcal{G}_{\mathsf{KRK}}}_1$: In this hybrid, we keep track of the message-ciphertext tuples (m, c) that are encrypted and evaluated, as well as the decryption shares d that are honestly generated with respect to (m, c). When a client combines the decryption shares, if it has obtained a sufficient number of honest decryption shares, it outputs m. For schemes on approximate number, it outputs $m + \delta + \sigma + \sum_{i=1}^{|\mathsf{grp}|/t} \rho_i$ where δ is calculated with $\mathsf{GetEncNoise}(\cdot)$ and $\mathsf{GetEvalNoise}(\cdot)$.
- $\mathbf{HYB}^{\mathcal{G}_{\mathsf{KRK}}}_2$: In this hybrid, whenever an uncorrupted client $\mathcal{P}_{\mathsf{cli}}$ is activated with the input $\langle \mathtt{Encrypt} : \mathsf{grp}, \mathsf{sid}, \mathsf{jpk}, m \rangle$, we let the client outputs $c \leftarrow \mathsf{MGHE}.\mathtt{Enc}(\mathsf{jpk}, 0^{|m|})$.
- $\mathbf{IDEAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\mathcal{F}_{\mathsf{MGHE}}, \mathcal{S}_{\mathsf{MGHE}}, \mathcal{Z}}$: This is the ideal execution where the ideal functionality $\mathcal{F}_{\mathsf{MGHE}}$ is attached to the simulator $\mathcal{S}_{\mathsf{MGHE}}$ and interacts with dummy client $\mathcal{P}_{\mathsf{cli}_1}, \ldots, \mathcal{P}_{\mathsf{cli}_n}$.

Claim 1.1. *If* MGHE *satisfies robustness, then* $\mathbf{REAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\Pi_{\mathsf{MGHE}},\mathcal{A},\mathcal{Z}}$ *and* $\mathbf{HYB}^{\mathcal{G}_{\mathsf{KRK}}}_{1}$ *are computationally indistinguishable.*

Proof. We prove the claim by showing that if there is an environment $\mathcal{Z}$ that distinguishes between $\mathbf{REAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\Pi_{\mathsf{MGHE}},\mathcal{A},\mathcal{Z}}$ and $\mathbf{HYB}^{\mathcal{G}_{\mathsf{KRK}}}_{1}$, then we can use it to construct an adversary $\mathcal{A}_{rob}$ that breaks the robustness of MGHE. For this, we let $\mathcal{A}_{cpa}$ simulate the interaction between the environment $\mathcal{Z}$ and the two worlds.

We let the adversary $\mathcal{A}_{rob}$ form the same groups $\mathsf{grp}_1, \ldots, \mathsf{grp}_k$ by specifying the index sets $I_1, \ldots, I_k$ in the $\mathrm{G}^{\mathrm{Rob}}_{\mathsf{MGHE}}$ game and corrupts the same clients by specifying the index set $I_{\mathcal{A}}$ as in the setup phase. When a client queries $\mathcal{G}_{\mathsf{KRK}}$ to retrieve the group public key, we let $\mathcal{A}_{rob}$ provides the one generated by $\mathrm{G}^{\mathrm{Rob}}_{\mathsf{MGHE}}$. Similarly, when the adversary $\mathcal{A}$ inputs $\mathcal{G}_{\mathsf{KRK}}$ for the key pairs of corrupted clients, we let $\mathcal{A}_{rob}$ respond with $(\mathsf{pk}_i, \mathsf{sk}_i)$ for $i \in I_{\mathcal{A}}$, which is also generated by the game $\mathrm{G}^{\mathrm{Rob}}_{\mathsf{MGHE}}$.

- **On input** $\langle \mathtt{Encrypt} : \mathsf{grp}, \mathsf{sid}, \mathsf{jpk}_j, m \rangle$ **to an uncorrupted client** $\mathcal{P}_{\mathsf{cli}}$: We then let $\mathcal{A}$ honestly sample a random coin r from the distribution and query its oracle $\mathcal{O}_{\mathsf{Enc}}$ with $(m; \omega)$. We observe that $\mathcal{O}_{\mathsf{Enc}}$ does $c \leftarrow \mathsf{MGHE}.\mathtt{Enc}(\mathsf{jpk}_j, m; \omega)$, which is exactly the behavior of an uncorrupted client in both of these worlds. Thus we write c from $\mathcal{O}_{\mathsf{Enc}}$ as output of $\mathcal{P}_{\mathsf{cli}}$.
- **On input** $\langle \mathtt{Encrypt} : \mathsf{grp}, \mathsf{sid}, \mathsf{jpk}_j, m; \omega \rangle$ **to a corrupted client** $\mathcal{P}_{\mathsf{cli}}$: We similarly let $\mathcal{A}$ query its oracle $\mathcal{O}_{\mathsf{Enc}}$ with $(m; \omega)$ where ω is the random coin selected by $\mathcal{Z}$. Then upon getting c from $\mathcal{O}_{\mathsf{Enc}}$, we write it as output of $\mathcal{P}_{\mathsf{cli}}$.
- **On input** $\langle \mathtt{Evaluate} : \mathsf{sid}, f, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{jpk}_j, c_j)_{j \in [\ell]}; \sigma \rangle$ **to a corrupted server** $\mathcal{P}_{\mathsf{srv}}$: We observe by definition of these two worlds, all ciphertexts to be evaluated must either be encrypted by the clients or result from an evaluation performed by the server. This implies that in $\mathrm{G}^{\mathrm{Rob}}_{\mathsf{MGHE}}$, these ciphertexts are all entries in the lookup table T. Thus, we let $\mathcal{A}_{rob}$ identify these ciphertexts in the lookup table T by specifying an index set L and query the oracle $\mathcal{O}_{\mathsf{Eval}}$ with $(f, L; \sigma)$. Observe that the oracle $\mathcal{O}_{\mathsf{Eval}}$ returns the ciphertext $\hat{c} \leftarrow \mathsf{MGHE}.\mathtt{Eval}(f, (T[l].\mathsf{jpk}, T[l].c)_{l \in L}; \sigma)$. This is exactly the behavior of the server in both of these worlds. Thus we write $\hat{c}$ as the output of $\mathcal{P}_{\mathsf{srv}}$.
- **On input** $\langle \mathtt{Share} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, c \rangle$ **to an uncorrupted** $\mathcal{P}_{\mathsf{cli}}$: Observe that we require that c must be either encrypted by clients or an evaluation performed by the server. Thus we similarly let $\mathcal{A}_{rob}$ identify the index l of c in the lookup table T of $\mathrm{G}^{\mathrm{Rob}}_{\mathsf{MGHE}}$. Then we let $\mathcal{A}_{rob}$ query $\mathcal{O}_{\mathsf{Dec}}$ with l and the index i representing the client $\mathcal{P}_{\mathsf{cli}}$. Note that $\mathcal{O}_{\mathsf{Dec}}$ does $d \leftarrow \mathsf{MGHE}.\mathtt{PDec}(\mathsf{sk}, c)$ for where sk is the secret key of the client, which reflects the behavior of the client in these two worlds. Thus we write d as the output of $\mathcal{P}_{\mathsf{cli}}$.
- **On input** $\langle \mathtt{Share} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, c; \rho \rangle$ **to a corrupted** $\mathcal{P}_{\mathsf{cli}}$: We similarly let $\mathcal{A}_{rob}$ identify the index l of c in the lookup table T of $\mathrm{G}^{\mathrm{Rob}}_{\mathsf{MGHE}}$. Then we let $\mathcal{A}_{rob}$ query $\mathcal{O}_{\mathsf{Dec}}$ with l and also with ρ. Note that $\mathcal{O}_{\mathsf{Dec}}$ does $d \leftarrow \mathsf{MGHE}.\mathtt{PDec}(\mathsf{sk}, c; \rho)$ for where sk is the secret key of the client, which reflects the behavior of a corrupted client in these two worlds. Thus we write d as the output of $\mathcal{P}_{\mathsf{cli}}$.

We observe that in $\mathbf{HYB}^{\mathcal{G}_{\mathsf{KRK}}}_{1}$, when an (uncorrupted) client $\mathcal{P}_{\mathsf{cli}}$ is activated to combine the decryption shares output by clients in a group grp' for a message m

that is intended for the group grp, where $\mathsf{grp}' \subseteq \mathsf{grp}$. The message m is output if at least $\frac{|\mathsf{grp}|}{t}$ decryption shares are honestly obtained and output by clients in grp'.

It follows that the environment $\mathcal{Z}$ can only distinguish between these two worlds if the algorithm $\mathsf{MGHE}.\mathtt{Combine}(\cdot)$ reconstructs a message different from m using the same set of decryption shares from grp' in $\mathbf{REAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\Pi_{\mathsf{MGHE}},\mathcal{A},\mathcal{Z}}$. This corresponds to the winning condition of $\mathrm{G}^{\mathrm{Rob}}_{\mathsf{MGHE}}$. Thus we let $\mathcal{A}_{rob}$ to output the same decryption shares from grp' to the game $\mathrm{G}^{\mathrm{Rob}}_{\mathsf{MGHE}}$.

$$\Delta_{\mathcal{Z}}\left(\mathbf{REAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\Pi_{\mathsf{MGHE}},\mathcal{A},\mathcal{Z}}, \mathbf{HYB}^{\mathcal{G}_{\mathsf{KRK}}}_{1}\right) \leq \mathbf{Adv}^{\mathrm{Rob}}_{\mathsf{MGHE}}(\mathcal{A}_{rob}). \tag{7}$$

□

Claim 1.2. *If* MGHE *is* IND-CPA^{pD} *secure,* $\mathbf{HYB}^{\mathcal{G}_{\mathsf{KRK}}}_{1}$ *and* $\mathbf{HYB}^{\mathcal{G}_{\mathsf{KRK}}}_{2}$ *are computationally indistinguishable.*

Proof. We prove the claim by demonstrating that if an environment $\mathcal{Z}$ can distinguish between $\mathbf{HYB}^{\mathcal{G}_{\mathsf{KRK}}}_{1}$ and $\mathbf{HYB}^{\mathcal{G}_{\mathsf{KRK}}}_{2}$, then we can use $\mathcal{Z}$ to construct an adversary $\mathcal{A}_{cpa}$ to break the IND-CPA^{pD} security of MGHE. For this, we let $\mathcal{A}_{cpa}$ simulate the interaction between the environment $\mathcal{Z}$ and the two worlds. We let $\mathcal{A}_{cpa}$ follow a similar setup as in proof of Claim 1.1 but interacts with the game $\mathrm{G}^{\text{IND-CPA}^{\mathrm{pD}}}_{\mathsf{MGHE}}$ instead. Then $\mathcal{A}_{cpa}$ handles its oracle queries as follows to simulate the responses based on the messages that the environment $\mathcal{Z}$ delivers to each party.

– **On input** $\langle \mathtt{Encrypt} : \mathsf{grp}, \mathsf{sid}, \mathsf{jpk}_j, m\rangle$ **to an uncorrupted client** $\mathcal{P}_{\mathsf{cli}}$: We consider the following two cases for this input.
  1. If the group grp_j is not corrupted (i.e., $|\mathsf{grp}_j \cap \mathsf{Corrupt}| < \frac{|\mathsf{grp}_j|}{t}$), we let $\mathcal{A}_{cpa}$ query its encryption oracle $\mathcal{O}_{\mathsf{Enc}}$ with the tuple $(\mathsf{jpk}_j, m_0, m_1)$, where $m_0 = m$ and $m_1 = 0^{|m|}$ for the m chosen by $\mathcal{Z}$. When $\mathcal{O}_{\mathsf{Enc}}$ responds with $c \leftarrow \mathsf{MGHE}.\mathtt{Enc}(\mathsf{jpk}_j, m)$, this simulates the behavior of an uncorrupted client in the world $\mathbf{HYB}^{1\,\mathcal{G}_{\mathsf{KRK}}}_{\Pi\mathsf{MGHE},\mathcal{A},\mathcal{Z}}$, where m is encrypted, and the encryption is output by $\mathcal{P}_{\mathsf{cli}}$. On the other hand, if $\mathcal{O}_{\mathsf{Enc}}$ returns $c \leftarrow \mathsf{MGHE}.\mathtt{Enc}(\mathsf{jpk}_j, 0^{|m|})$, it represents the scenario where the dummy party $\mathcal{P}_{\mathsf{cli}}$ outputs the encryption of $0^{|m|}$, which is returned from $\mathcal{F}_{\mathsf{MGHE}}$ and generated by $\mathcal{S}_{\mathsf{MGHE}}$. Accordingly, we write c returned by $\mathcal{O}_{\mathsf{Enc}}$ as the output of the client $\mathcal{P}_{\mathsf{cli}}$.
  2. Otherwise, if grp_j is corrupted, we let $\mathcal{A}_{cpa}$ query its encryption oracle $\mathcal{O}_{\mathsf{Enc}}$ with the tuple $(\mathsf{jpk}_j, m; \omega)$, where ω is *honestly* sampled from the distribution since we are simulating the behavior of an *uncorrupted* client. Note that the oracle $\mathcal{O}_{\mathsf{Enc}^\star}$ returns $c \leftarrow \mathsf{MGHE}.\mathtt{Enc}(\mathsf{jpk}_j, m; \omega)$. In $\mathbf{HYB}^{\mathcal{G}_{\mathsf{KRK}}}_{1}$, this is exactly the behavior of an uncorrupted client. Also, in $\mathbf{HYB}^{\mathcal{G}_{\mathsf{KRK}}}_{2}$, this is what the simulator $\mathcal{S}_{\mathsf{MGHE}}$ does when it is given the actual message m by $\mathcal{F}_{\mathsf{MGHE}}$. Thus, we then write c from $\mathcal{O}_{\mathsf{Enc}^\star}$ as the output of the client $\mathcal{P}_{\mathsf{cli}}$.

– **On input** $\langle \texttt{Encrypt} : \mathsf{grp}, \mathsf{sid}, \mathsf{jpk}_j, m; \omega \rangle$ **to a corrupted client** $\mathcal{P}_{\mathsf{cli}}$: We let $\mathcal{A}_{cpa}$ query its oracle with $(m; \omega)$, where ω is the random coins chosen by $\mathcal{Z}$. The oracle $\mathcal{O}_{\mathsf{Enc}^\star}$ returns $c \leftarrow \mathsf{MGHE}.\texttt{Enc}(\mathsf{jpk}_j, m; \omega)$. In $\mathbf{HYB}_1^{\mathcal{G}_{\mathsf{KRK}}}$, this input from $\mathcal{Z}$ is delivered to the adversary $\mathcal{A}$, who then computes $c \leftarrow \mathsf{MGHE}.\texttt{Enc}(\mathsf{jpk}_j, m; \omega)$ using the random coins r provided by $\mathcal{Z}$, and writes c as the output of a corrupted client. Similarly, in $\mathbf{HYB}_2^{\mathcal{G}_{\mathsf{KRK}}}$, when the simulator $\mathcal{S}_{\mathsf{MGHE}}$ sends the input to $\mathcal{F}_{\mathsf{MGHE}}$, the functionality also performs the encryption honestly using the random coins chosen by $\mathcal{Z}$. The simulator $\mathcal{S}_{\mathsf{MGHE}}$ then writes c received from $\mathcal{F}_{\mathsf{MGHE}}$ as the output of the corrupted client. Accordingly, we write c from $\mathcal{O}_{\mathsf{Enc}^\star}$ as the output of a corrupted client $\mathcal{P}_{\mathsf{cli}}$.
– **On input** $\langle \texttt{Evaluate} : \mathsf{sid}, f, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{jpk}_j, c_j)_{j \in [\ell]}; \sigma \rangle$ **to a corrupted server** $\mathcal{P}_{\mathsf{srv}}$: Note that by definition, all ciphertexts to be evaluated must either be encrypted by the clients or result from an evaluation performed by the server. This implies that in $\mathrm{G}_{\mathsf{MGHE}}^{\text{IND-CPA}^{\text{PD}}}$, these ciphertexts are all entries in the lookup table T. Thus, we let $\mathcal{A}_{cpa}$ first identify these ciphertexts in the lookup table T by specifying an index set L and query the oracle $\mathcal{O}_{\mathsf{Eval}}$ with the circuit f, the index set L, and the random coin σ. If $\perp$ is returned, $\mathcal{A}_{rob}$ aborts, and nothing is written on any tape. Otherwise, the oracle $\mathcal{O}_{\mathsf{Eval}}$ returns the ciphertext $\hat{c} \leftarrow \mathsf{MGHE}.\texttt{Eval}(f, (\mathsf{T}[l].\mathsf{jpk}, \mathsf{T}[l].c)_{l \in L})$.

  In $\mathbf{HYB}_1^{\mathcal{G}_{\mathsf{KRK}}}$, since we consider a semi-malicious adversary, the adversary $\mathcal{A}$ evaluates the circuit f honestly to obtain the ciphertext $\hat{c}$. Similarly, in $\mathbf{HYB}_2^{\mathcal{G}_{\mathsf{KRK}}}$, by the definition of $\mathcal{F}_{\mathsf{MGHE}}$, when the server is corrupted, the functionality also evaluates the circuit honestly. After $\hat{c}$ is obtained, it is written as the output of a corrupted client. Thus, the behavior of these two executions is identical for this input and aligns with the behavior of $\mathcal{O}_{\mathsf{Eval}}$. Accordingly, we write $\hat{c}$ from $\mathcal{O}_{\mathsf{Eval}}$ as the output of the corrupted server $\mathcal{P}_{\mathsf{srv}}$.
– **On input** $\langle \texttt{Share} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, c \rangle$ **to a uncorrupted** $\mathcal{P}_{\mathsf{cli}}$: We let $\mathcal{A}_{cpa}$ query it oracle $\mathcal{O}_{\mathsf{Dec}}$ with an index l identifying the position of c in the lookup table T. If the oracle returns $\perp$, then $\mathcal{A}_{cpa}$ aborts the query. This indicates that the messages being decrypted in $\mathbf{HYB}_1^{\mathcal{G}_{\mathsf{KRK}}}$ and $\mathbf{HYB}_2^{\mathcal{G}_{\mathsf{KRK}}}$ differ. Consequently, after the execution, the environment $\mathcal{Z}$ can trivially distinguish between these two worlds by inspecting the final output tape of a client.

  Otherwise, the oracle $\mathcal{O}_{\mathsf{Dec}}$ returns $\{d_i\}_{\mathsf{cli}_i \in \mathsf{grp}}$ where $d_i \leftarrow \mathsf{MGHE}.\texttt{PDec}(\mathsf{sk}_i, c)$ for $\mathsf{cli}_i \in \mathsf{grp}$. Note that in both of the worlds $\mathbf{HYB}_1^{\mathcal{G}_{\mathsf{KRK}}}$ and $\mathbf{HYB}_2^{\mathcal{G}_{\mathsf{KRK}}}$, the partial-decryptions output by clients are $\mathsf{MGHE}.\texttt{PDec}(\cdot)$ in both two worlds. Therefore, we can then write the d_i from $\mathcal{O}_{\mathsf{Dec}}$ as the outputs of the clients.
– **On input** $\langle \texttt{Share} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, c; \rho \rangle$ **to a corrupted** $\mathcal{P}_{\mathsf{cli}}$: We note that for corrupted client, the adversary $\mathcal{A}_{cpa}$ has the secret keys of the corrupted client. Thus we let $\mathcal{A}_{cpa}$ directly run $d \leftarrow \mathsf{MGHE}.\texttt{PDec}(\mathsf{sk}_i, c; \rho)$. Then we write d as the output of the corrupted client.
– **On input** $\langle \texttt{Combine} : [\mathsf{grp}, \mathsf{sid}, c, \{d_i\}_{\mathsf{cli}_i \in \mathsf{grp}'}] \rangle$ **to a (uncorrupted or corrupted) client** $\mathcal{P}_{\mathsf{cli}}$: Note that in both $\mathbf{HYB}_1^{\mathcal{G}_{\mathsf{KRK}}}$ and $\mathbf{HYB}_2^{\mathcal{G}_{\mathsf{KRK}}}$, if insufficient decryption share is obtained, then $\mathcal{P}_{\mathsf{cli}}$ outputs $\perp$. Otherwise, it outputs the original message m (plus an error term if approximate).

Observe that by robustness, $m' \leftarrow \mathsf{MGHE}.\mathtt{Combine}(c, \{d_i\}_{\mathsf{cli}_i \in \mathsf{grp}'})$ is indistinguishable from m. Thus, we write $\perp$ if not enough decryption shares are obtained; otherwise, we write m' as the output of $\mathcal{P}_{\mathsf{cli}}$.

Finally, when $\mathcal{Z}$ outputs a bit b, we let $\mathcal{A}_{cpa}$ also output b. Thus $\mathcal{Z}$'s distinguishing advantage between these two worlds directly translates to $\mathcal{A}_{cpa}$'s advantage in the IND-CPA^{pD} game i.e.,

$$\Delta_{\mathcal{Z}}\left(\mathbf{HYB}_1^{\mathcal{G}_{\mathsf{KRK}}}, \mathbf{HYB}_2^{\mathcal{G}_{\mathsf{KRK}}}\right) \leq \mathbf{Adv}^{\text{IND-CPA}^{\mathrm{pD}}}_{\mathsf{MGHE}}(\mathcal{A}_{cpa}). \tag{8}$$

□

Claim 1.3. *If* MGHE *is* SIM-PDEC *secure with respect to a threshold simulator* $\mathtt{Sim}_{th}$*, then* $\mathbf{HYB}_2^{\mathcal{G}_{\mathsf{KRK}}}$ *and* $\mathbf{IDEAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\mathcal{F}_{\mathsf{MGHE}}, \mathcal{S}_{\mathsf{MGHE}}, \mathcal{Z}}$ *are statistically indistinguishable.*

Proof. By the definition of SIM-PDEC, there exists a threshold simulator $\mathtt{Sim}_{th}$ such that a partial-decryption generated by an uncorrupted client using $d \leftarrow \mathsf{MGHE}.\mathtt{PDec}(\mathsf{sk}_i, c)$ is statistically indistinguishable from a share generated via $d \leftarrow \mathtt{Sim}_{th}(c, m, \{\mathsf{sk}_i\}_{i \in I_{\mathcal{A}}})$. For corrupted clients, the decryption process remains the same in both hybrids since they are both computed as $d \leftarrow \mathsf{MGHE}.\mathtt{PDec}(\mathsf{sk}_i, c; \rho)$, or they directly injects a fake decryption share. Additionally, for approximate schemes, we know that δ_{ct} and δ are indistinguishable by previous hop. Thus we have that the reconstruct messages in these two worlds are also indistinguishable from each other in case of approximate schemes..

Therefore, the partial-decryptions output by any client $\mathcal{P}_{\mathsf{cli}}$ (whether uncorrupted or corrupted) in both of $\mathbf{HYB}_2^{\mathcal{G}_{\mathsf{KRK}}}$ and $\mathbf{IDEAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\mathcal{F}_{\mathsf{MGHE}}, \mathcal{S}_{\mathsf{MGHE}}, \mathcal{Z}}$ are statistically indistinguishable. □

Therefore, by combing Claims 1.1, 1.2, and 1.3, we can then conclude the proof for the theorem. □

D.2 Full proof of Theorem 2

Theorem 2. *Let* MGHE *be a multi-group homomorphic encryption scheme on approximate number with threshold factor* t*. Then, the protocol* Π_{MGHE} *UC-realizes the functionality* $\mathcal{F}_{\mathsf{MGHE}}$ *in the presence of a global key registry* $\mathcal{G}_{\mathsf{KRK}}$ *against a partial-decryption manipulating semi-malicious adversary that non-adaptively corrupts parties, provided that:*

- MGHE *satisfies robustness, and the adversary corrupts fewer than* $\frac{|\mathsf{grp}|}{t}$ *clients in any group.*
- MGHE *is* IND-CIRC *secure and the server is not corrupted.*
- MGHE *is* IND-CPA^{pD}*-secure, and there exists at least one uncorrupted client who provides input to the computation, such that the group* grp *under which it provides input contains no more than* $\frac{|\mathsf{grp}|}{t}$ *corrupted clients.*
- MGHE *satisfies* SIM-PDEC *security with respect to a simulator* $\mathtt{Sim}_{th}$*.*

Proof. Note that the correctness of the protocol follows Theorem 1. We only show security in the corollary. We consider the same setting as in the proof of Theorem 1.To show the security of Π_{MGHE}, we consider the following sequence of hybrids:

- $\mathbf{REAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\Pi_{\mathsf{MGHE}},\mathcal{A},\mathcal{Z}}$: This represents the execution of the protocol Π_{MGHE} in the real world, involving the environment $\mathcal{Z}$ and the adversary $\mathcal{A}$.
- $\mathbf{HYB}^{\mathcal{G}_{\mathsf{KRK}}}_1$: In this hybrid, we keep track of the message-ciphertext tuples (m, c) that are encrypted and evaluated, as well as the decryption shares d that are honestly generated with respect to (m, c). When a client combines the decryption shares, if it has obtained a sufficient number of honest decryption shares, it outputs $m+\delta+\sigma+\sum_{i=1}^{|\mathsf{grp}|/t} \rho_i$ where δ is calculated with $\mathsf{GetEncNoise}(\cdot)$ and $\mathsf{GetEvalNoise}(\cdot)$.
- $\mathbf{HYB}^{\mathcal{G}_{\mathsf{KRK}}}_2$: In this hybrid, when an uncorrupted server $\mathcal{P}_{\mathsf{srv}}$ is activated with the input $\langle \mathtt{Evaluate} : \mathsf{sid}, f, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{jpk}_j, c_j)_{j\in[\ell]} \rangle$, we let $\mathcal{P}_{\mathsf{srv}}$ output the ciphertext
$$\hat{c}^\star \leftarrow \mathsf{MGHE}.\mathtt{Eval}(f^\star, (\mathsf{jpk}_j, c_j)_{j\in[\ell]})$$
where $f^\star$ is a randomly sampled circuit from the circuit space $\mathcal{F}_\kappa$. The noise δ is then re-calculated with respect to $f^\star$ using $\mathsf{GetEncNoise}(\cdot)$ and $\mathsf{GetEvalNoise}(\cdot)$.
- $\mathbf{HYB}^{\mathcal{G}_{\mathsf{KRK}}}_3$: In this hybrid, whenever an uncorrupted client $\mathcal{P}_{\mathsf{cli}}$ is activated with the input $\langle \mathtt{Encrypt} : \mathsf{grp}, \mathsf{sid}, \mathsf{jpk}, m \rangle$, we let the client outputs $c \leftarrow \mathsf{MGHE}.\mathtt{Enc}(\mathsf{jpk}, 0^{|m|})$.
- $\mathbf{IDEAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\mathcal{F}_{\mathsf{MGHE}},\mathcal{S}_{\mathsf{MGHE}},\mathcal{Z}}$: This is the ideal execution where the ideal functionality $\mathcal{F}_{\mathsf{MGHE}}$ is attached to the simulator $\mathcal{S}_{\mathsf{MGHE}}$ and interacts with dummy client $\mathcal{P}_{\mathsf{cli}_1}, \ldots, \mathcal{P}_{\mathsf{cli}_n}$.

Note that by Claim 1.1, we know that $\mathbf{REAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\Pi_{\mathsf{MGHE}},\mathcal{A},\mathcal{Z}}$ and $\mathbf{HYB}^{\mathcal{G}_{\mathsf{KRK}}}_1$ are computational indistinguishable by the robustness of MGHE. Now we claim that:

Claim 2.1. *If* MGHE *is* IND-CIRC *secure,* $\mathbf{HYB}^{\mathcal{G}_{\mathsf{KRK}}}_1$ *and* $\mathbf{HYB}^{\mathcal{G}_{\mathsf{KRK}}}_2$ *are computationally indistinguishable.*

Proof. We show that if there exists an environment $\mathcal{Z}$ that can distinguish between $\mathbf{HYB}^{\mathcal{G}_{\mathsf{KRK}}}_1$ and $\mathbf{HYB}^{\mathcal{G}_{\mathsf{KRK}}}_2$, we can construct an IND-CIRC adversary $\mathcal{A}_{circ}$ against MGHE using this environment. To do so, we let $\mathcal{A}_{circ}$ simulate the interaction between $\mathcal{Z}$ and these two worlds. Specifically, $\mathcal{A}_{circ}$ forms the same groups by specifying the index set $I_1, \ldots, I_k$ in the game $\mathrm{G}^{\text{IND-CIRC}}_{\mathsf{MGHE}}$ and corrupts the same clients by defining the index set $I_{\mathcal{A}}$ as in the setup phase. Then, $\mathcal{A}_{cpa}$ handles its oracle queries based on the input provided by the environment $\mathcal{Z}$ to each party.

- **On input** $\langle \mathtt{Encrypt} : \mathsf{grp}, \mathsf{sid}, \mathsf{jpk}_j, m \rangle$ **to an uncorrupted client** $\mathcal{P}_{\mathsf{cli}}$: We let $\mathcal{A}_{cpa}$ query its oracle $\mathcal{O}_{\mathsf{Enc}}$ with $(\mathsf{jpk}_j, m; \omega)$ where r is an honestly sampled random coins. Note that in both $\mathbf{HYB}^{\mathcal{G}_{\mathsf{KRK}}}_1$ and $\mathbf{HYB}^{\mathcal{G}_{\mathsf{KRK}}}_2$. An uncorrupted client runs and outputs $c \leftarrow \mathsf{MGHE}.\mathtt{Enc}(\mathsf{jpk}_j, m; \omega)$ for an honestly sampled random coins ω, which is exactly the behavior of the oracle $\mathcal{O}_{\mathsf{Enc}}$. Thus we write c output from $\mathcal{O}_{\mathsf{Enc}}$ as the output of an uncorrupted client $\mathcal{P}_{\mathsf{cli}}$.
- **On input** $\langle \mathtt{Encrypt} : \mathsf{grp}, \mathsf{sid}, \mathsf{jpk}_j, m; \omega \rangle$ **to a corrupted client** $\mathcal{P}_{\mathsf{cli}}$: We let $\mathcal{A}_{cpa}$ query its oracle $\mathcal{O}_{\mathsf{Enc}}$ with $(\mathsf{jpk}_j, m; \omega)$, where ω is the random coins

provided by $\mathcal{Z}$ to a corrupted client. In both hybrids, since we consider a semi-malicious adversary, the ciphertext is generated as $c \leftarrow \mathsf{MGHE.Enc}(\mathsf{jpk}_j, m; \omega)$ and is written as the output of a corrupted client. This behavior corresponds to that of the oracle $\mathcal{O}_{\mathsf{Enc}}$. Therefore, we write the ciphertext c output by $\mathcal{O}_{\mathsf{Enc}}$ as the output of a corrupted client $\mathcal{P}_{\mathsf{cli}}$.

– **On input** $\langle \mathtt{Evaluate} : \mathsf{sid}, f, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{jpk}_j, c_j)_{j\in[\ell]} \rangle$ **to an uncorrupted server** $\mathcal{P}_{\mathsf{srv}}$: Similarly as in the proof of Theorem 1, we let $\mathcal{A}_{cpa}$ first identify these ciphertexts in the lookup table T of $\mathrm{G}^{\text{IND-CIRC}}_{\mathsf{MGHE}}$ by specifying an index set L. Then, we let $\mathcal{A}_{circ}$ queries the oracle $\mathcal{O}_{\mathsf{Eval}}$ with circuit-index set pair $(f, L, f^\star, L)$ where $f^\star$ is a circuit sampled uniformly at random from the circuit space $\mathcal{F}_\kappa$. If the oracle outputs $\perp$, we let $\mathcal{A}_{cpa}$ abort this query and nothing is written on any tape. This ensures that the output of a client $\mathcal{P}_{\mathsf{cli}}$ during decryption remains the same in both worlds, thereby preventing trivial distinguishing. Otherwise, the evaluated ciphertext output from $\mathcal{O}_{\mathsf{Eval}}$ is either

$$\hat{c} \leftarrow \mathsf{MGHE.Eval}(f, (\mathsf{T}[l].\mathsf{jpk}, \mathsf{T}[l].c)_{l\in L})$$

or

$$\hat{c}^\star \leftarrow \mathsf{MGHE.Eval}(f^\star, (\mathsf{T}[l].\mathsf{jpk}, \mathsf{T}[l].c)_{l\in L}).$$

Observe that in $\mathbf{HYB}_1^{\mathcal{G}_{\mathsf{KRK}}}$, an uncorrupted server $\mathcal{P}_{\mathsf{srv}}$ outputs $\hat{c}$, while in $\mathbf{HYB}_2^{\mathcal{G}_{\mathsf{KRK}}}$, $\hat{c}^\star$ corresponds to the output of an uncorrupted server. Therefore, we write $\hat{c}$ or $\hat{c}^\star$ returned by $\mathcal{O}_{\mathsf{Eval}}$ as the output of an uncorrupted server $\mathcal{P}_{\mathsf{srv}}$.

– **On input** $\langle \mathtt{Share} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, c \rangle$ **to an uncorrupted client** $\mathcal{P}_{\mathsf{cli}}$: This is handled the same way as in the proof of Claim 1.2.

– **On input** $\langle \mathtt{Share} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, c; \rho \rangle$ **to an corrupted client** $\mathcal{P}_{\mathsf{cli}}$: This is handled the same way as in the proof of Claim 1.2.

– **On input** $\langle \mathtt{Combine} : [\mathsf{grp}, \mathsf{sid}, c, \{d_i\}_{\mathsf{cli}_i\in\mathsf{grp}'}] \rangle$ **to a client** $\mathcal{P}_{\mathsf{cli}}$: This is handled the same way as in the proof of Claim 1.2.

Finally, we let $\mathcal{A}_{circ}$ output the same bit output by $\mathcal{Z}$. Thus, the distinguishing advantage of $\mathcal{Z}$ translates directly to the advantage of $\mathcal{A}_{circ}$ in the IND-CIRC game:

$$\Delta_{\mathcal{Z}}\left(\mathbf{HYB}_1^{\mathcal{G}_{\mathsf{KRK}}}, \mathbf{HYB}_2^{\mathcal{G}_{\mathsf{KRK}}}\right) \leq \mathbf{Adv}^{\text{IND-CIRC}}_{\mathsf{MGHE}}(\mathcal{A}_{circ}). \tag{9}$$

□

Claim 2.2. *If* MGHE *is* IND-CPA^{pD} *secure,* $\mathbf{HYB}_2^{\mathcal{G}_{\mathsf{KRK}}}$ *and* $\mathbf{HYB}_3^{\mathcal{G}_{\mathsf{KRK}}}$ *are computationally indistinguishable.*

Proof. We consider the same IND-CPA^{pD} adversary $\mathcal{A}_{cpa}$ as defined in the proof of Theorem 2, but instead let $\mathcal{A}_{cpa}$ query its oracle $\mathcal{O}_{\mathsf{Eval}}$ with a circuit $f^\star$ sampled from the circuit space $\mathcal{F}_\kappa$ at random. Note that In $\mathbf{HYB}_2^{\mathcal{G}_{\mathsf{KRK}}}$, the ciphertext $\hat{c}$ is computed as

$$\hat{c} \leftarrow \mathsf{MGHE.Eval}(f^\star, \{\mathsf{jpk}\}_{j\in[\ell]}, f(\{m_j\}_{j\in[\ell]}))$$

for some circuit $f^\star$ selected by $\mathcal{A}_{cpa}$, where each m_j is encrypted by either an uncorrupted or a corrupted client. In contrast, in $\mathbf{HYB}_3^{\mathcal{G}_{\mathsf{KRK}}}$, the ciphertext $\hat{c}$, the ciphertext $\hat{c}$ is computed as

$$\hat{c} \leftarrow \mathsf{MGHE.Eval}(f^\star, \{\mathsf{jpk}\}_{j\in[\ell]}, f(\{m'_j\}_{j\in[\ell]}))$$

where each m'_j is either $0^{|m_j|}$ or some plaintext m_j if it is from an uncorrupted client or m_j if it is from a corrupted client or encrypted for a corrupted group. The remaining part follows the proof of Claim 1.2. □

Also, by Claim 1.3, we have that $\mathbf{HYB}_3^{\mathcal{G}_{\mathsf{KRK}}}$ and $\mathbf{IDEAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\mathcal{F}_{\mathsf{MGHE}},\mathcal{S}_{\mathsf{MGHE}},\mathcal{Z}}$ statistically indistinguishable by SIM-PDEC of MGHE. Therefore, by combining Claim 2.1, 2.2, and Claim 1.1 and 1.3, we can conclude the proof for this theorem. □

We can then consider a stronger security by requiring the statistical indistinguishability of the circuit for schemes with exact evaluation. In Corollary 1, we show the server-side confidentiality with the SIM-CIRC security.

D.3 Full proof of Corollary 1

Corollary 1. *Let* MGHE *be a multi-group homomorphic encryption scheme on exact number with threshold factor t. Then, the protocol Π_{MGHE} UC-realizes the functionality $\mathcal{F}_{\mathsf{MGHE}}$ in the presence of a global key registry $\mathcal{G}_{\mathsf{KRK}}$ against a partial-decryption manipulating semi-malicious adversary that non-adaptively corrupts parties, provided that:*

- MGHE *satisfies robustness, and the adversary corrupts fewer than* $\frac{|\mathsf{grp}|}{t}$ *clients in any group.*
- MGHE *is* SIM-CIRC *secure and the server is not corrupted.*
- MGHE *is* IND-CPA^{pD}*-secure, and there exists at least one uncorrupted client who provides input to the computation, such that the group* grp *under which it provides input contains no more than* $\frac{|\mathsf{grp}|}{t}$ *corrupted clients.*
- MGHE *satisfies* SIM-PDEC *security with respect to a simulator* $\mathtt{Sim}_{th}$.

Proof. We consider the same setting as in the proof of Theorem 1. From Theorem 2, we only replace the last two games. For completeness, we recall all of the hybrid worlds here.

– $\mathbf{REAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\Pi_{\mathsf{MGHE}},\mathcal{A},\mathcal{Z}}$: This represents the execution of the protocol Π_{MGHE} in the real world, involving the environment $\mathcal{Z}$ and the adversary $\mathcal{A}$.
– $\mathbf{HYB}_1^{\mathcal{G}_{\mathsf{KRK}}}$: In this hybrid, we keep track of the message-ciphertext tuples (m, c) that are encrypted and evaluated, as well as the decryption shares d that are honestly generated with respect to (m, c). When a client combines the decryption shares, if it has obtained a sufficient number of honest decryption shares, it outputs m.

– $\mathbf{HYB}_2^{\mathcal{G}_{\mathsf{KRK}}}$: In this hybrid, when an uncorrupted server $\mathcal{P}_{\mathsf{srv}}$ is activated with the input $\langle \mathtt{Evaluate} : \mathsf{sid}, f, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{jpk}_j, c_j)_{j\in[\ell]}\rangle$, we let $\mathcal{P}_{\mathsf{srv}}$ output the ciphertext $\hat{c} \leftarrow \mathtt{Sim}_{\mathsf{circ}}(\{\mathsf{jpk}_j\}_{j\in[\ell]}, \hat{m})$ produced by the circuit privacy simulator $\mathtt{Sim}_{\mathsf{circ}}$ where $\hat{m} = f(m_1, \ldots, m_j)$ and $c_j \leftarrow \mathsf{MGHE}.\mathtt{Enc}(\mathsf{jpk}_j, m_j)$ for $j \in [\ell]$.
– $\mathbf{HYB}_3^{\mathcal{G}_{\mathsf{KRK}}}$: In this hybrid, whenever an uncorrupted client $\mathcal{P}_{\mathsf{cli}}$ is activated with the input $\langle \mathtt{Encrypt} : \mathsf{grp}, \mathsf{sid}, \mathsf{jpk}, m\rangle$, we let the client outputs $c \leftarrow \mathsf{MGHE}.\mathtt{Enc}(\mathsf{jpk}, 0^{|m|})$.
– $\mathbf{IDEAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\mathcal{F}_{\mathsf{MGHE}}, \mathcal{S}_{\mathsf{MGHE}}, \mathcal{Z}}$: This is the ideal execution where the ideal functionality $\mathcal{F}_{\mathsf{MGHE}}$ is attached to the simulator $\mathcal{S}_{\mathsf{MGHE}}$ and interacts with dummy client $\mathcal{P}_{\mathsf{cli}_1}, \ldots, \mathcal{P}_{\mathsf{cli}_n}$.

Similarly, by Claim 1.1, we know that $\mathbf{REAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\Pi_{\mathsf{MGHE}}, \mathcal{A}, \mathcal{Z}}$ and $\mathbf{HYB}_1^{\mathcal{G}_{\mathsf{KRK}}}$ are computational indistinguishable by the robustness of MGHE. Now we claim that:

Claim 2.3. *If* MGHE *is* SIM-CIRC *secure with respect to a circuit-privacy simulator* $\mathtt{Sim}_{\mathsf{circ}}$*, then* $\mathbf{HYB}_1^{\mathcal{G}_{\mathsf{KRK}}}$ *and* $\mathbf{HYB}_2^{\mathcal{G}_{\mathsf{KRK}}}$ *are statistically indistinguishable.*

Proof. By the definition of SIM-CIRC, there exists a circuit-privacy simulator $\mathtt{Sim}_{\mathsf{circ}}$ such that a ciphertext evaluated with $\hat{c} \leftarrow \mathsf{MGHE}.\mathtt{Eval}(f, (jpk_j, c_j)_{j\in[\ell]})$ is statistically indistinguishable from $c \leftarrow \mathtt{Sim}_{\mathsf{circ}}(\{jpk_j\}_{j\in[\ell]}, \hat{m}$ where $\hat{m} = f(m_1, \ldots, m_\ell)$ and $c_j \leftarrow \mathsf{MGHE}.\mathtt{Enc}(\mathsf{jpk}_j, m_j)$ for $j \in [\ell]$. Observe that these two outputs correspond to the outputs of an uncorrupted server when it is activated to evaluated a circuit f in these two worlds. Thus we have that the distributions of the two worlds are statistically indistinguishable. □

Claim 2.4. *If* MGHE *is* IND-CPA^{pD} *secure,* $\mathbf{HYB}_2^{\mathcal{G}_{\mathsf{KRK}}}$ *and* $\mathbf{HYB}_3^{\mathcal{G}_{\mathsf{KRK}}}$ *are computationally indistinguishable.*

Proof. We consider the same IND-CPA^{pD} adversary $\mathcal{A}_{cpa}$ as defined in the proof of Claim 2, but with a change of how $\mathcal{A}_{cpa}$ handles it queries to $\mathcal{O}_{\mathsf{Eval}}$. Note that In $\mathbf{HYB}_2^{\mathcal{G}_{\mathsf{KRK}}}$, the ciphertext $\hat{c}$ is computed as

$$\hat{c} \leftarrow \mathtt{Sim}_{\mathsf{circ}}(\{\mathsf{jpk}\}_{j\in[\ell]}, f(\{m_j\}_{j\in[\ell]}))$$

for some circuit f, where each m_j is encrypted by either an uncorrupted or a corrupted client. In contrast, in $\mathbf{HYB}_3^{\mathcal{G}_{\mathsf{KRK}}}$, the ciphertext $\hat{c}$, the ciphertext $\hat{c}$ is computed as

$$\hat{c} \leftarrow \mathtt{Sim}_{\mathsf{circ}}(\{\mathsf{jpk}\}_{j\in[\ell]}, f(\{m'_j\}_{j\in[\ell]}))$$

where each m'_j is either $0^{|m_j|}$ or some plaintext m_j if it is from an uncorrupted client or m_j if it is from a corrupted client or encrypted for a corrupted group.

Furthermore, by Claim 2.3, we know that the ciphertext $\hat{c}$ obtained from $\mathsf{MGHE}.\mathtt{Eval}(\cdot)$ and $\mathtt{Sim}_{\mathsf{circ}}(\cdot)$ are statistically indistinguishable. Thus, the output of $\mathcal{P}_{\mathsf{srv}}$ in these two worlds corresponds to the behavior of $\mathcal{O}_{\mathsf{Eval}}$ when $b = 0$ or $b = 1$ in the IND-CPA^{pD} game, respectively. Upon receiving $\hat{c}$ from $\mathcal{O}_{\mathsf{Eval}}$, we can then write it as the output of $\mathcal{P}_{\mathsf{srv}}$. The remaining part follows the proof of Claim 1.2. □

Also, by Claim 1.3, we have that $\mathbf{HYB}_3^{\mathcal{G}_{\mathsf{KRK}}}$ and $\mathbf{IDEAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\mathcal{F}_{\mathsf{MGHE}},\mathcal{S}_{\mathsf{MGHE}},\mathcal{Z}}$ statistically indistinguishable by SIM-PDEC of MGHE. Therefore, by combing Claims 2.3 and 2.4, and Claims 1.1 and 1.3, we can then conclude the proof for this corollary. □

D.4 Full proof of Theorem 3

Theorem 3. *The protocol $\Pi^{\mathsf{SM}}_{\mathsf{OtF\text{-}MPC}}$ UC-realizes $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ in $\mathcal{F}_{\mathsf{MGHE}}$-hybrid model with a global key registry $\mathcal{G}_{\mathsf{KRK}}$ against a partial-decryption manipulating semi-malicious adversary that corrupts a group up to the threshold.*

Proof. Let $\mathcal{A}$ be an adversary interacting with parties executing $\Pi^{\mathsf{SM}}_{\mathsf{OtF\text{-}MPC}}$ in $\mathcal{F}_{\mathsf{MGHE}}$-hybrid model. We construct a simulator $\mathcal{S}$ such that no environment $\mathcal{Z}$ can distinguish between

- $\mathbf{REAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\Pi^{\mathsf{SM}}_{\mathsf{OtF\text{-}MPC}},\mathcal{A},\mathcal{Z}}$: the environment $\mathcal{Z}$ interacting with $\mathcal{A}$ and parties running $\Pi^{\mathsf{SM}}_{\mathsf{OtF\text{-}MPC}}$ in the $\mathcal{F}_{\mathsf{MGHE}}$-hybrid model
- $\mathbf{IDEAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\mathcal{F}_{\mathsf{OtF\text{-}MPC}},\mathcal{S},\mathcal{Z}}$: the environment $\mathcal{Z}$ interacting with $\mathcal{S}$ and the ideal functionality $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$.

We let the simulator $\mathcal{S}$ runs an internal copy of $\mathcal{A}$ such that $\mathcal{S}$ forwards all communications from $\mathcal{Z}$ to $\mathcal{A}$ and back. $\mathcal{S}$ also runs internal copies of $\{\mathcal{P}_{\mathsf{cli}_i},\ldots,\mathcal{P}_{\mathsf{cli}_n}\}$ such that $\mathcal{S}$ simulates the interaction between $\mathcal{A}$ and $\{\mathcal{P}_{\mathsf{cli}_i},\ldots,\mathcal{P}_{\mathsf{cli}_n},\mathcal{P}_{\mathsf{srv}}\}$ as if $\mathcal{S}$ runs the functionality $\mathcal{F}_{\mathsf{MGHE}}$. When $\mathcal{A}$ sends a (backdoor) message to corrupt a party, we let $\mathcal{S}$ send the same backdoor message to $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ to corrupt the same party.

The simulation of each phase is as follows:

– **Input Phase, Corrupted Client:** When a corrupted client is activated with the input

$$\langle \mathtt{Upload} : \mathsf{cli},\mathsf{grp},\mathsf{sid},\mathsf{uid},m\rangle,$$

we observe that the simulator $\mathcal{S}$ receives m from $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ in the world $\mathbf{IDEAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\mathcal{F}_{\mathsf{OtF\text{-}MPC}},\mathcal{S},\mathcal{Z}}$.
Now in $\mathbf{REAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\Pi^{\mathsf{SM}}_{\mathsf{OtF\text{-}MPC}},\mathcal{A},\mathcal{Z}}$, the adversary $\mathcal{A}$ sends (through a corrupted client) the input

$$\langle \mathtt{Encrypt} : \mathsf{grp},\mathsf{sid},\mathsf{jpk},m;\omega\rangle$$

to $\mathcal{F}_{\mathsf{MGHE}}$. We let the simulator forward this input back to $\mathcal{A}$ in the name of $\mathcal{F}_{\mathsf{MGHE}}$. Upon receiving the ciphertext c from $\mathcal{A}$, we let $\mathcal{S}$ send c to (the corrupted) $\mathcal{P}_{\mathsf{cli}}$ that it internally runs.

– **Input Phase, Uncorrupted Client:** When an uncorrupted client is activated with the input

$$\langle \mathtt{Upload} : \mathsf{cli},\mathsf{grp},\mathsf{sid},\mathsf{uid},m\rangle,$$

we observe that $\mathcal{S}$ receives either m or $|m|$, depending on the corruption status of grp, from $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ in the world $\mathbf{IDEAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\mathcal{F}_{\mathsf{OtF\text{-}MPC}},\mathcal{S},\mathcal{Z}}$.

To simulate the world $\mathbf{REAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\Pi^{\mathsf{SM}}_{\mathsf{OtF\text{-}MPC}},\mathcal{A},\mathcal{Z}}$, we let $\mathcal{S}$ first retrieve jpk from $\mathcal{G}_{\mathsf{KRK}}$. Then $\mathcal{S}$ sends to $\mathcal{A}$, in name of $\mathcal{F}_{\mathsf{MGHE}}$, the input

$$\langle \texttt{Encrypt} : \mathsf{grp}, \mathsf{sid}, \mathsf{jpk}, m \,/\, |m| \rangle.$$

Upon receiving a ciphertext c from $\mathcal{A}$, we let $\mathcal{S}$ send c (in the name of $\mathcal{F}_{\mathsf{MGHE}}$) to the party $\mathcal{P}_{\mathsf{cli}}$ that it internally runs.

– **Computation Phase, Corrupted Server**: When a corrupted server is activated with the input

$$\big\langle \texttt{Evaluate} : \mathsf{srv}, \mathsf{sid}, \mathsf{uid}, f, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)_{j\in[\ell]} \big\rangle,$$

we observe that the simulator $\mathcal{S}$ knows the circuit f from the corrupted server $\mathcal{P}_{\mathsf{srv}}$ and receives the evaluated plaintext $\hat{m} = f(\{m_j\}_{j\in[ell]}$ [16] from $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ in the world $\mathbf{IDEAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\mathcal{F}_{\mathsf{OtF\text{-}MPC}},\mathcal{S},\mathcal{Z}}$.
Now in $\mathbf{REAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\Pi^{\mathsf{SM}}_{\mathsf{OtF\text{-}MPC}},\mathcal{A},\mathcal{Z}}$, the adversary $\mathcal{A}$ sends (through a corrupted client) the input

$$\big\langle \texttt{Evaluate} : \mathsf{sid}, f, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{jpk}_j, c_j)_{j\in[\ell]}; \sigma \big\rangle$$

to $\mathcal{F}_{\mathsf{MGHE}}$, we let $\mathcal{S}$ forward this back to $\mathcal{A}$ in the name of $\mathcal{F}_{\mathsf{MGHE}}$. When the adversary then outputs an evaluated ciphertext $\hat{c}$, we let $\mathcal{S}$ send $\hat{c}$ to (the corrupted) server $\mathcal{P}_{\mathsf{srv}}$ to match with the behavior of $\mathcal{F}_{\mathsf{MGHE}}$.

– **Computation Phase, Uncorrupted Server:** When an uncorrupted server is activated with the input

$$\big\langle \texttt{Evaluate} : \mathsf{srv}, \mathsf{sid}, \mathsf{uid}, f, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)_{j\in[\ell]} \big\rangle,$$

we observe that the simulator $\mathcal{S}$ only receives evaluated plaintext $\hat{m} = f(\{m_j\}_{j\in[ell]}$ from $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ in the world $\mathbf{IDEAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\mathcal{F}_{\mathsf{OtF\text{-}MPC}},\mathcal{S},\mathcal{Z}}$.
Now in the world $\mathbf{REAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\Pi^{\mathsf{SM}}_{\mathsf{OtF\text{-}MPC}},\mathcal{A},\mathcal{Z}}$, we let the simulator retrieve $\{\mathsf{jpk}_j\}_{j\in[\ell]}$from $\mathcal{G}_{\mathsf{KRK}}$. Then $\mathcal{S}$ send (in name of $\mathcal{F}_{\mathsf{MGHE}}$) to $\mathcal{A}$ the input

$$\big\langle \texttt{Evaluate} : \mathsf{sid}, \sqcup, \hat{m}, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{jpk}_{j,u}, c_{j,u})_{j\in[\ell]} \big\rangle.$$

When the adversary returns an evaluated ciphertext $\hat{c}$, we then let $\mathcal{S}$ send $\hat{c}$ to $\mathcal{P}_{\mathsf{srv}}$ that it internally runs.

– **Output Phase, Corrupted Client:** When a corrupted client is activated with the input

$$\langle \texttt{Retrieve} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid} \rangle,$$

we observe that $\mathcal{S}$ learns the message m if it is an evaluated message from the server, and $\mathcal{S}$ learns nothing if it is uploaded by a client since we assume that the adversary has not corrupted the group, from $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ in the world $\mathbf{IDEAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\mathcal{F}_{\mathsf{OtF\text{-}MPC}},\mathcal{S},\mathcal{Z}}$.

[16] We note that the adversary does not inject $\hat{m}^\star$ since we consider a semi-malicious adversary now.

Now in $\mathbf{REAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\Pi^{\mathsf{SM}}_{\mathsf{OtF\text{-}MPC}},\mathcal{A},\mathcal{Z}}$ to $\mathcal{F}_{\mathsf{MGHE}}$, when the adversary sends (through the corrupted client) the input

$$\langle \mathtt{Share} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, c; \rho \rangle$$

to $\mathcal{F}_{\mathsf{MGHE}}$, we first let the simulator check if it has seen the ciphertext c before. If yes, then $\mathcal{S}$ sends back this input to $\mathcal{A}$. Upon getting the decryption share d from $\mathcal{A}$, we let $\mathcal{S}$ keep a record of it and send it to the (corrupted) client $\mathcal{P}_{\mathsf{cli}}$. Later, when the adversary sends (through the corrupted client) the input

$$\langle \mathtt{Combine} : [\mathsf{grp}, \mathsf{sid}, c, \{d_i\}_{i \in \mathsf{grp}'}] \rangle,$$

we let simulator checks whether there are enough decryption shares recorded before in $\{d_i\}_{i \in \mathsf{grp}'}$. If so, we let $\mathcal{S}$ output m that it receives from $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ earlier to $\mathcal{P}_{\mathsf{cli}}$ in the message

$$\{\mathtt{"pt"} : \mathsf{grp}, \mathsf{sid}, c, m\}.$$

– **Output Phase, Uncorrupted Client:** When an uncorrupted client $\mathcal{P}_{\mathsf{cli}}$ is activated with the input

$$\langle \mathtt{Retrieve} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid} \rangle,$$

we observe that the simulator receives an evaluated plaintext m from $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ from $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ in the world $\mathbf{IDEAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\mathcal{F}_{\mathsf{OtF\text{-}MPC}},\mathcal{S},\mathcal{Z}}$.
To simulate the world $\mathbf{REAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\Pi^{\mathsf{SM}}_{\mathsf{OtF\text{-}MPC}},\mathcal{A},\mathcal{Z}}$, we let $\mathcal{S}$ find the corresponding ciphertext c in its record. Then $\mathcal{S}$ sends to $\mathcal{A}$ in name of $\mathcal{F}_{\mathsf{MGHE}}$ the input

$$\langle \mathtt{Share} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, c, m \rangle$$

Upon getting the decryption share d from $\mathcal{A}$, we let $\mathcal{S}$ send it to the client $\mathcal{P}_{\mathsf{cli}}$ that it internally runs.
Now we let $\mathcal{S}$ then check if it has obtained decryption shares from $\mathcal{A}$. If yes, we then let $\mathcal{S}$ send to the client the plaintext m that it receives from $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ in a message

$$\{\mathtt{"pt"} : \mathsf{grp}, \mathsf{sid}, c, m\}.$$

We note that in each case, $\mathcal{S}$ simulates the interaction between $\mathcal{A}$, parties in the protocol $\Pi^{\mathsf{SM}}_{\mathsf{OtF\text{-}MPC}}$ in $\mathcal{F}_{\mathsf{MGHE}}$-hybrid model. Therefore, the environment's view is indistinguishable in the real and ideal executions. □

D.5 Full proof of Theorem 4

Theorem 4. *The protocol $\Pi^{\mathsf{SE\text{-}1P}}_{\mathsf{OtF\text{-}MPC}}$ UC-realizes $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ in $[\mathcal{F}_{\mathsf{MGHE}}, \mathcal{F}_{\mathsf{zkSNARK}}, \mathcal{F}_{\mathsf{AUTH}}]$-hybrid model in presence of a global key registry $\mathcal{G}_{\mathsf{KRK}}$ and a global random oracle $\mathcal{G}_{\mathsf{RO}}$ against a full malicious adversary that corrupts a group up to the threshold.*

Proof. We follow the proof of Theorem 3 for the confidential layer with $\mathcal{F}_{\mathsf{MGHE}}$. Now we additionally prove the theorem by considering the interactions involving $\mathcal{F}_{\mathsf{zkSNARK}}$ and $\mathcal{F}_{\mathsf{Auth}}$.

– **Input Phase, Corrupted Client**: In $\mathbf{REAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\Pi^{\mathsf{SE\text{-}1P}}_{\mathsf{OtF\text{-}MPC}},\mathcal{A},\mathcal{Z}}$, when $\mathcal{A}$ sends an a message to $\mathcal{F}_{\mathsf{AUTH}}$ to register it (on behalf of the corrupted client). We let $\mathcal{S}$ parses the message as

$$\{\texttt{"up"} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, (c, \pi)\}\,.$$

Then $\mathcal{S}$ stores this message in internally for later use.
Next, $\mathcal{S}$ sends the input $\langle \texttt{Observe} : \mathsf{sid}\rangle$ to $\mathcal{G}_{\mathsf{RO}}$ to obtain the query trace tr_z made by the environment $\mathcal{Z}$. Then for each (x, v) in tr_z, we let $\mathcal{S}$ send the input $\langle \texttt{IsProg} : x\rangle$ to obtain a list of programmed points pg_z made by $\mathcal{Z}$. Using this information, $\mathcal{S}$ invokes the extractor to compute

$$w \leftarrow \mathtt{Ext}((\mathsf{jpk}, c), \pi, \mathsf{tr}_z \setminus \mathsf{pg}_z).$$

We let $\mathcal{S}$ parse the witness w as (m, ω) with respect to R_{Enc}.
Then in $\mathbf{IDEAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\mathcal{F}_{\mathsf{OtF\text{-}MPC}},\mathcal{S},\mathcal{Z}}$, we let $\mathcal{S}$ send the following input to $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ on behalf of the corrupted client:

$$\{\texttt{"up"} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, m\}\,.$$

In all other cases, including when the message cannot be parsed or extraction cannot be made, $\mathcal{S}$ sets $m = \bot$ when sending the input to $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$.

– **Input Phase, Uncorrupted Client**: In $\mathbf{REAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\Pi^{\mathsf{SE\text{-}1P}}_{\mathsf{OtF\text{-}MPC}},\mathcal{A},\mathcal{Z}}$, upon receiving the ciphertext c from $\mathcal{A}$ (sending back to $\mathcal{F}_{\mathsf{MGHE}}$ as in proof of Theorem 3), since $\mathcal{A}$ is not involved in $\mathcal{F}_{\mathsf{zkSNARK}}$, we let $\mathcal{S}$ directly invoke $(\pi, \mathsf{pg}) \leftarrow \mathtt{Sim}_{zk}{}^{\mathcal{G}_{\mathsf{RO}}[\mathsf{sid}]}(c)$ to generate an argument π for the relation R_{Enc} and program $\mathcal{G}_{\mathsf{RO}}$ with pg respectively. We then let $\mathcal{S}$ generate a message

$$\{\texttt{"up"} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, (c_1, c_2, \pi)\}$$

When a party sends an input to $\mathcal{F}_{\mathsf{AUTH}}$ to retrieve this message, we let $\mathcal{S}$ deliver the message to the party (in the name of $\mathcal{F}_{\mathsf{AUTH}}$).

– **Computation Phase, Corrupted Server**: In $\mathbf{REAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\Pi^{\mathsf{SE\text{-}1P}}_{\mathsf{OtF\text{-}MPC}},\mathcal{A},\mathcal{Z}}$, when $\mathcal{A}$ sends a message to $\mathcal{F}_{\mathsf{AUTH}}$ (through the corrupted server) to register the evaluation of a circuit, we let $\mathcal{S}$ parse it as

$$\left\{\texttt{"eval"} : \mathsf{srv}, \hat{\mathsf{grp}}, \mathsf{sid}, \mathsf{uid}, (\hat{c}, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)_{j\in[\ell]}, \pi)\right\}.$$

Similarly, $\mathcal{S}$ stores the message internally for later use.
Next, $\mathcal{S}$ queries $\mathcal{G}_{\mathsf{RO}}$ to obtain the query trace tr_z made by the environment $\mathcal{Z}$, and the list of points pg_z programmed by $\mathcal{Z}$. Then we let $\mathcal{S}$ then runs the extractor

$$w \leftarrow \mathtt{Ext}(\hat{c}, (\mathsf{jpk}_j, c_j)_{j\in[\ell]}, \pi, \mathsf{tr}_z \setminus \mathsf{pg}_z),$$

with respect to the relation R^f_{Eval} (where f is obtained by corrupting the server) and parses w as a random coin σ.
Then in $\mathbf{IDEAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\mathcal{F}_{\mathsf{OtF\text{-}MPC}},\mathcal{S},\mathcal{Z}}$, we let $\mathcal{S}$ send the following input to $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$:

$$\left\{\texttt{"eval"} : \mathsf{srv}, \hat{\mathsf{grp}}, \mathsf{sid}, \mathsf{uid}, \sqcup, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)_{j\in[\ell]}\right\},$$

where $\hat{m}^\star$ sent to $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ is set as an empty placeholder $\sqcup$ to allow $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ to evaluate $\hat{m} = f(\{m_j\}_{j\in[\ell]})$ and record the result $\hat{m}$.
In all other cases including extraction failure or message cannot be parsed, $\mathcal{S}$ sends the input

$$\left\{\texttt{"eval"} : \mathsf{srv}, \hat{\mathsf{grp}}, \mathsf{sid}, \mathsf{uid}, \bot, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)_{j\in[\ell]}\right\}$$

to $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ by setting $\bot$ as injected message to yield an invalid evaluated result $\hat{m} = \bot$.

– **Computation Phase, Uncorrupted Server**: In $\mathbf{IDEAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\mathcal{F}_{\mathsf{OtF\text{-}MPC}},\mathcal{S},\mathcal{Z}}$, when an uncorrupted server $\mathcal{P}_{\mathsf{srv}}$ is activated with the input

$$\left\langle \texttt{Evaluate} : \mathsf{srv}, \mathsf{sid}, \mathsf{uid}, f, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)_{j\in[\ell]}\right\rangle,$$

we observe that each message m_j for $j \in [\ell]$ must satisfy $m_j \neq \bot$ to proceed to the next step of evaluation by the definition of $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$.
Now observe that in $\mathbf{REAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\Pi^{\mathsf{SE\text{-}1P}}_{\mathsf{OtF\text{-}MPC}},\mathcal{A},\mathcal{Z}}$, when $\mathcal{P}_{\mathsf{srv}}$ is activated with the input to evaluate a circuit, it first checks each input ciphertext that it receives from $\mathcal{F}_{\mathsf{AUTH}}$. When the message at position $(\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)$ originates from a client, the uncorrupted server parses it as

$$\left\{\texttt{"up"} : \mathsf{cli}, \mathsf{grp}_j, \mathsf{sid}_j, (c_j, \pi)\right\}$$

When $\mathcal{P}_{\mathsf{cli}}$ (that is run internally by $\mathcal{S}$) verifies that π is a valid argument for the instance (jpk_j, c_j) using $\mathcal{F}^{R_{\mathsf{Enc}}}_{\mathsf{zkSNARK}}$, we let $\mathcal{S}$ check if it has π in its record. If yes, π will always be accepted, meaning it is honestly generated. Otherwise, the behavior of $\mathcal{F}^{R_{\mathsf{Enc}}}_{\mathsf{zkSNARK}}$ is exactly the same as what we have the simulator $\mathcal{S}$ do in case of a corrupted client during the *input phase*. Recall that in case the verification is not successful, $\mathbf{IDEAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\mathcal{F}_{\mathsf{OtF\text{-}MPC}},\mathcal{S},\mathcal{Z}}$, the simulator $\mathcal{S}$ writes $m = \bot$ to $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$. which causes $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ to abort during evaluation. Thus we have that the condition to abort an execution is the same in both of these two worlds.
On the other hand, if the message is from the server, in $\mathbf{REAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\Pi^{\mathsf{SE\text{-}1P}}_{\mathsf{OtF\text{-}MPC}},\mathcal{A},\mathcal{Z}}$, the uncorrupted server parses it as

$$\left\{\texttt{"eval"} : \mathsf{srv}, \mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j, (c_j, (\mathsf{grp}_{j'}, \mathsf{sid}_{j'}, \mathsf{uid}_{j'})_{j'\in[\ell']}, \pi)\right\}.$$

Similarly, if π is generated honestly by the server, it means that the relation R^f_{Eval} is satisfied with respect to some circuit f. By the definition of $\mathcal{F}_{\mathsf{MGHE}}$, then it has

$$m_j = f(\{m_{j'}\}_{j'\in[\ell']}),$$

where m_j and $\{m_{j'}\}_{j' \in [\ell'}$ are the underlying messages of c_j and $\{c_{j'}\}_{j' \in [\ell']}$ respectively. We recall that when the verification fails, $\mathcal{S}$ is defined to write $\hat{m} = \perp$ to $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$, which prevent $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ from further execution in $\mathbf{IDEAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\mathcal{F}_{\mathsf{OtF\text{-}MPC}},\mathcal{S},\mathcal{Z}}$. Thus, the condition for proceeding to the next step of evaluation is equivalent in both the real and ideal worlds.
If all these checks pass, we then let $\mathcal{S}$ simulate $\mathbf{REAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\Pi^{\mathsf{SE\text{-}1P}}_{\mathsf{OtF\text{-}MPC}},\mathcal{A},\mathcal{Z}}$. We recall that the simulator $\mathcal{S}$ receives $\hat{m} = f(\{m_j\}_{j\in[\ell]})$ from $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ in the real world, and the simulation of interaction between $\mathcal{A}$ and $\mathcal{F}_{\mathsf{MGHE}}$ is the same as in proof of Theorem 3. Then upon receiving $\hat{c}$ from $\mathcal{A}$, we let $\mathcal{S}$ invoke $(\pi, \mathsf{pg}) \leftarrow \mathtt{Sim}_{zk}{}^{\mathcal{G}_{\mathsf{RO}}[\mathsf{sid}]}(\hat{c}$ to generate an argument π for the relation R^f_{Eval} and program $\mathcal{G}_{\mathsf{RO}}$ with pg. We then let $\mathcal{S}$ store the message

$$\left\{\texttt{"eval"} : \mathsf{srv}, \hat{\mathsf{grp}}, \mathsf{sid}, \mathsf{uid}, (\hat{c}, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)_{j\in[\ell]}, \pi)\right\}.$$

When another party (that is internally run by $\mathcal{S}$) send input to $\mathcal{F}_{\mathsf{AUTH}}$ to retrieve this message, we let $\mathcal{S}$ output it to the party in name of $\mathcal{F}_{\mathsf{AUTH}}$.

– **Output Phase, Corrupted Client**: In $\mathbf{REAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\Pi^{\mathsf{SE\text{-}1P}}_{\mathsf{OtF\text{-}MPC}},\mathcal{A},\mathcal{Z}}$, when $\mathcal{A}$ sends a message containing decryption share to $\mathcal{F}_{\mathsf{AUTH}}$, we let $\mathcal{S}$ parse the message as

$$\{\texttt{"sh"} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, (c, d, \pi)\}.$$

Then we similarly let $\mathcal{S}$ query $\mathcal{G}_{\mathsf{RO}}$ to obtain the query trace tr_z made by the environment $\mathcal{Z}$, and the list of points pg_z programmed by $\mathcal{Z}$. Then $\mathcal{S}$ calls the extractor to obtain the witness

$$w \leftarrow \mathtt{Ext}((c, d), \pi, \mathsf{tr}_z \setminus \mathsf{pg}_z),$$

and parse $w = (\mathsf{sk}, \gamma, \rho)$. Note that if $w = \perp$, we then let $\mathcal{S}$ ignore this message. Otherwise, we then let $\mathcal{S}$ record this decryption share and mark it as valid one in its record for later use.

– **Output Phase, Uncorrupted Client**: Note that we assume the group grp is not corrupted over the threshold. In $\mathbf{IDEAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\mathcal{F}_{\mathsf{OtF\text{-}MPC}},\mathcal{S},\mathcal{Z}}$, if the message originates from a client (i.e., it begins with the identifier $\{\texttt{"up"} : \cdot\}$), $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ aborts the execution. The same is followed by an uncorrupted client $\mathcal{P}_{\mathsf{cli}}$ executing $\Pi_{\mathsf{OtF\text{-}MPC}}$ in $\mathbf{REAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\Pi^{\mathsf{SE\text{-}1P}}_{\mathsf{OtF\text{-}MPC}},\mathcal{A},\mathcal{Z}}$. On the other hand, if the message originates from the server (beginning with $\{\texttt{"eval"} : \cdot\}$), the embedded message $\hat{m}$ is considered valid only if $\hat{m} = f(\{m_j\}_{j\in[\ell]})$, where $\{m_j\}_{j\in[\ell]}$ are the input messages referenced in the message.
In $\Pi_{\mathsf{OtF\text{-}MPC}}$, this process is divided into two sub-phases. When $\mathcal{P}_{\mathsf{cli}}$ is activated with the sub-input $\langle\texttt{Share} : \mathsf{grp}, \mathsf{sid}, \mathsf{uid}\rangle$, it first retrieves the corresponding message from $\mathcal{F}_{\mathsf{AUTH}}$ and parses it as

$$\left\{\texttt{"eval"} : \mathsf{srv}, \hat{\mathsf{grp}}, \mathsf{sid}, \mathsf{uid}, (\hat{c}, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)_{j\in[\ell]}, \pi\right\}.$$

If π are generated honestly by the server, they will be accepted. Otherwise, by the definition of $\mathcal{F}_{\mathsf{zkSNARK}}$, we can extract a circuits f such that

$$\hat{c} = \mathsf{MGHE}.\mathtt{Eval}(f, (\mathsf{jpk}_j, c_j)_{j\in[\ell]}).$$

If verification of π fails, the client aborts the execution. Also recall from the earlier definition of $\mathcal{S}$ for a corrupted server that, when $\hat{c}$ is an incorrect evaluation, we let $\mathcal{S}$ write $\hat{m} = \perp$ to $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$, causing the protocol to abort during output phase. This results in consistent abortion conditions in both the ideal and real worlds.

The simulation of interaction between $\mathcal{A}$ and $\mathcal{F}_{\mathsf{MGHE}}$ follows the proof of Theorem 3. Now, upon receiving the decryption shares d from $\mathcal{A}$, then $\mathcal{S}$ invokes $(\pi, \mathsf{pg}) \leftarrow \mathtt{Sim}_{zk}{}^{\mathcal{G}_{\mathsf{RO}}[\mathsf{sid}]}$ to generate an argument π for the relation R_{Dec} and programs $\mathcal{G}_{\mathsf{RO}}$ with pg. Then $\mathcal{S}$ constructs the message

$$\{\texttt{"sh"} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, (c, d, \pi)\}$$

and delivers it to a party in the name of $\mathcal{F}_{\mathsf{AUTH}}$ when a party sends input to $\mathcal{F}_{\mathsf{AUTH}}$ to retrieve it.

Combing of decryption shares similarly follows proof of Theorem 3. Now, by the definition of $\mathcal{F}_{\mathsf{zkSNARK}}$, if the arguments π are generated honestly by clients, they will always be accepted. Since we assume that the adversary does not corrupt more clients than allowed by the threshold, an uncorrupted client will possess enough decryption share d, which are honestly generated using the correct secret keys. Consequently, by the definition of $\mathcal{F}_{\mathsf{MGHE}}$, the pair (c, d) will reconstruct to an evaluated message $\hat{m}$, such that

$$\hat{m} = f(\{m_j\}_{j \in [\ell]}),$$

which is later output to $\mathcal{P}_{\mathsf{cli}}$. Notably, in $\mathbf{IDEAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\mathcal{F}_{\mathsf{OtF\text{-}MPC}}, \mathcal{S}, \mathcal{Z}}$, the functionality $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ simply outputs the same evaluated plaintext $\hat{m}$ to an uncorrupted client. Therefore, the outputs in both the ideal and real worlds remain consistent.

□

D.6 Full proof of Theorem 5

Theorem 5. *The protocol $\Pi^{\mathsf{SS\text{-}2P}}_{\mathsf{OtF\text{-}MPC}}$ UC-realizes $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ in $[\mathcal{F}_{\mathsf{MGHE}}, \mathcal{F}_{\mathsf{NIZK}}, \mathcal{F}_{\mathsf{AUTH}}]$-hybrid model in presence of a global key registry $\mathcal{G}_{\mathsf{KRK}}$ and a global random oracle $\mathcal{G}_{\mathsf{RO}}$ against a full malicious adversary that corrupts a group up to the threshold.*

Proof. We only prove for the cases when a client or the server is corrupted. For the uncorrupted clients and server, the proof generally follows the proof of Theorem 4 since an argument is always accepted if it is honestly generated by an uncorrupted party by the completeness of NIZK. To reduce to the soundness of NIZK, we now let the simulator $\mathcal{S}$ attached to $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ simulate the interaction between the adversary (through corrupted parties) and functionalities $\mathcal{F}_{\mathsf{MGHE}}$ and $\mathcal{F}_{\mathsf{NIZK}}$ as follows.

– **Input Phase, Corrupted Client**: In $\mathbf{REAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\Pi^{\mathsf{SS\text{-}2P}}_{\mathsf{OtF\text{-}MPC}}, \mathcal{A}, \mathcal{Z}}$, when $\mathcal{A}$ sends an input to $\mathcal{F}_{\mathsf{AUTH}}$ to register a message (on behalf of the corrupted client), we

let $\mathcal{S}$ parse the message as

$$\{\texttt{"up"} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, (c_1, c_2, \pi)\}\,.$$

Then $\mathcal{S}$ stores this message in internally for later use.
Next, $\mathcal{S}$ sends to $\mathcal{A}$ in name of $\mathcal{F}_{\mathsf{NIZK}}$ to obtain m as witness. Then $\mathcal{S}$ checks in its record if $\mathcal{A}$ has sent input to $\mathcal{F}_{\mathsf{MGHE}}$ (through the corrupted client) to encrypt a message m to c_1 and c_2 using the random coins ω_1 and ω_2. If yes, in $\mathbf{IDEAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\mathcal{F}_{\mathsf{OtF\text{-}MPC}},\mathcal{S},\mathcal{Z}}$, the simulator $\mathcal{S}$ then sends the input

$$\{\texttt{"up"} : \mathsf{cli}, \mathsf{grp}, \mathsf{sid}, \mathsf{uid}, m\}\,.$$

to $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ through the corrupted client. In all other cases, including failure to verify the existence of random coins used for encryption, or failure to parse the message, we let $\mathcal{S}$ set $m = \perp$ when sending the input to $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$.

– **Computation Phase, Corrupted Server**: In $\mathbf{REAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\Pi^{\mathsf{SS\text{-}2P}}_{\mathsf{OtF\text{-}MPC}},\mathcal{A},\mathcal{Z}}$, when $\mathcal{A}$ sends an input to $\mathcal{F}_{\mathsf{AUTH}}$ to register a message containing the evaluation of a circuit, we let $\mathcal{S}$ parse it as

$$\left\{\texttt{"eval"} : \mathsf{srv}, \hat{\mathsf{grp}}, \mathsf{sid}, \mathsf{uid}, (\hat{c}_1, \hat{c}_2, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)_{j\in[\ell]}, \pi)\right\}.$$

Then $\mathcal{S}$ stores the message internally for later use.
Similarly, $\mathcal{S}$ checks in its record if $\mathcal{A}$ has sent inputs to $\mathcal{F}_{\mathsf{MGHE}}$ with two random coins σ_1 and σ_2, and a circuit f, such that $\hat{c}_1$ and $\hat{c}_2$ are the results of evaluating f on the respective input ciphertexts using σ_1 and σ_2. If yes, in $\mathbf{IDEAL}^{\mathcal{G}_{\mathsf{KRK}}}_{\mathcal{F}_{\mathsf{OtF\text{-}MPC}},\mathcal{S},\mathcal{Z}}$, the simulator $\mathcal{S}$ then sends the input

$$\left\{\texttt{"eval"} : \mathsf{srv}, \hat{\mathsf{grp}}, \mathsf{sid}, \mathsf{uid}, f, \sqcup, (\mathsf{grp}_j, \mathsf{sid}_j, \mathsf{uid}_j)_{j\in[\ell]}\right\},$$

to $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ to evaluate $\hat{m} = f(\{m_j\}_{j\in[\ell]})$ and record the result $\hat{m}$.
In all other cases, $\mathcal{S}$ sends an input to $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$ to use $\perp$ to directly inject an invalid evaluated result $\hat{m} = \perp$.

We note that after the input and computation phase, if a ciphertext tuple in the real world is invalid, then its respective plaintext in the ideal world is marked with $\perp$ in $\mathcal{F}_{\mathsf{OtF\text{-}MPC}}$, which is the same as in the proof of Theorem 4. Thus the proof for the output phase generally follows the proof of Theorem 4 when we assume that a group is not corrupted over the threshold. □